

Department of Information Engineering and Computer Science

Master's Degree in
Artificial Intelligence Systems

Final Dissertation

Edge-Based Anomaly Detection

Optimizing PatchCore for Industry 4.0

Supervisor
Prof. Farid Melgani

Student
Alessandro Sartore 256152

Academic year 2025/26

Contents

Abstract **4**

1 Introduction and Theoretical Background **6**
1.1 Visual Inspection and Anomaly Detection in Industry 4.0 6
1.2 Evolution of Approaches: From Rule-based to Deep Learning 7
1.3 Edge AI: Hardware Accelerators and Neural Processing Units (NPUs) 7
1.4 Industrial Setup and Operational Constraints On the Production Line 8
1.5 Industrial Case Study and Practical Constraints 9

2 Related Works **12**
2.1 Background and Taxonomies of Anomaly Detection 12
2.1.1 Taxonomy of Learning Paradigms 12
2.1.2 Methodological Paradigms 13
2.1.3 Evaluation Metrics 14
2.2 Industrial Anomaly Detection Methods 15
2.2.1 PaDiM: Patch Distribution Modeling 15
2.2.2 PatchCore: Memory Bank with Coreset Subsampling 16
2.2.3 Reverse Distillation: Teacher-Encoder with Student-Decoder 16
2.2.4 DFM: Deep Feature Matching 16
2.2.5 FastFlow: 2D Normalizing Flows 17
2.2.6 EfficientAD: Lightweight Student-Teacher 17
2.2.7 STFPM: Student-Teacher Feature Pyramid Matching 18
2.2.8 DRAEM: Discriminatively Trained Reconstruction Embedding 18
2.2.9 CFA: Coupled-Hypersphere Feature Adaptation 18
2.2.10 SuperSimpleNet: Unifying Supervision Regimes 19
2.3 Comparative Analysis 19
2.3.1 Experimental Setup 20
2.4 Quantitative Results 20
2.4.1 MVTec AD Benchmark 20
2.4.2 VisA: High-Variance Analysis 21
2.4.3 WTM Cable Factory Dataset (Proprietary) 22
2.4.4 Computational Efficiency and Edge Feasibility 24
2.4.5 Analysis and Selection Justification 25
2.4.6 Limitations of the Analysis 26
2.5 Qualitative Analysis and Defect Localization 26
2.5.1 MVTec AD: The Dilation Effect 27
2.5.2 VisA: High-Variance Backgrounds 28
2.5.3 WTM Cable Dataset: The Industrial Reality 29
2.6 Why Not Transformers, VLMs, and LLMs 29

3 System Architecture **32**
3.1 Graphical User Interface: Streamlit and FastAPI . . . 32
3.1.1 Technology Stack . . . 32
3.1.2 User Workflow . . . 33
3.1.3 Intended User Profile . . . 34
3.2 Docker Containerization and Resource Isolation . . . 34
3.3 DeepX Compilation Pipeline . . . 34
3.3.1 Compilation Workflow . . . 34
3.3.2 Target Hardware . . . 35
3.4 Automatic Threshold Selection: 3-Sigma Integration . . . 35
3.4.1 Automatic Calibration . . . 35
3.4.2 User Override: The k-Sigma Slider . . . 36
3.4.3 Integration with Inference . . . 36

4 PatchCore Architecture and Methodology **37**
4.1 High-Level Architecture Overview . . . 37
4.2 Feature Extraction with Pre-trained Backbone . . . 38
4.3 Memory Bank Construction via Greedy Coreset Subsampling . . . 38
4.4 Anomaly Scoring with k-Nearest Neighbors . . . 39
4.5 Complexity Analysis and Bottleneck Identification . . . 40
4.6 Computational Bottleneck Summary and Motivation . . . 41
4.7 Threshold Selection: The 3-Sigma Method . . . 41
4.7.1 The Absence of Labeled Defective Data . . . 41
4.7.2 Alternative Thresholding Methods . . . 44

5 Architectural Optimizations for Edge Inference **47**
5.1 The Three Optimization Axes . . . 47
5.2 Approximate Nearest Neighbor Search with HNSW . . . 47
5.2.1 From Exact k-NN to Approximate k-NN . . . 48
5.2.2 HNSW Fundamentals . . . 48
5.2.3 Index Construction . . . 49
5.2.4 Query Processing . . . 50
5.2.5 Integration with PatchCore . . . 50
5.2.6 Complexity Analysis and The Recall Trade-off . . . 50
5.3 Memory Footprint Reduction via Hard Prototype Mining . . . 51
5.3.1 Motivation: Coreset Limitations . . . 51
5.3.2 Concept of Hard Prototypes . . . 51
5.3.3 Hard Prototype Mining Algorithm . . . 51
5.3.4 Integration with PatchCore . . . 52
5.3.5 Theoretical Impact on Memory and Complexity . . . 53
5.4 The Knowledge Distillation Paradigm . . . 53
5.4.1 Motivation: Learning Normality Rather Than Storing It . . . 53
5.4.2 Foundations of Reverse Distillation . . . 53
5.4.3 Custom Hybrid Architecture Details . . . 53
5.4.4 Anomaly Detection Mechanism . . . 55
5.4.5 Theoretical Advantage for NPU Compilation . . . 55
5.5 Hardware-Aware Compilation: Operator Fusion . . . 55
5.5.1 The "Memory Wall" Bottleneck . . . 55
5.5.2 Aggressive Fusion Strategy . . . 55
5.6 Summary of Optimization Strategies . . . 56

6 Experimental Results and Edge Evaluation **57**
6.1 Algorithmic Evaluation: HNSW Approximation . . . 57
6.1.1 Final AUROC and Throughput Benchmark . . . 58
6.2 End-to-End Edge Deployment Pipeline . . . 59
6.2.1 Algorithmic vs. Framework Acceleration . . . 59
6.2.2 The DeepX NPU Breakthrough: Hardware Acceleration . . . 59
6.3 Solution 2 Evaluation: Hard Prototype Mining (HPM) . . . 60
6.3.1 Ablation Study: The Boundary Ratio (β) at 5% Compression . . . 60
6.3.2 Compression Trade-off: HPM vs. Classic Coreset . . . 61
6.3.3 End-to-End Edge Deployment of the HPM Pipeline . . . 61
6.4 Solution 3 Evaluation: The Knowledge Distillation Paradigm . . . 62
6.4.1 Framework Optimization and Memory Bank Elimination . . . 62
6.4.2 The NPU Synergy: Speed at the Cost of Precision . . . 63
6.4.3 Conclusion for Solution 3 . . . 63
6.5 Qualitative Analysis of Failure Modes . . . 63
6.5.1 Reverse Distillation: Background Hallucinations and ROI Masking . . . 63
6.5.2 HNSW PatchCore: Sensitivity to Point Anomalies . . . 64
6.6 Kria KV260 Target Hardware Feasibility Study . . . 64
6.6.1 Raw NPU Capacity (`run_model` Profiling) . . . 64
6.6.2 End-to-End Embedded Performance . . . 65
6.7 Global Comparative Analysis and Industrial Selection . . . 66
6.7.1 Meeting the Factory Constraints . . . 66
6.7.2 Final Architectural Selection . . . 66

7 Conclusions and Future Work **67**
7.1 Summary of Contributions . . . 67
7.2 Architectural Recommendations for Industry 4.0 . . . 67
7.2.1 The Primary Choice: Optimized HPM PatchCore . . . 67
7.2.2 The High-Throughput Alternative: Reverse Distillation . . . 68
7.3 Future Developments . . . 68

Bibliography **68**

Abstract

Context and Motivations

In the era of Industry 4.0, visual quality inspection has become a critical component of manufacturing, particularly in sectors enforcing Zero-Defect Manufacturing policies. For WTM, a company specializing in the production of high-performance industrial cables, ensuring the structural and aesthetic integrity of the extruded cable is paramount. Traditional rule-based machine vision systems struggle to adapt to the physiological variations of industrial materials, making Deep Learning the most promising alternative for automated defect detection.

However, deploying Deep Learning in a factory environment introduces a significant architectural challenge: the transition from cloud-based computing to Edge AI. To minimize latency, guarantee data privacy, and maintain operational independence from network connectivity, inference algorithms must be deployed directly on the factory floor using constrained embedded systems. This thesis focuses on the AMD Kria KV260 System-on-Module (SOM) paired with a DeepX DX-M1 Neural Processing Unit (NPU), a low-power, high-efficiency hardware target chosen for the WTM extrusion line.

The fundamental motivation of this research is to bridge the gap between academic anomaly detection models, which are designed with virtually unlimited computational resources, and the strict hardware constraints of real-world industrial embedded systems.

The Problem Statement

A robust standard for unsupervised visual anomaly detection in industrial contexts is PatchCore, which offers an optimal balance between defect sensitivity and computational footprint. By extracting mid-level feature representations from a pre-trained convolutional neural network and storing them in a massive memory bank, PatchCore achieves exceptional defect sensitivity (an Image AUROC of 0.993 on the proprietary industrial dataset) without requiring any defective samples during training.

Despite its algorithmic brilliance, the native PatchCore architecture is fundamentally incompatible with Edge deployment. Its inference phase relies on computing the exact L_2 distance (k-Nearest Neighbors) between the query features and a dense memory bank containing tens of thousands of high-dimensional vectors. When deployed out-of-the-box on the target AMD Kria KV260 hardware, the baseline model achieved an unfeasible throughput of just 0.7 Frames Per Second (FPS) and consumed over 1.5 GB of dynamic RAM. These metrics drastically violate the operational constraints of the WTM production line, which strictly dictate a real-time inference throughput of at least 25 FPS to ensure continuous synchronization with the high-speed process.

Methodology and Optimization Techniques

To dismantle these computational bottlenecks and enable real-time inline inspection, this thesis proposes a systematic, hardware-aware optimization pipeline divided into three main methodological axes:

1. Topological Search Approximation: The first optimization step replaced the exact k-NN distance computation with a Hierarchical Navigable Small World (HNSW) graph. By navigating a multi-layered topological approximation of the feature space, the search complexity was reduced from linear to logarithmic, drastically cutting inference latency while preserving the relative distance ranking required for accurate anomaly scoring.

2. Boundary-Aware Coreset Compression (Hard Prototype Mining): To address the massive physical memory footprint of the index, a novel subsampling technique called Hard Prototype Mining (HPM) was introduced. Unlike standard greedy coreset algorithms that uniformly sample the entire feature manifold, HPM selectively targets the "hard" prototypes near the normal-abnormal

decision boundary. This intelligent feature selection allowed the memory bank to be compressed to just 5% of its original size while strictly dominating classic subsampling methods in terms of retained accuracy.

3. Hardware Compilation and Execution Paradigms: The final optimization axis involved shifting the execution from the host ARM CPU to the dedicated DeepX DX-M1 NPU. This was achieved by quantizing the neural network backbone to 8-bit integers (INT8) and utilizing the DeepX compiler's Operator Fusion capabilities to minimize internal SRAM read/write cycles. Simultaneously, an entirely different architectural paradigm, Patchcore with Reverse Distillation, was evaluated. This approach completely eliminates the memory bank in favor of a student-teacher reconstruction network, mapping the entire anomaly detection workload into pure feedforward neural operations perfectly suited for the NPU.

Achieved Results

The application of these optimization techniques successfully transformed a theoretically brilliant algorithm into a practically deployable industrial application. The extensive end-to-end evaluation on the physical Kria KV260 hardware yielded two optimal solutions:

- **The Industrial Choice (Optimized HPM PatchCore):** The synergistic integration of the HPM algorithm ($\beta = 0.75$), HNSW graph search, and NPU INT8 operator fusion resulted in an optimal throughput of **24.81 FPS** with a peak dynamic memory consumption of just **125 MB**. Most importantly, this massive 31× speedup over the baseline was achieved with negligible loss in defect sensitivity, maintaining a robust Image AUROC of **0.9793** (compared to the 0.981 baseline). This architecture was selected as the primary deployment model for WTM, as it comfortably satisfies the strict timing constraints without compromising the Zero-Defect policy.
- **The High-Speed Alternative (Reverse Distillation):** By shifting the entire computational workload to the DeepX accelerator, the Reverse Distillation paradigm achieved a staggering **66.2 FPS** while consuming only **55 MB** of RAM. While this aggressive approach induced a slight drop in accuracy (Image AUROC of **0.968**) due to the inherent sensitivity of decoder networks to INT8 quantization, it stands as the ultimate solution for manufacturing scenarios prioritizing extreme line speed and power efficiency.

In conclusion, this thesis successfully demonstrates that deploying Artificial Intelligence on the Edge requires a deep, interconnected co-design between algorithmic structures and physical hardware. By intelligently addressing both memory and search complexities, the resulting system transitions state-of-the-art defect detection from the laboratory to the factory floor.

1 Introduction and Theoretical Background

This chapter presents the technological and scientific background of this thesis work. It analyzes the paradigms of modern visual inspection within Industry 4.0, the evolution of Computer Vision algorithms toward Deep Learning, and emerging hardware architectures for Edge Computing, highlighting the stringent constraints imposed by manufacturing production environments.

1.1 Visual Inspection and Anomaly Detection in Industry 4.0

In the era of Industry 4.0, the automation of quality control represents an enabling factor for maximizing production efficiency, reducing waste, and guaranteeing safety standards [40, 64]. The integration of cyber-physical systems and Internet of Things technologies has transformed traditional production lines into interconnected ecosystems, where data collection and analysis occur in real time. In this paradigm, quality control is no longer relegated to an offline final testing phase, often conducted on a sampling basis, but is integrated directly inline to prevent the propagation of defective components along subsequent links of the value chain [26]. Automated Optical Inspection has progressively replaced manual human inspection. The latter, in fact, is structurally inadequate to sustain the rhythms of modern production and is physiologically subject to lapses in attention, visual fatigue, and marked evaluative subjectivity across different operators. These human factors introduce unacceptable variability in scrap rate metrics, leading to the risk of releasing non-compliant products to the market, with resulting reputational damage, or discarding perfectly valid parts, leading to direct economic loss [30].

Although classical Machine Learning favors strongly supervised approaches, where models are trained on large and perfectly balanced datasets of positive and negative classes, the direct application of this scheme in manufacturing contexts proves impractical in reality [50]. The main systemic obstacle is the severe data imbalance intrinsic to optimized industrial processes. Modern Zero-Defect Manufacturing policies ensure that the vast majority of produced parts, often exceeding 99.9%, are conforming [48]. Conversely, defective samples constitute rare events and exhibit highly unpredictable morphological forms. It is economically, if not logically, impossible to collect an exhaustive dataset covering every single potential variant of anomaly: from superficial micro-scratches to geometric deformations, up to chromatic alterations due to oxidation processes, mechanical friction, or contamination by lubricating fluids. Attempting to manually catalog and annotate at the pixel level all these deviations would entail an unmanageable combinatorial explosion. Furthermore, a supervised classifier trained on a limited set of known defects would be extremely fragile and lacks generalization capabilities when facing defect types never encountered during the training phase [77].

To overcome this structural obstacle, industrial research has made a decisive shift, focusing on Unsupervised or Semi-Supervised Anomaly Detection paradigms [63, 4]. This approach reverses the classical logic: the neural model is trained exclusively on a pool of "good" samples, easily obtainable from normal production runs and requiring no costly human annotation. The mathematical objective of the algorithm thus becomes to learn the statistical distribution of "normality", mapping input data into a low-dimensional latent space that exclusively represents the characteristic features of the conforming product. During inference, each new frame acquired by the industrial camera is compared against the learned normality model. Any spatial, textural, or morphological deviation that lies outside the expected confidence margin is identified as an Out-of-Distribution sample and generates an anomaly signal [9]. It is fundamental to note that state-of-the-art Anomaly Detection systems are not limited to providing a global binary response for the entire image, but generate dense error heatmaps capable of localizing and segmenting the defect at the single-pixel level. This high level of

spatial granularity proves crucial in industrial environments, as it enables line technicians to perform rapid root-cause analysis and promptly recalibrate upstream machinery, closing the Quality 4.0 control loop.

1.2 Evolution of Approaches: From Rule-based to Deep Learning

Historically, the development of industrial visual inspection systems has been dominated by rule-based approaches, or, classical image processing algorithms. In this paradigm, defect detection logic relies on a manual feature engineering process [61, 9]. Historical techniques such as adaptive thresholding, morphological contour extraction via differential filters (e.g., Sobel, Canny Edge Detector), spatial frequency analysis (e.g., Gabor filters), and geometric Template Matching, require an exhaustive design and calibration phase by domain experts [61]. The intrinsic limitation of such systems lies in their geometric and photometric rigidity. Although they are computationally lightweight, inherently interpretable, and easy to implement on low-power hardware, they are extremely fragile in dynamic environments. The production line is not an optical laboratory: slight physiological variations in LED illuminator intensity (e.g., due to thermal decay of the diode) [32], micro-angular misalignments of the part on the conveyor belt [47], or minimal variations in surface texture of the background, cause the pre-set optimal thresholds to shift [30]. This inevitably translates into a drastic and sudden increase in false positives or, worse, false negatives, forcing engineers to continuously manually recalibrate the system [59, 39].

An initial evolution toward more flexible systems came with the introduction of traditional Machine Learning methods (e.g., Support Vector Machines, Random Forests), coupled with local feature descriptors robust to scale and rotation, such as SIFT [37] or SURF [7]. However, these models still inherited the limitation of the semantic gap [58]: the algorithm did not understand the image, but only a limited set of pre-extracted hand-crafted metrics [33]. The advent of Deep Learning [33], and in particular, Convolutional Neural Networks (CNNs) [31], marked a theoretical break, shifting the paradigm from Feature Engineering to Representation Learning [8]. Instead of requiring a human operator to define heuristic rules to identify a scratch or a fold [67], convolutional models automatically extract hierarchies of increasingly complex and abstract features directly from raw pixels. Early network layers act as universal filters for edges and primary patterns [70], while deeper layers capture the global semantics of the object [76], guaranteeing high translational invariance and robustness to background disturbances.

Furthermore, the massive adoption of Transfer Learning [78] has eliminated the need to train neural architectures from scratch. Very deep classification models (such as ResNet [24] or WideResNet [73]), pre-trained on large generalist datasets containing millions of images (e.g., ImageNet [19]), are now employed as universal, high-quality feature extractors. This technological evolution has enabled unprecedented accuracy, making systems virtually immune to factory environmental fluctuations and paving the way for complex statistical and memory-based models for state-of-the-art (SOTA) Anomaly Detection, such as PatchCore [51] and PaDiM [17].

1.3 Edge AI: Hardware Accelerators and Neural Processing Units (NPUs)

Despite the extraordinary extraction capabilities and generalist robustness of Deep Learning, the computational cost associated with deep neural network inference strongly conflicts with the strict requirements of industrial automation [46]. Historically, the computational burden, on the order of tens of Giga-FLOPs per single frame (e.g., ~ 4 GFLOPs for a baseline ResNet-50 and exceeding 15 GFLOPs for heavier architectures), has relegated the execution of these models to powerful centralized Cloud servers or the use of expensive Data Center-derived graphics cards (GPUs) [13]. However, outsourcing logical inference to remote servers is incompatible with modern factory architectures [46]. The continuous transmission of high-resolution uncompressed video streams would quickly saturate the network infrastructure's bandwidth [56]. In addition to raising critical issues related to cybersecurity and data privacy, the cloud-based paradigm introduces non-deterministic network latencies, which are unacceptable on a production line operating under strict real-time constraints where deterministic

response times must remain below a specific millisecond threshold (typically < 50 ms) [46].

The solution lies in the Edge Computing paradigm, or Edge AI: native execution of Artificial Intelligence models directly at the "edge" of the network, i.e., onboard the machine or on the Industrial PC interfaced with the camera [56, 13]. For local processing to be sustainable without employing high-power-consumption GPU workstations, often incompatible with thermal specifications or industrial dust-rich environments that prevent active cooling [46], the semiconductor industry has introduced a new class of accelerators: Neural Processing Units (NPUs) [13]. To understand the advantage of NPUs, it is necessary to analyze the bottlenecks of traditional processors:

- **CPU (Central Processing Unit):** Based on the Von Neumann architecture, CPUs are optimized for the execution of complex sequential logic, branching, and operating system management. The continuous transfer of weights and activations from DRAM to cache renders them ineffective and slow for the massive matrix multiplications required by neural networks [13].
- **GPU (Graphics Processing Unit):** Parallel-architecture processors excel in heavy parallel computation for training and complex inference, but their general-purpose design entails a thermal dissipation ranging from 50 to 300 Watts, very high integration costs, and the need for dedicated power buses [13, 46].

Unlike CPUs and GPUs, NPUs are Application-Specific Integrated Circuits (ASICs). They are structurally designed to exclusively accelerate a single arithmetic operation: the Multiply-Accumulate (MAC), the true computational engine of each convolutional layer. Through the use of systolic multiplier arrays and ultra-fast on-chip SRAM memory allocation to minimize expensive accesses to external memory, NPUs can process hundreds of tensors in a single clock cycle, guaranteeing energy efficiency ratios 10 to 50 times higher than traditional GPUs [13].

However, this significant leap in efficiency and Frames Per Second (FPS) imposes a severe mathematical compromise [13]. To maximize data throughput on the internal bus and double register density, industrial NPUs invariably require the neural model to undergo a Post-Training Quantization (PTQ) process. Tensors (weights, biases, and activations) originally expressed in 32-bit floating point (Float32), characterized by very high dynamics and precision, are remapped and compressed into a discrete space of 8-bit integers (INT8), limited to values between -128 and 127 [21]. This arithmetic truncation operation, while being the key to edge acceleration, introduces critical engineering challenges. In the presence of photometric outliers (e.g., specular reflections on metal surfaces), the quantization scale risks allocating most of the limited 8-bit resolution to encoding noise. This flattens the subtle and crucial feature variations that distinguish a conforming part from a defective one, thereby compromising the diagnostic accuracy of the implemented SOTA model [21].

1.4 Industrial Setup and Operational Constraints On the Production Line

The transition from a development environment to the deployment of an Anomaly Detection solution in a real industrial setup goes far beyond mere algorithmic calculation or the maximization of accuracy metrics (e.g., AUROC) [46]. A production line for critical components with continuous topology, such as the extrusion, drawing, and intertwining of metal cables, operates at uninterrupted high-speed flow. This kinematics imposes extremely stringent operational, temporal, and architectural constraints on the cyber-physical vision system:

- **Deterministic Latency and Throughput:** In an in-line context, the algorithm must not only process frames at a framerate compatible with the belt speed (typically ≥ 25 FPS, i.e., adequate throughput), but must also guarantee rigidly deterministic inference latency. If the belt advances at speed v and the physical distance between the inspection camera and the pneumatic reject actuator is d, the maximum time available to the system to acquire the image, pre-process it, infer the anomaly, and communicate the signal (OK/KO) to the PLC is strictly limited to $T_{max} = \frac{d}{v}$. Missing this stringent time window results in the defect passing beyond the ejection station, nullifying the entire purpose of quality control regardless of the AI model's accuracy [46].

- **Optical Robustness and Sensor Dynamics:** Unlike standardized datasets, in-line acquisition is subject to strong optical disturbances. The use of standard lenses instead of more expensive telecentric optics introduces parallax errors and perspective variations due to physiological vibrations of the cable on the Z axis. Furthermore, reflective metal surfaces, when subjected to high-intensity industrial illuminators which are necessary to compensate for short exposure times (typically ≈ 1000 μs), do not behave as Lambertian surfaces but generate specular reflections. These luminous hotspots cause local saturation of the CMOS sensor photodiodes, severely limiting the dynamic range and forcing the implementation of robust and invariant preprocessing architectures upstream of the quantized neural network [60].

- **Concurrent Management of Host (CPU) Resources:** The typical architecture of a vision station includes an Industrial PC installed onboard the line. It is fundamental to note that this host processor is not dedicated solely to visual inspection. The operating system must orchestrate in parallel a multiplicity of critical processes: synchronous frame acquisition via GigE Vision or USB3 Vision protocols, deterministic real-time communication with PLCs via field buses (e.g., Profinet, EtherCAT, or Modbus TCP), updating the human-machine interface, and asynchronous storage of statistical data in the database [46].

Exclusive use of the host CPU to execute the heavy tensor calculations required by Deep Learning would inevitably lead to saturation of computational cores and unexpected cache memory fluctuations. In the industrial domain, this phenomenon introduces strong temporal jitter in network communications. If the CPU, busy computing neural model convolutions, delays sending the signal to the PLC beyond the watchdog timer threshold, the entire machinery instantly enters a safety alarm state, halting the entire factory production [46]. In this delicate ecosystem, the NPU accelerator assumes a fundamentally strategic importance. It does not merely function as a tool to increase FPS, but acts primarily as an asynchronous hardware offloader. By delegating massive tensor computation to the NPU's systolic matrix, the CPU is relieved of the heavy computational load and limited to orchestration tasks (I/O management and lightweight statistical post-processing). This hybrid workload partitioning is the key to guaranteeing the reliability, safety, and uninterrupted uptime of the entire industrial cycle [13].

1.5 Industrial Case Study and Practical Constraints

To ground the previously discussed constraints in a concrete industrial scenario, this thesis considers a real-world inspection setup for the quality control of continuous high-performance cables at the WTM production facility. The primary challenge of this visual inspection task is driven by the extreme material diversity of the manufactured products. WTM production lines handle an extensive portfolio of cable variants, ranging from bare metallic wires (e.g., highly reflective copper and aluminum) to complex multi-core cables encased in diverse polymer sheaths, which can be opaque, vividly colored, or completely transparent.

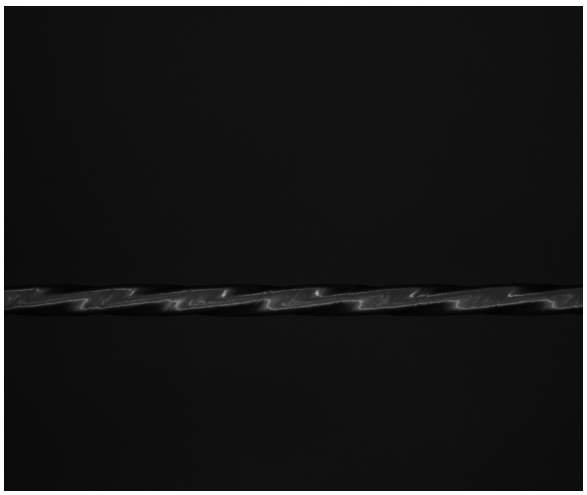
This material heterogeneity directly dictates the complex morphology of the target defects, which primarily fall into two macroscopic categories:

- **Surface Scratches and Abrasions:** Micro-lacerations or gouges occurring during the extrusion or mechanical routing phases. On metallic surfaces, these scratches sharply disrupt the specular highlights; conversely, on transparent or opaque polymers, they manifest as subtle textural shadows or structural indentations that are extremely difficult to isolate from the background.

- **Taping Phase Errors:** High-performance cables often feature concentrically wrapped protective or shielding layers (such as metallic foils, PTFE, or mica tapes). Taping anomalies include incorrect overlap ratios (where the tape pitch is too wide, exposing the core, or too narrow), missing tape segments, localized folds, and wrinkled edges that compromise the cable's electrical or mechanical insulation.

To provide a concrete visual reference of these phenomena, Figure 1.1 illustrates the direct comparison between a conforming cable, which serves as our expected morphological baseline, and typical

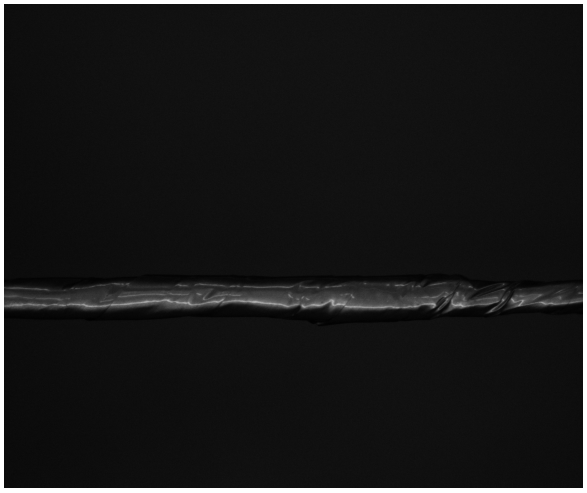
defects encountered during the actual taping phase. As previously discussed, these anomalies disrupt the structural regularity of the cylinder, posing a significant challenge for automated detection algorithms due to the highly reflective nature of the dark background.



(a) Golden Standard (Defect-free)



(b) Severe Tape Gap / Tear



(c) Tape Wrinkle / Overlap Error



(d) Morphological Distortion

Figure 1.1: Visual inspection of WTM high-performance cables. Image (a) represents the target topological regularity of a conforming sample. Images (b), (c), and (d) demonstrate diverse unpredictable anomalies occurring during the manufacturing process, highlighting the severe contrast challenges and structural deformations the Anomaly Detection model must overcome.

A typical vision system configuration includes a high-resolution industrial camera, specifically a **LUCID Triton 5MP polarized camera** (featuring a 2448×2048 pixel resolution, 3.45 μm pixel size, and GigE Vision interface), equipped with a synchronized high-intensity illumination system to ensure sufficient contrast under high-speed motion conditions [60]. The processing unit is commonly an Industrial PC operating without dedicated GPU acceleration, which imposes additional constraints on the feasibility of deep learning-based solutions in real time [46]. Within this context, modern deep anomaly detection models must be carefully adapted to satisfy strict computational and latency requirements. In particular, when state-of-the-art methods such as PatchCore [51] are deployed on embedded or edge-oriented hardware platforms through Post-Training Quantization (FP32 $\rightarrow$ INT8) [21], a non-negligible degradation in inference throughput and potential drops in the Area Under the Receiver Operating Characteristic (AUROC) metric may occur, making real-time processing challenging under production line constraints. This motivates the need for optimized pipelines and architectural adaptations capable of maintaining detection performance while respecting the deterministic timing requirements imposed by industrial environments [13, 46].

The primary objective of this thesis work is to bridge this gap by designing and evaluating an optimized edge-oriented Anomaly Detection pipeline. The target pipeline aims to achieve a deterministic throughput matching or exceeding the line requirement ($\geq$ 25 FPS) while keeping the quantization-induced degradation bounded ($\Delta_{\text{AUROC}} \leq X\%$). By offloading tensor convolutions to the dedicated edge NPU, the proposed architecture minimizes CPU jitter, ensuring that total latency remains strictly within the deterministic window (T_{max}) allowed by the physical kinematics of the production line.

2 Related Works

This chapter summarizes the most relevant papers on the topic of industrial anomaly detection, focusing on the recent developments based on deep learning. The goal is to provide a structured overview of the available methods, their underlying assumptions, their advantages and disadvantages, in the context of actual inspection systems in industry.

Particular focus is given to unsupervised and semi-supervised techniques, which are now the de facto standard techniques in manufacturing scenarios, where annotated defect information is limited. Some representative state-of-the-art architectures are also discussed, especially those that have been either evaluated or related to the contributions of this thesis.

2.1 Background and Taxonomies of Anomaly Detection

It is important to come up with a common ground for understanding the landscape of anomaly detection before the specific algorithms, evaluated and accelerated in this thesis, are presented. The goals of this section are to present the basic taxonomies which define various approaches, the methodological paradigms which guide the present work and the metric used for evaluating performance.

2.1.1 Taxonomy of Learning Paradigms

There are a number of orthogonal dimensions in which anomaly detection methods can be classified. For industrial applications, the most relevant difference is on the type of available training data.

Supervised vs. Unsupervised vs. Semi-Supervised Learning

In a fully supervised setting, the model is trained on a large, balanced dataset containing both normal and anomalous samples, with explicit labels for each class. While this approach yields strong discriminative performance when the anomaly classes are well-defined and comprehensively sampled, it is fundamentally ill-suited for industrial anomaly detection. There are two principal reasons for this incompatibility[16].

First, modern Zero-Defect Manufacturing policies guarantee that defective products are extremely rare, and in fact, less than 0.1% of production is defective. This gives rise to a long tail distribution which makes it impossible or impractical to gather adequate number of defective samples for a powerful supervised classifier. Second, and more basically, the notion of "anomaly" is itself open: An anomaly is anything not normal, and the space of what is possible is infinite. Any supervised classification algorithm trained on a finite set of known defect types will inevitably be blind to new defect types that are not part of the training set[27].

These limitations have driven the industrial anomaly detection community toward **unsupervised** and **semi-supervised** paradigms. In these settings, the model is trained exclusively on defect-free samples, learning a representation of "normality" against which test samples are compared. During inference, any significant deviation from this learned representation is flagged as anomalous. This approach has the decisive advantage of requiring no labeled defect data for training, while remaining sensitive to previously unseen anomaly types[16].

Few-Shot Settings

A related dimension concerns the quantity of training data available. While traditional unsupervised methods assume access to hundreds or thousands of normal images, industrial constraints often limit the available data, particularly during the production ramp-up phase for new product variants. This has motivated the development of **few-shot anomaly detection** methods, which can be further subdivided[43]:

- **Few-Normal-Shot**: There are only a few (usually less than 100) normal samples available to the model. This is typical when a new product line is being introduced and historical information is not available[28].

- **Few-Abnormal-Shot**: Few examples of anomalies are labeled and provided to the model along with the normal data. With a few samples of defect from past production runs or expert annotation, this weakly supervised approach can substantially enhance the robustness of defect detection[43].

The context of the cable factory offers only little training data for each new cable type and is incapable of systematically collecting defective samples, which is the reason why the unsupervised and Few-Normal-Shot regime is dominant in the present thesis.

2.1.2 Methodological Paradigms

Within the unsupervised anomaly detection literature, two dominant methodological paradigms have emerged: reconstruction-based methods and feature-embedding methods. A third category, memory-based methods, can be viewed as a specialization of the feature-embedding approach[35].

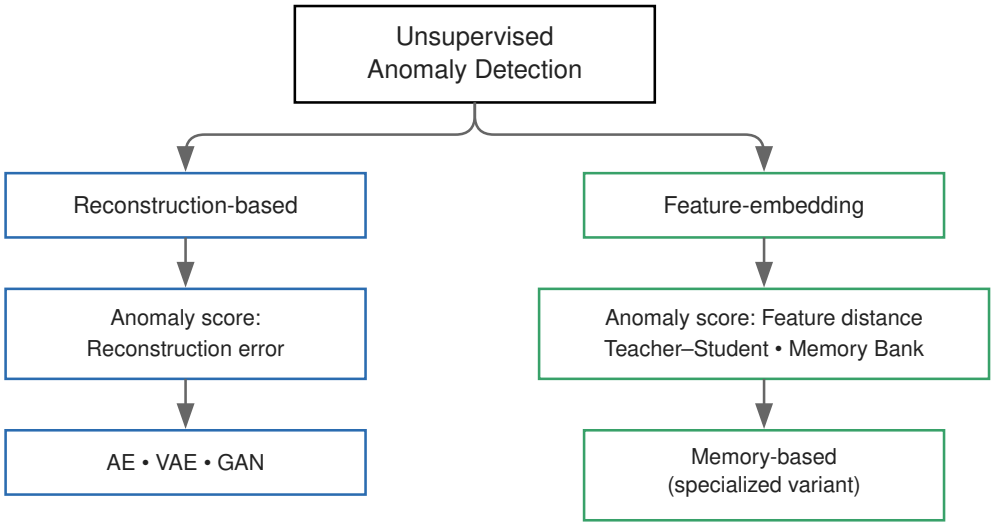


Figure 2.1: Taxonomy of unsupervised anomaly detection methods for industrial image inspection. Reconstruction-based methods detect anomalies through reconstruction error, whereas feature-embedding methods rely on distances in the learned feature space. Memory-based approaches are a specialized class of feature-embedding methods. Adapted from [16] and [35].

Reconstruction-Based Methods

Reconstruction-based methods are founded on a simple premise: a model trained to reconstruct normal samples will successfully reconstruct normal test inputs but will fail to reconstruct anomalous regions, which were absent from the training distribution. The anomaly score is derived from the reconstruction error between the input and its reconstructed output[35].

Early implementations employed **Autoencoders (AEs)**[10], which compress the input through a bottleneck latent space and then reconstruct it. Variants include **Variational Autoencoders (VAEs)**[2], which impose a probabilistic structure on the latent space, and **Generative Adversarial Networks (GANs)**[54], which use a discriminator to improve reconstruction fidelity.

The principal advantage of reconstruction-based methods is their natural ability to produce pixel-level anomaly maps: the reconstruction error can be computed at each pixel, providing fine-grained localization of defects. However, they suffer from several limitations[35]. First, modern deep networks are often capable of generalizing to reconstruct anomalous inputs, particularly when the anomaly is

subtle, leading to false negatives. Second, the training process can be unstable, particularly for GAN-based architectures. Third, the inference time is typically higher than competing approaches due to the need to pass each input through the full encoder-decoder pipeline[51].

These limitations have motivated the development of more sophisticated reconstruction-based architectures designed to prevent the model from merely copying the input. Notable examples include **DRAEM**[74], which simulates anomalies during training by pasting synthetic defect patterns onto normal images, and **RIAD**[75], which treats anomaly detection as a masked image inpainting task to forcefully eliminate the identity mapping of anomalous pixels

Feature-Embedding Methods

Feature-embedding methods adopt a fundamentally different strategy. Instead of reconstructing the input at the pixel level, these methods leverage deep neural networks, typically pre-trained on large-scale datasets such as ImageNet, as fixed feature extractors. The key insight is that these pre-trained networks produce rich, semantically meaningful representations that capture both low-level textures and high-level structures[35].

During training, feature-embedding methods construct a model of the feature distribution for normal samples. During inference, test images are passed through the same feature extractor, and the distance between test features and the learned normal distribution determines the anomaly score.

Within this paradigm, several subcategories have emerged[16]:

Teacher-Student Architectures A pre-trained teacher network extracts features from normal images. A smaller student network is trained to regress to the teacher's output on normal data. The key assumption is that the student, having only seen normal samples, will fail to generalize to anomalous inputs, producing feature deviations that signal anomalies. Examples include **STFPM**[66], **EfficientAD**[6], and **Reverse Distillation**[18], which innovatively inverts the traditional feature flow to prevent the student from merely copying the teacher.

Distribution Modeling Approaches Instead of training a student network, these methods explicitly model the distribution of normal features using parametric or non-parametric density estimators. **PaDiM**[17] models the feature distribution at each spatial location as a multivariate Gaussian, using the Mahalanobis distance as the anomaly score. **FastFlow**[72] employs normalizing flows to map the complex feature distribution to a simple Gaussian base distribution, enabling exact likelihood estimation.

Memory Bank Approaches These methods store a representative subset of normal features directly: the memory bank. During inference, test features are compared against the memory bank using nearest-neighbor search. **PatchCore**[51], the baseline algorithm for this thesis, belongs to this category. Its key innovation is the use of greedy coreset sampling to reduce the memory bank size while preserving the diversity of normal features.

The principal advantage of feature-embedding methods is their inference speed: once the feature extractor is computed (typically 20-50ms on modern hardware), the anomaly score requires only distance computations, not a full reconstruction pass. Their primary limitation is the reduced spatial resolution of deep features, which can make precise pixel-level localization challenging compared to reconstruction-based methods[74].

2.1.3 Evaluation Metrics

Rigorous evaluation of anomaly detection methods requires metrics that capture both image-level classification performance and pixel-level localization accuracy.

Image-Level Metrics

At the image level, the task is binary classification: does the image contain an anomaly? The standard metric is the **Area Under the Receiver Operating Characteristic curve (AUROC)**[20]. The

ROC curve plots the true positive rate against the false positive rate at various threshold settings. AUROC summarizes the model's ability to discriminate between normal and anomalous samples across all thresholds, with 1.0 representing perfect separation and 0.5 representing random guessing.

The **F1-score** is also commonly reported, particularly when a specific operating point (threshold) has been selected[45]. The F1-score is the harmonic mean of precision and recall, providing a single metric that balances false positives and false negatives.

Pixel-Level Metrics

For tasks requiring defect localization, pixel-level metrics evaluate how precisely the model identifies the anomalous region. The **Pixel-Level AUROC** treats each pixel as an independent binary classification problem: is this pixel anomalous or not? While widely used, this metric can be misleading because it weights all pixels equally, including the large number of normal pixels that dominate any image[9].

More sophisticated metrics include the **Area Under the Per-Region Overlap (AUPRO)** curve, which evaluates how well the predicted anomaly region overlaps with the ground truth defect region on a per-defect basis, and the **Per-Image Missed Overlap (PIMO)** metric, which penalizes complete failures to detect a defect[9].

The present thesis reports both image-level and pixel-level AUROC, as well as F1-scores at the optimal threshold determined by the 3-sigma method described in Chapter 4.7. This multi-metric approach ensures a comprehensive characterization of detection and localization performance.

2.2 Industrial Anomaly Detection Methods

The period from 2020–2022 witnessed a proliferation of unsupervised anomaly detection methods specifically designed for industrial visual inspection. Unlike earlier approaches that relied on handcrafted features or shallow classifiers, these methods leverage deep pre-trained feature extractors to achieve unprecedented accuracy on benchmark datasets such as MVTec AD[9]. This section reviews ten representative methods from this period, organized by their underlying architectural paradigm. The comparative analysis that follows in Section 2.3 justifies the selection of PatchCore as the baseline for the acceleration work presented in this thesis.

2.2.1 PaDiM: Patch Distribution Modeling

PaDiM [17] was introduced by Defard et al. at ICPR 2020 and represents a foundational probabilistic approach to anomaly detection. The core idea is to model the distribution of normal feature representations using multivariate Gaussian distributions, where the Mahalanobis distance is employed as the anomaly scoring function.

The method operates by extracting feature maps from multiple layers of a pre-trained CNN, typically a ResNet, capturing information at different semantic levels. For each spatial location, feature vectors from the selected layers are concatenated to form a rich multi-scale representation. During training, which relies exclusively on normal images, PaDiM estimates a multivariate Gaussian distribution for every spatial position, parameterized by a mean vector μ_{ij} and a covariance matrix Σ_{ij}.

During inference, the Mahalanobis distance between the feature vector of a test patch and the corresponding learned Gaussian distribution is computed. This distance serves as an anomaly score, quantifying the degree of deviation from the normal data distribution at each spatial location.

The main contribution of PaDiM lies in its ability to exploit correlations across different semantic levels of the CNN. By combining low-level texture information, which facilitates accurate localization, with high-level semantic representations, which enhance robustness to appearance variations, the method achieves strong performance in both anomaly detection and localization tasks [17].

Despite its effectiveness, PaDiM presents a significant limitation in terms of computational efficiency. Specifically, the need to estimate, store, and invert a covariance matrix for every spatial location leads to substantial memory requirements and computational overhead, making the approach less suitable for high-resolution industrial inspection scenarios [51].

2.2.2 PatchCore: Memory Bank with Coreset Subsampling

PatchCore [51], introduced by Roth et al. at CVPR 2022, serves as the baseline architecture for this thesis. The method follows a memory bank paradigm: during training, patch-level features are extracted from normal images using a pre-trained WideResNet backbone, and a representative subset of these features is stored in a memory bank. During inference, the anomaly score of each test patch is computed as the distance to its nearest neighbor within the memory bank.

The effectiveness of PatchCore is largely attributed to two key innovations:

1. **Multi-scale feature aggregation**: feature representations are extracted from multiple network layers, typically `layer2` and `layer3`, and subsequently aggregated to capture both fine-grained texture information and higher-level structural patterns.

2. **Greedy coreset subsampling**: rather than storing all extracted feature vectors, PatchCore selects a compact subset that preserves the diversity of the normal feature distribution as much as possible. This strategy reduces memory consumption by approximately 90–99% while maintaining comparable detection performance.

Anomaly detection and localization are carried out through a k-Nearest Neighbors (k-NN) search between the test features and the memory bank. The resulting patch-level distances are used to generate anomaly maps, while the image-level anomaly score is obtained from the maximum patch-level distance [51].

Although PatchCore achieves state-of-the-art performance on standard industrial anomaly detection benchmarks, it exhibits several limitations. In particular, inference efficiency is strongly influenced by the size of the memory bank, and the reliance on ImageNet pre-training may reduce its effectiveness when applied to domains characterized by visual patterns that differ substantially from those encountered during pre-training [16].

2.2.3 Reverse Distillation: Teacher-Encoder with Student-Decoder

Reverse Distillation [18], proposed by Deng and Li at CVPR 2022, addresses a fundamental limitation of traditional knowledge distillation approaches. In conventional teacher–student architectures, both networks are typically encoders with similar structures, which reduces the diversity of anomalous representations.

Reverse Distillation inverts this relationship. The architecture consists of:

- A pre-trained **teacher encoder** that extracts multi-scale feature representations from input images.

- A **trainable student decoder** that receives the teacher's one-class embedding as input and attempts to reconstruct the teacher's multi-scale feature maps.

Unlike conventional student networks, the decoder does not process raw images directly. Instead, it learns to *reverse distill* information from a compact latent embedding back into detailed feature representations. To facilitate this process, a trainable One-Class Bottleneck Embedding (OCBE) module compresses normal patterns into a compact representation while suppressing anomaly-related perturbations. During inference, anomalies are identified by measuring the discrepancy between the feature representations generated by the teacher encoder and those reconstructed by the student decoder [18].

However, unlike memory-based or probabilistic approaches, Reverse Distillation requires a dedicated training phase to optimize the student decoder. Consequently, the method introduces additional optimization complexity and longer training times compared with training-free anomaly detection techniques [35].

2.2.4 DFM: Deep Feature Matching

Deep Feature Matching (DFM) represents a simpler family of anomaly detection methods that operate directly on pre-trained deep feature representations without relying on complex distributional modeling[23]. In its canonical formulation, often exemplified by baseline architectures such as SPADE

[14], DFM extracts features from a pre-trained CNN (typically from the final convolutional layer or a selected intermediate layer) and compares test-time features with a reference set of normal features using standard distance metrics, such as Euclidean distance or cosine similarity.

Although less sophisticated than approaches such as PaDiM or PatchCore, DFM-based methods provide several advantages: minimal training requirements, as they generally require no optimization beyond feature extraction; extremely low inference latency, since anomaly detection is performed through direct distance computation; and high interpretability due to the explicit comparison between normal and test feature representations. These properties have made DFM a widely adopted baseline for evaluating anomaly detection methods, as discussed in the following sections [14].

However, because DFM relies on simple distance-based comparisons without explicitly modeling spatial relationships or incorporating advanced feature aggregation strategies, it typically achieves lower localization accuracy when dealing with subtle, complex, or highly structured defects compared with more advanced anomaly detection approaches [16].

2.2.5 FastFlow: 2D Normalizing Flows

FastFlow [72], introduced by Yu et al., addresses a fundamental limitation of existing feature-embedding-based anomaly detection methods: the difficulty of accurately estimating the probability density of visual features in high-dimensional spaces. Traditional approaches typically rely on simplified parametric assumptions (e.g., Gaussian distributions) or non-parametric estimators (e.g., kernel density estimation), which may fail to capture the complex underlying structure of feature distributions.

FastFlow employs **2D normalizing flows** as a flexible probability density estimator. Normalizing flows are deep generative models that learn an invertible transformation between a complex data distribution and a simple base distribution, typically an isotropic Gaussian distribution. A key advantage of normalizing flows is their ability to perform exact likelihood estimation, as the probability density of an input sample can be analytically computed using the change-of-variables formula.

During training, FastFlow learns to transform normal image features into a tractable latent distribution. During inference, features extracted from test images are passed through the learned flow model, and the resulting log-likelihood under the base distribution is used as the anomaly score. High likelihood values indicate that the sample is consistent with normal patterns, whereas low likelihood values suggest anomalous behavior [72]. The framework is compatible with various feature extractors, including convolutional neural networks such as ResNet and transformer-based architectures.

Despite its theoretical advantages and strong probabilistic formulation, FastFlow introduces additional computational complexity during the training stage. Normalizing flow models are computationally expensive to optimize and require considerable GPU memory resources, resulting in longer and more demanding training procedures compared with distance-based approaches such as PaDiM or PatchCore [16].

2.2.6 EfficientAD: Lightweight Student-Teacher

EfficientAD [6] was specifically designed for edge deployment scenarios, with the objective of achieving millisecond-level inference latency while preserving competitive anomaly detection accuracy. The method combines a lightweight student-teacher architecture with an efficient patch embedding mechanism.

Two main innovations distinguish EfficientAD from previous student-teacher approaches. First, the student network is designed with substantially lower complexity than the teacher network, enabling fast inference while maintaining sufficient sensitivity to anomalous patterns. Second, EfficientAD introduces a novel **logistic-distortion loss** that prevents the student network from accurately reconstructing anomalous regions that are not present during training. This mechanism mitigates a common limitation of student-teacher methods, where excessive generalization capability may cause the student to reproduce anomalous features.

EfficientAD is engineered for extreme inference speed, leveraging a highly streamlined student-teacher architecture that prioritizes minimal computational overhead. By design, it achieves rapid, real-time performance on standard CPU hardware, positioning it among the most efficient anomaly detection approaches currently available. However, as discussed in the comparative analysis (Section

2.3), this computational efficiency introduces a trade-off in detection sensitivity. The limited representational capacity of its lightweight student network may result in diminished performance when dealing with highly complex, fine-grained, or subtle defect patterns compared with more memory-intensive approaches such as PatchCore [6].

2.2.7 STFPM: Student-Teacher Feature Pyramid Matching

STFPM [66], introduced by Wang et al., extends the student–teacher framework to multiple spatial resolutions through a feature pyramid architecture. A fixed teacher network, pre-trained on ImageNet, extracts feature representations at different pyramid levels. A trainable student network, initialized with the same architecture, is optimized to reproduce the teacher's feature maps at each corresponding scale.

During training using only normal images, the student network learns to approximate the teacher's multi-scale feature representations. At inference time, the anomaly score is obtained by measuring the discrepancy between teacher and student features across all pyramid levels, which is then aggregated into a dense anomaly heatmap. The multi-scale design of STFPM provides effective localization capabilities: lower-level features preserve fine-grained texture information, making them suitable for detecting small surface defects such as scratches, while higher-level features encode semantic and structural information useful for identifying global shape deformations [66].

However, a major limitation of STFPM is the potential over-generalization of the student network. Since the student has a high representational capacity and shares the same architecture as the teacher, it may learn to reproduce teacher features even for anomalous inputs, resulting in missed detections. This structural weakness motivated the development of more constrained student architectures and additional regularization mechanisms, such as the penalty-based strategies introduced in EfficientAD [6].

2.2.8 DRAEM: Discriminatively Trained Reconstruction Embedding

DRAEM [74], proposed by Zavrtanik et al. at ICCV 2021, introduces a fundamentally different perspective compared with previous anomaly detection approaches. Instead of relying exclusively on reconstruction errors or feature-space distances, DRAEM formulates anomaly detection as a discriminative learning problem.

The method introduces a **synthetic anomaly generation strategy**: during training, artificial anomaly patterns are created by applying random Perlin noise-based textures onto normal images. This process generates a large collection of paired normal and synthetic anomalous samples without requiring manually annotated real defect images. Using these samples, DRAEM jointly trains two components: a reconstruction network that transforms anomalous inputs into anomaly-free images, and a discriminative embedding network that learns a decision boundary between normal and anomalous feature representations.

Unlike conventional reconstruction-based approaches, DRAEM's discriminative module can be trained in an end-to-end manner, avoiding the need for heuristic post-processing steps when generating anomaly maps. On some benchmarks, DRAEM achieves significant improvements over previous unsupervised anomaly detection methods and reaches performance levels comparable to some fully supervised approaches [74].

Nevertheless, the effectiveness of DRAEM is strongly influenced by the quality and diversity of the synthetic anomaly generation process. When real-world defects differ substantially from the generated Perlin noise patterns, such as in cases involving subtle structural missing components or complex manufacturing failures, the learned discriminative boundary may become sub-optimal, increasing the probability of false negatives for unseen anomaly types [16].

2.2.9 CFA: Coupled-Hypersphere Feature Adaptation

CFA [34], implemented in the Anomalib framework [1], introduces a geometric perspective for adapting deep feature representations. The main idea is to learn a transformation from pre-trained feature embeddings into a target-oriented feature space structured around coupled hyperspheres, where normal patterns are explicitly organized within a compact region.

The CFA architecture consists of:

- A backbone CNN feature extractor, typically based on WideResNet50, which extracts high-level visual representations.
- A descriptor network that transforms the extracted features into target-oriented embeddings.
- A memory bank that stores representative normal feature prototypes.
- A loss function composed of two complementary terms: an **attraction loss**, which encourages normal features to move inside the hypersphere, and a **repulsion loss**, which pushes synthetic anomalous features away from the hypersphere boundary [34].

During inference, anomaly detection is performed by measuring the distance between test features and the nearest hypersphere prototype. This distance is used as the anomaly score, where larger deviations indicate a higher likelihood of abnormality. The coupled-hypersphere formulation provides an explicit geometric representation of the normal feature distribution, improving both anomaly detection and spatial localization capabilities [34].

Despite its theoretically appealing geometric formulation, CFA presents several practical limitations. Its performance is highly sensitive to hyperparameter selection, particularly the hypersphere radii and margin constraints that define the normal feature region. Moreover, similarly to other memory-based approaches, CFA strongly depends on the quality of the stored prototypes: contamination within the normal training dataset can distort the learned feature boundary and significantly reduce detection performance [34].

2.2.10 SuperSimpleNet: Unifying Supervision Regimes

SuperSimpleNet [49] represents one of the most recent developments among the anomaly detection methods considered in this work. Unlike previous approaches that are generally designed for a single supervision setting, most commonly unsupervised learning, SuperSimpleNet is proposed as a unified framework capable of operating under multiple supervision regimes encountered in real-world industrial scenarios:

1. **Unsupervised**: only normal training samples are available.
2. **Weakly supervised**: image-level anomaly labels are available without pixel-level annotations.
3. **Mixed supervision**: both normal samples and labeled anomalous samples are provided.
4. **Fully supervised**: pixel-level anomaly annotations are available for training.

Based on the SimpleNet framework [36], SuperSimpleNet introduces three main improvements: an advanced synthetic anomaly generation strategy inspired by DRAEM, an enhanced classification head for improved anomaly discrimination, and a refined training procedure. The method achieves inference times below 10 ms while maintaining competitive detection performance across different supervision settings. Its capability to adapt to varying levels of annotation availability makes it particularly suitable for industrial environments, where the amount and quality of defect annotations may differ significantly between production lines [49].

Despite its high flexibility and efficiency, SuperSimpleNet inherits a key limitation from its predecessor: the dependence on synthetic anomaly generation in the feature space. When real manufacturing defects involve complex structural changes that cannot be effectively represented through simulated perturbations such as Perlin or Gaussian noise, the learned classification boundary may become less reliable, potentially reducing detection performance on previously unseen anomaly categories [49].

2.3 Comparative Analysis

The preceding section reviewed ten anomaly detection methods. While each method demonstrates strong performance on standard benchmarks, industrial deployment imposes additional constraints beyond raw accuracy. This section presents a comparative empirical analysis of these methods on three datasets, with the specific goal of selecting the most suitable baseline for the edge acceleration work described in subsequent chapters.

2.3.1 Experimental Setup

Datasets. The evaluation employs three datasets spanning different levels of complexity and domain specificity:

- **MVTec AD** [9]: The standard benchmark for industrial anomaly detection, comprising 15 categories (10 objects, 5 textures) with 5,354 images. Defects include scratches, dents, contaminations, and structural anomalies across different materials and lighting conditions.

- **VisA** [79]: A more challenging dataset with 12 categories and 10,821 images, featuring higher intra-class variance and more complex defect patterns. VisA is characterized by severe class imbalance: defect samples constitute only 11.1% of total data [79], making it a robust test for generalization.

- **Cable Factory Dataset (Proprietary)**: A custom dataset collected at the WTM cable production facility, consisting of 450 normal images and 280 anomalous images covering two defect types: surface scratches and taping phase errors. Images are acquired using the 5MP polarized LUCID setup introduced in Section 1.5, with strong illumination synchronization to eliminate ambient light.

Hardware. All experiments here are conducted on a workstation equipped with an NVIDIA RTX 4060 GPU (8 GB VRAM), AMD Ryzen 7 5800X, and 32 GB RAM DDR4. This hardware serves as the *accuracy reference platform*; the subsequent acceleration work targets the AMD Kria KV260 System-on-Module (SOM) integrated with the DeepX NPU, which imposes stricter memory and latency constraints.

Metrics. Following standard practice in the literature [9, 1], we report:

- **Image-level AUROC**: Classification performance at the image level.

- **Pixel-level AUROC**: Localization performance at the pixel level.

- **Image-level F1-score**: Harmonic mean of precision and recall at optimal threshold.

- **Pixel-level F1-score**: Corresponding F1 for pixel-wise predictions.

- **Inference time (CPU)**: Simulating deployment on the industrial PC without GPU acceleration.

- **Memory footprint**: Model weights plus memory bank storage requirements.

2.4 Quantitative Results

This section provides a detailed performance evaluation of the ten anomaly detection methods, partitioned by dataset to highlight domain-specific behaviors and model stability.

2.4.1 MVTec AD Benchmark

The evaluation on the MVTec AD dataset reveals significant performance disparities across the ten selected architectures. Table 2.1 provides an overview of the mean performance across all metrics, while Figure 2.2 illustrates the distribution of Image-AUROC across the 15 individual categories.

Discussion. The table 2.1 confirms that PatchCore achieves the highest overall accuracy. However, the boxplot in Figure 2.2 provides a more critical insight: **model stability**. PatchCore exhibits the narrowest interquartile range (IQR) near the top of the scale, indicating consistent performance across all 15 categories. Conversely, models like CFA or SuperSimpleNet show high variance, performing well on some categories but failing significantly on others. This robustness is the primary factor driving the selection of PatchCore as the baseline for the edge acceleration work in this thesis.

Model	Image AUROC	Pixel AUROC	Image F1	Pixel F1
PatchCore	0.982	0.980	0.970	0.581
Reverse Distillation	0.976	0.973	0.968	0.565
FastFlow	0.941	0.970	0.936	0.546
EfficientAD	0.936	0.942	0.929	0.594
DFM	0.927	0.925	0.935	0.382
PaDiM	0.902	0.964	0.920	0.474
STFPM	0.883	0.952	0.912	0.490
DRAEM	0.788	0.882	0.876	0.348
CFA	0.700	0.932	0.858	0.422
SuperSimpleNet	0.473	0.598	0.828	0.101

Table 2.1: Average performance results on the MVTec dataset across all 15 categories. Models are ranked by Image AUROC (descending).

2.4.2 VisA: High-Variance Analysis

The VisA dataset introduces complex background noise, multiple objects per image, and high intra-class variance, making it significantly more challenging than standard benchmarks like MVTec AD. To thoroughly evaluate model robustness under these conditions, performance is analyzed across two dimensions: average metric scores and category-level variance.

Table 2.2 details the aggregated metrics for all evaluated architectures. The models are ranked by their Image AUROC performance.

Model	Image AUROC	Pixel AUROC	Image F1	Pixel F1
Reverse Distillation	0.961	0.987	0.919	0.462
PatchCore	0.937	0.981	0.891	0.425
EfficientAD	0.899	0.967	0.867	0.392
FastFlow	0.888	0.968	0.846	0.331
DFM	0.885	0.920	0.849	0.238
STFPM	0.823	0.909	0.818	0.368
PaDiM	0.822	0.963	0.811	0.266
DRAEM	0.722	0.760	0.741	0.134
CFA	0.688	0.927	0.762	0.261
SuperSimpleNet	0.504	0.662	0.726	0.024

Table 2.2: Average performance results on the VisA dataset across all 11 categories. Models are ranked by Image AUROC (descending).

Average Performance Analysis

The aggregated results reveal a clear hierarchy among the evaluated paradigms. **Reverse Distillation** emerges as the top performer (Image AUROC: 0.961), closely followed by **PatchCore** (0.937). These two methods significantly outperform the rest of the field in classification accuracy.

Crucially, the VisA dataset exposes a severe degradation in precise defect localization across all models. While Pixel AUROC remains relatively high (inflated by the vast number of easily classified normal background pixels), the **Pixel F1-score** drops drastically. Even the best-performing model, Reverse Distillation, achieves only a 0.462 Pixel F1, while PatchCore follows at 0.425. This highlights the inherent difficulty of localizing anomalies when the background itself exhibits high structural variance. Models like DRAEM (0.134) and SuperSimpleNet (0.024) practically fail at pixel-level segmentation on this dataset.

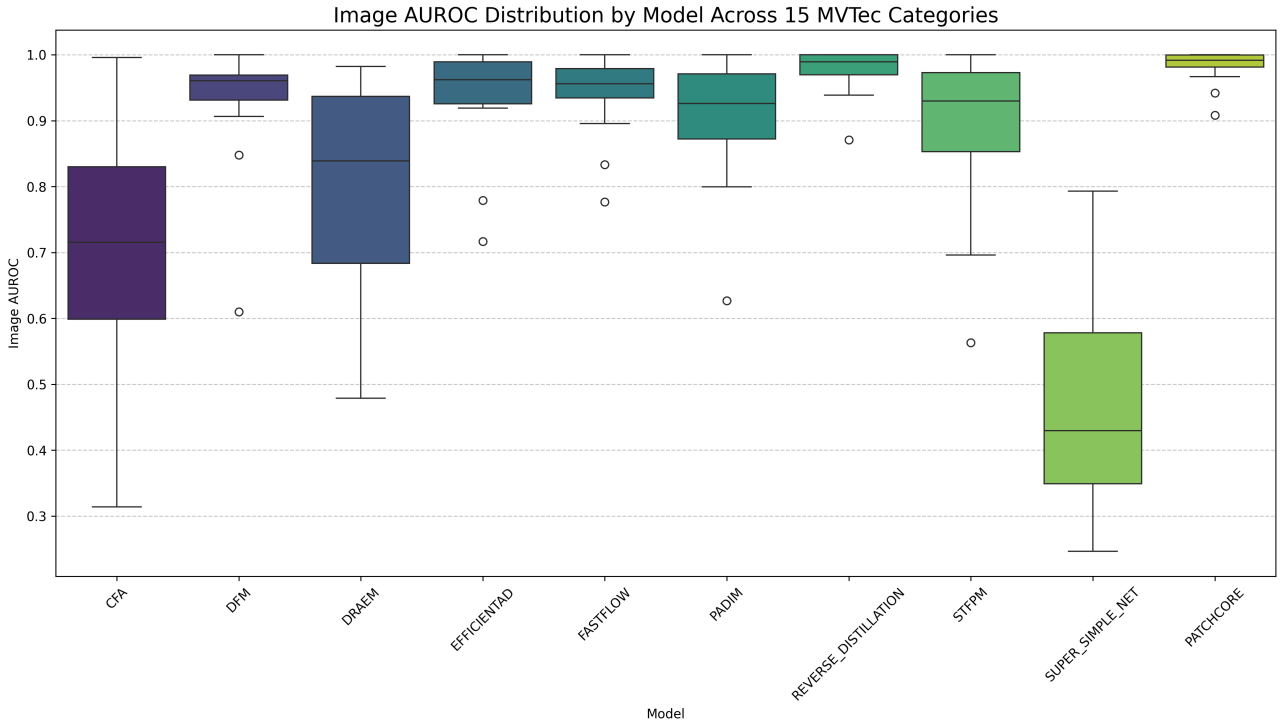


Figure 2.2: Distribution of Image-AUROC for each model, highlighting performance stability across diverse industrial object and texture categories.

Robustness and Variance

Average metrics alone can mask category-specific failures. To assess inter-category robustness, Figure 2.3 illustrates the distribution of Image AUROC scores across the 11 individual VisA categories.

The boxplot provides critical insights for industrial deployment:

- **High Robustness**: Reverse Distillation and PatchCore not only exhibit the highest median scores but also demonstrate remarkably tight interquartile ranges. Their performance remains consistently high across almost all categories, proving strong generalization capabilities.
- **Moderate Vulnerability**: Methods like EfficientAD and FastFlow maintain good median performance but show a wider spread, indicating sensitivity to specific types of complex backgrounds.
- **Catastrophic Failures**: Reconstruction-based methods struggle severely with VisA's variance. DRAEM shows a massive interquartile range and completely fails on at least one category (producing an outlier score of 0.0). Similarly, CFA and SuperSimpleNet exhibit broad distributions and low medians, confirming their unsuitability for high-variance industrial environments.

In conclusion, while Reverse Distillation achieves a marginal empirical advantage over PatchCore on this specific dataset, its requirement for a full student-decoder training phase (as discussed in Section 2.4.4) adds significant computational overhead. PatchCore delivers near-identical robustness and classification accuracy while preserving the rapid, training-free memory bank construction, reaffirming its selection as the optimal baseline for edge acceleration in the subsequent chapters.

2.4.3 WTM Cable Factory Dataset (Proprietary)

The final evaluation targets the proprietary WTM cable factory dataset. Unlike MVTec AD and VisA, which serve as broad academic benchmarks, this dataset represents the exact operational environment, illumination conditions, and defect morphologies targeted by the industrial deployment.

Evaluation Constraints: The Absence of Pixel Ground Truth. It is crucial to note that performance on this dataset is evaluated strictly at the image level (Image AUROC and Image F1). Pixel-level metrics are omitted because the dataset lacks pixel-wise ground truth (GT) masks. As

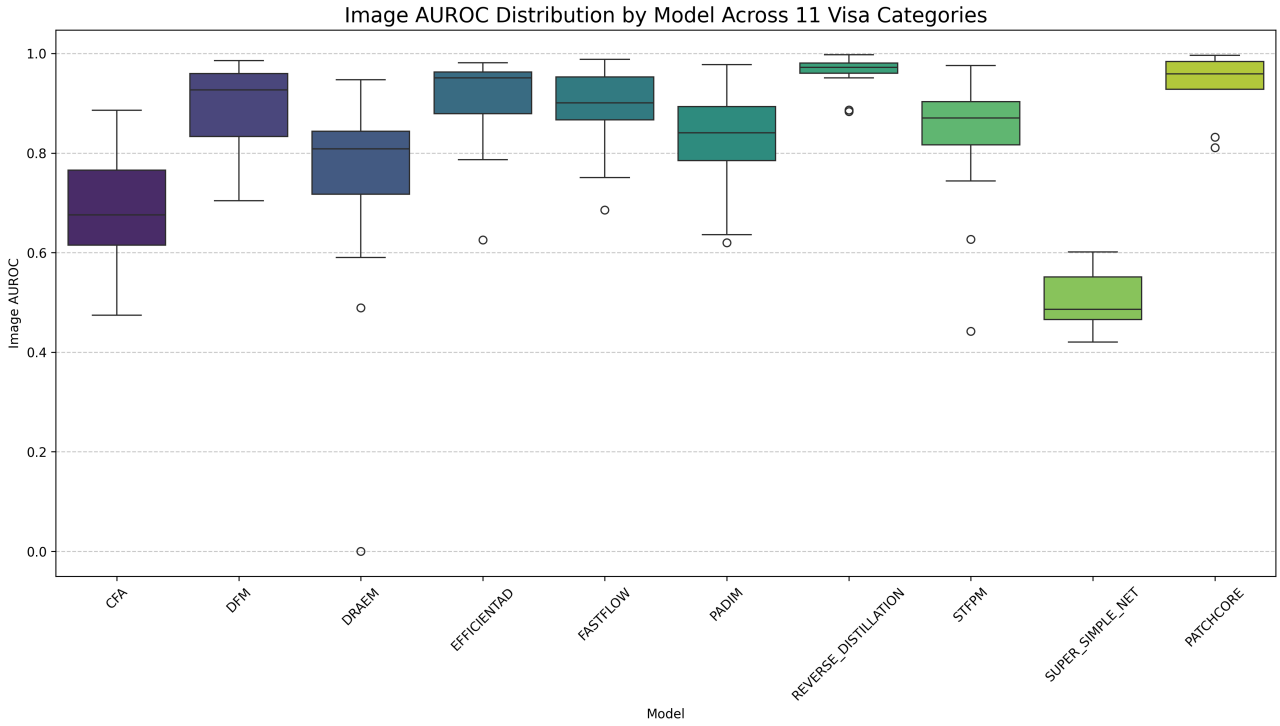


Figure 2.3: Distribution of Image AUROC scores across the 11 VisA categories. The boxplot illustrates the median, interquartile range, and outlier performance for each model.

argued in Section 1.1, generating pixel-perfect annotations for real industrial production runs is cost-prohibitive and impractical for factory operators. This constraint reflects the reality of industrial edge deployments, where models must be validated using only image-level binary labels (OK/KO).

Table 2.3 presents the performance and computational requirements for the evaluated models.

Model	**Image AUROC**	**Image F1**	**Train Time (s)**	**Test Time (s)**
PatchCore	0.993	0.977	13.95	16.18
EfficientAD	0.978	0.962	404.32	16.56
PaDiM	0.971	0.960	9.19	15.44
Reverse Distillation	0.955	0.941	230.82	13.68
FastFlow	0.921	0.916	81.21	15.82
CFA	0.917	0.936	183.56	12.97
DFM	0.907	0.899	8.52	15.30
STFPM	0.781	0.908	274.43	15.13
SuperSimpleNet	0.630	0.586	192.21	15.32
DRAEM	0.467	0.909	193.92	17.52

Table 2.3: Performance and computational requirements on the proprietary WTM dataset. Models are ranked by Image AUROC (descending). Pixel metrics are omitted due to the absence of ground truth masks.

Discussion and Trade-off Analysis. The results on the WTM dataset definitively validate the architectural choices made for this thesis. As illustrated in Table 2.3, **PatchCore** dominates the classification task, achieving an Image AUROC of 0.993.

However, raw accuracy is only one dimension of industrial viability. The scatter plot in Figure 2.4 visualizes the critical trade-off between accuracy and training time. The models clearly separate into distinct behavioral clusters:

- **The "Fast & Accurate" Cluster (Top-Left):** PatchCore and PaDiM reside in the ideal operational quadrant. They deliver state-of-the-art accuracy while requiring less than 15 seconds

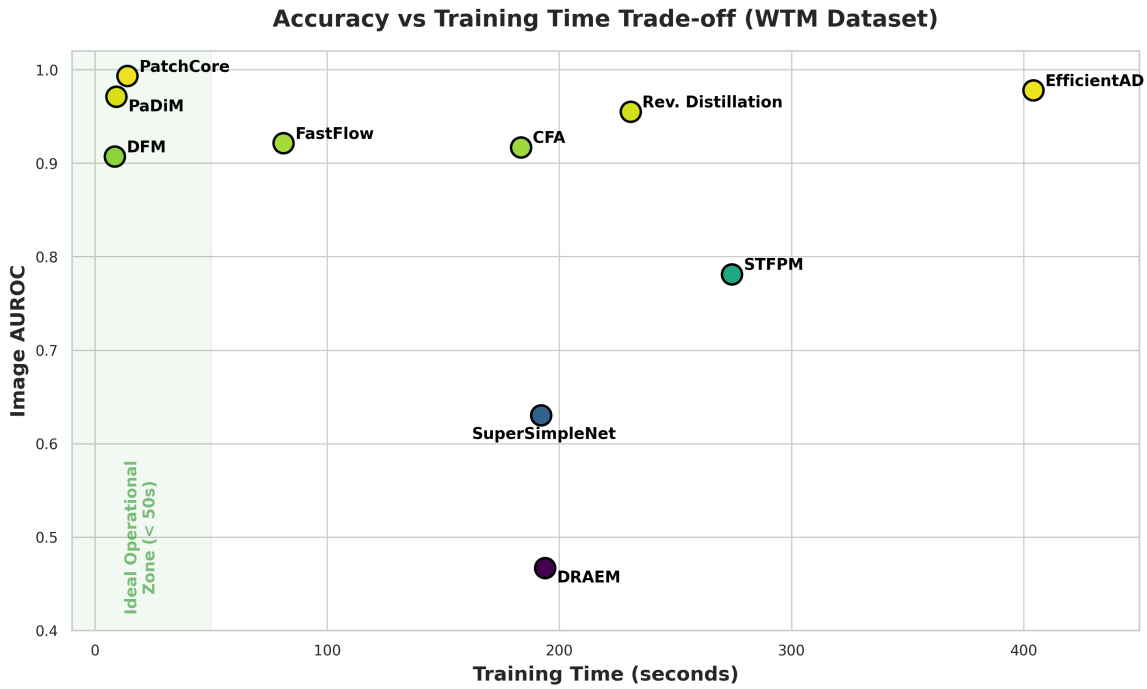


Figure 2.4: Trade-off analysis between model accuracy (Image AUROC) and training time on the WTM dataset. The ideal deployment candidate resides in the top-left quadrant (high accuracy, low training time).

to build their probabilistic/memory-based representations of normality.

- **The "Slow & Accurate" Cluster (Top-Right):** EfficientAD and Reverse Distillation achieve excellent accuracy (0.978 and 0.955 AUROC, respectively). However, their student-teacher architectures require full gradient descent optimization, taking 404 seconds and 230 seconds.
- **The "Suboptimal" Cluster (Bottom):** Models like STFPM, SuperSimpleNet, and DRAEM fail to learn robust representations of the specific cable defects, offering low accuracy despite substantial training times.

This immense discrepancy in training time is the decisive factor for the software architecture presented in Chapter 3. Since factory operators need to acquire new datasets and train models on the fly directly on the industrial PC, a training phase lasting a few seconds (PatchCore) is operationally seamless, whereas waiting nearly seven minutes (EfficientAD) for a network to converge breaks the user workflow. Consequently, PatchCore's position in the top-left quadrant of Figure 2.4 solidifies its selection as the baseline for DeepX NPU acceleration.

2.4.4 Computational Efficiency and Edge Feasibility

While the accuracy metrics established in the previous sections confirm the theoretical viability of several models, real-time industrial deployment necessitates a strict evaluation of computational overhead. As defined in Section 1.4, the target inspection system must operate within stringent latency budgets and limited memory capacities characteristic of edge hardware.

To assess edge feasibility, a dedicated profiling experiment was conducted. The top three performing architectures from the accuracy benchmarks are **PatchCore**, **EfficientAD**, and **Reverse Distillation**. Now they were isolated for latency and memory analysis.

Methodology. The profiling was executed on the reference CPU hardware (AMD Ryzen 7 5800X) to simulate a standard Industrial PC (IPC) environment prior to NPU acceleration. A standardized batch of 100 normal test images from the WTM dataset was processed by each model. To prevent initialization overhead from skewing the results, a 10-step warm-up phase was executed before precise timekeeping began. Memory footprint was captured at the operating system level using the `runlim`[11] utility to record the peak physical RAM consumed during the entire inference process.

Table 2.4 summarizes the computational profiles of the top three candidates.

Model	Weights (MB)	Peak RAM (MB)	Latency (ms)	FPS
PatchCore	244.52	1456.0	129.28	7.74
Reverse Distillation	693.87	2162.2	133.20	7.51
EfficientAD	59.34	1227.6	134.45	7.44

Table 2.4: Inference profiling on reference CPU hardware (average over 100 test images). Weights = model weights stored on disk. Peak RAM = peak system RAM measured with `runlim`. FPS is computed as 1000/Latency.

Latency and Throughput Analysis. Counterintuitively, **PatchCore** achieves the lowest CPU latency (129.28 ms/img). While *EfficientAD* reports < 10 ms in its paper[6], that result relies on advanced compilation toolchains (e.g., OpenVINO) to fuse dense convolutional layers, whereas our measurement reflects raw, unoptimized PyTorch execution. Since applying such compilers to all baselines falls outside this pre-NPU profiling scope, PatchCore, which leveraging highly optimized linear algebra routines (BLAS) for its k-NN search, remains the most competitive out-of-the-box algorithm here.

Memory Footprint Constraints. The memory profiling exposes severe deployment constraints. **Reverse Distillation** is entirely unsuited for resource-constrained edge devices, demanding nearly 700 MB just for model weights and causing a system-level RAM peak of over 2.1 GB during inference. **EfficientAD** validates its lightweight design with merely 59.34 MB of weights and the lowest peak RAM (1.22 GB). **PatchCore** occupies a middle ground, requiring 244.52 MB for weights and a 1.45 GB system RAM peak to maintain its coreset memory bank in active memory.

Conclusion: The Mandate for Hardware Acceleration. The profiling results present a clear engineering mandate. PatchCore possesses the ideal combination of rapid, user-friendly training, state-of-the-art accuracy, and comparatively competitive generic CPU inference.

However, achieving 7.74 FPS on a high-end desktop-class Ryzen 7 processor implies that throughput would collapse to unacceptable levels (e.g., 1-2 FPS) if deployed on the low-power fanless CPUs typically found in factory environments. Furthermore, a 1.45 GB memory footprint poses a significant challenge for direct porting to the constrained SRAM of an AI accelerator. Therefore, retaining PatchCore's algorithmic superiority while mitigating its computational bottlenecks via hardware-aware acceleration (quantization, approximate search, and DeepX NPU compilation) forms the core objective of the subsequent chapters.

2.4.5 Analysis and Selection Justification

The empirical results from both the standard benchmarks (MVTec AD, VisA) and the proprietary WTM dataset reveal clear trade-offs among the evaluated methods. The following observations directly inform the selection of PatchCore as the baseline for this thesis:

Absolute Accuracy on Cable Defects. PatchCore achieves the highest image-level AUROC on the cable factory dataset (0.993), outperforming the next best method, EfficientAD (0.978), and dominating pure feature-matching approaches like DFM (0.907). This near-perfect classification score indicates that PatchCore's multi-scale feature embeddings are exceptionally well-suited for capturing the specific visual anomalies of industrial cables, such as surface scratches and taping phase errors [51].

Training Usability. As highlighted in Section 2.4.3, industrial feasibility depends heavily on the time required to build a new model on the edge device. PatchCore requires less than 15 seconds to construct its coreset memory bank from normal images. Conversely, student-teacher architectures like EfficientAD and Reverse Distillation demand hundreds of seconds (up to nearly 7 minutes) of active backpropagation. For a non-expert operator using the GUI, the near-instantaneous training of PatchCore is a decisive operational advantage.

The Memory Bank: A Clear Acceleration Target. While PatchCore proved to be the fastest model on the generic desktop CPU (129 ms per image), an analysis of its underlying operations reveals a critical challenge for edge deployment. The inference is entirely dominated by the k-Nearest Neighbor

(k-NN) search over the memory bank. To achieve this speed, the CPU must hold approximately 1.45 GB of data in active RAM. When porting this architecture to the constrained environment of a Kria (which relies on limited internal SRAM), this exact k-NN search and massive memory footprint become catastrophic bottlenecks.

PatchCore's Suitability for Acceleration. The memory bank architecture of PatchCore presents three clear optimization axes that form the core of the engineering work in the subsequent chapters:

1. **Search complexity reduction**: The exact k-NN search is $O(N)$. Approximate methods, such as Hierarchical Navigable Small World (HNSW) graphs, can reduce this to $O(\log N)$ while maintaining high recall.

2. **Memory bank compression**: While the default coreset algorithm already reduces storage, advanced Hard Prototype Mining can further prune redundant vectors without accuracy degradation, directly addressing the RAM bottleneck.

3. **Architectural distillation**: The memory bank can be entirely replaced by a teacher-student Reverse Distillation paradigm, trading the $O(N)$ search bottleneck for a fixed $O(1)$ feedforward pass, strictly optimized for NPU systolic arrays.

By contrast, methods like EfficientAD are already architecturally optimized for edge execution (occupying only 59 MB of weights) and offer significantly less headroom for structural acceleration, while lacking PatchCore's rapid training capabilities.

2.4.6 Limitations of the Analysis

Several limitations of this comparative analysis should be acknowledged:

Cable Dataset Scale. The proprietary cable dataset is relatively small compared to MVTec AD (5,354 images). Generalization claims on cable defects are therefore tentative; larger-scale validation would be required for production deployment.

Desktop vs. Industrial Hardware. The inference times and RAM peaks reported in Table 2.4 were measured on a high-end desktop CPU (Ryzen 7 5800X) with dedicated DDR4 memory. In a real industrial scenario using a fanless IPC, concurrent processes (PLC communication, HMI updates, data logging) would increase effective latency. The reported 7.7 FPS should be interpreted as an optimistic upper bound for purely CPU-bound execution, further justifying the need for NPU acceleration.

Conclusion of Comparative Analysis

Based on the empirical evidence presented throughout this chapter, **PatchCore is selected as the baseline architecture** for the edge acceleration work described in this thesis. The selection is justified by three factors: (1) superior, near-perfect accuracy on the target cable factory defects; (2) rapid, operator-friendly training times compared to backpropagation-based methods; and (3) a clear, addressable architectural bottleneck on the k-NN search over a massive memory bank that provides ideal headroom for the hardware-aware optimizations developed in Chapter 5.

The following section 4 provides a detailed exposition of the internal PatchCore architecture, establishing the mathematical foundation necessary to understand how the memory bank is constructed and subsequently accelerated.

2.5 Qualitative Analysis and Defect Localization

To contextualize the quantitative metrics presented in Section 2.4 and in particular to explain the discrepancy between high Image AUROC and moderate Pixel F1-scores, a visual inspection of the model outputs is highly illustrative. This section presents qualitative results of the PatchCore baseline across the three evaluated datasets, highlighting the specific domain challenges of each environment.

2.5.1 MVTec AD: The Dilation Effect

Figure 2.5 presents PatchCore's predictions on the MVTec AD dataset. As visually evident, the model successfully isolates and highlights the true anomalies. However, the resulting anomaly heatmaps are inherently dilated due to the bilinear upsampling and Gaussian smoothing applied during the post-processing of the patch-level scores.

Consequently, the thresholded predicted masks cover a broader bounding area than the pixel-perfect human-annotated ground truth masks. This structural dilation explains the strict penalty in Pixel F1-scores seen in Table 2.1, confirming that the model provides robust region-level defect localization suitable for industrial routing, even if it lacks exact pixel-level segmentation boundaries.

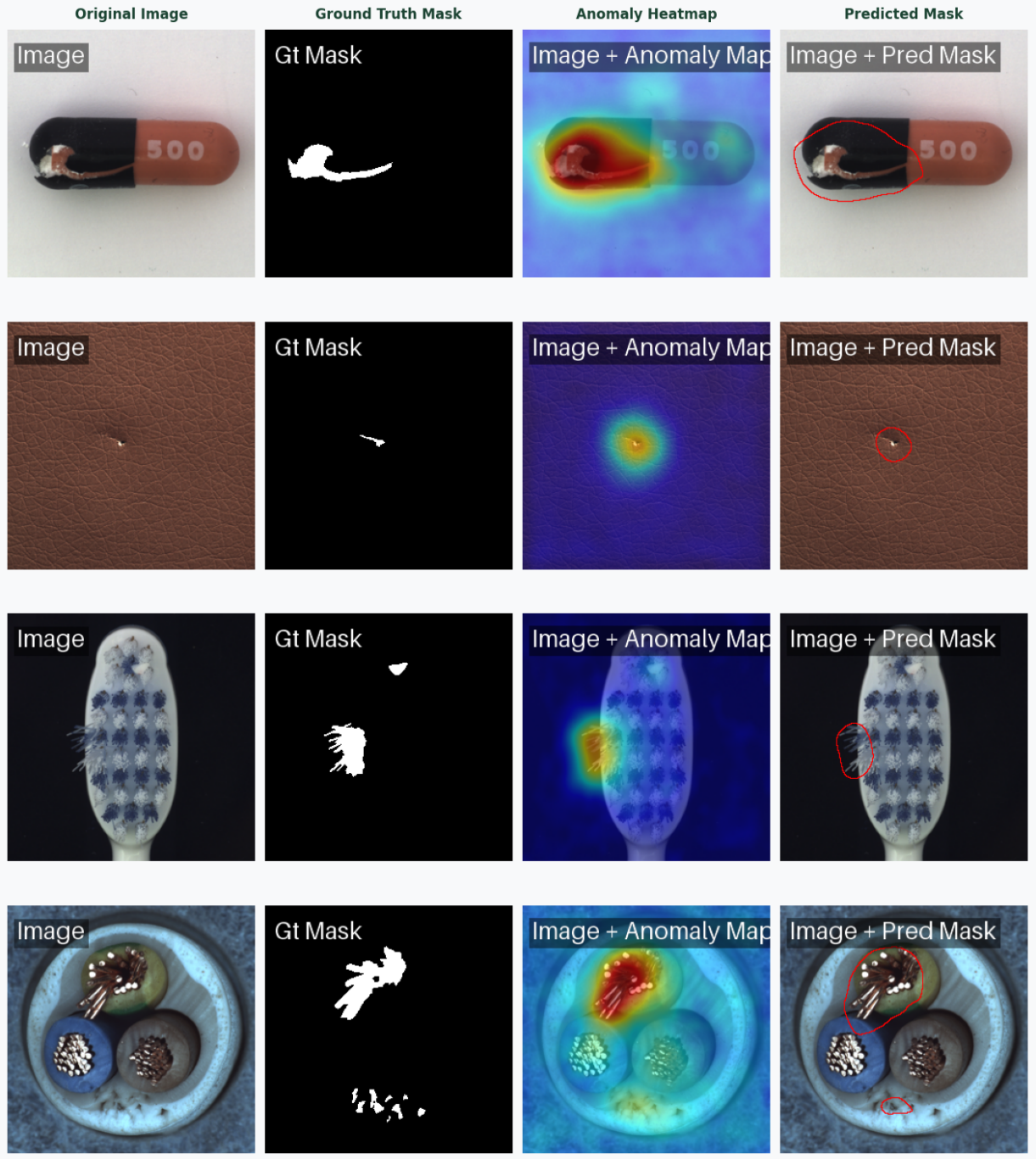


Figure 2.5: Qualitative analysis on MVTec AD. The heatmap interpolation naturally produces dilated predicted masks compared to the exact ground truth, explaining the lower Pixel F1-scores despite successful localization.

2.5.2 VisA: High-Variance Backgrounds

The challenge escalates with the VisA dataset, as demonstrated in Figure 2.6. Unlike MVTec, where objects are typically centered on uniform backgrounds, VisA features multiple objects (e.g., capsules, pasta, candles) scattered across highly textured and variable backgrounds.

The qualitative results reveal why models struggle with pixel-level metrics on this dataset (as discussed in Section 2.4.2). While PatchCore maintains a strong anomaly peak on the actual defect, the complex background occasionally induces low-level thermal noise in the anomaly map. This visual evidence perfectly corroborates the drop in Pixel F1-score to 0.425, demonstrating the difficulty of thresholding subtle defects against high-variance backgrounds.

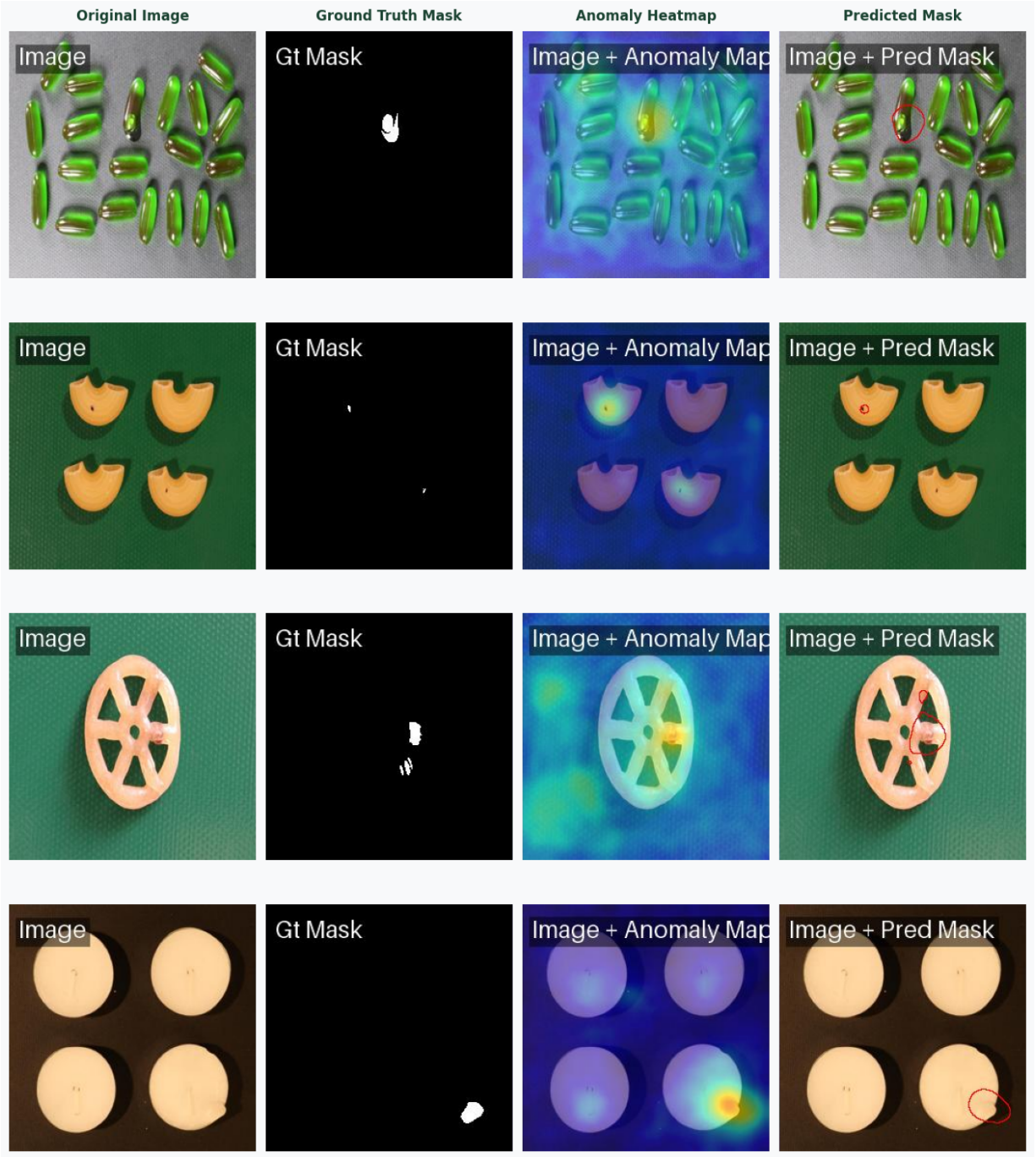


Figure 2.6: Qualitative analysis on VisA. The presence of multiple objects and textured backgrounds introduces scattered noise in the anomaly heatmap, challenging the pixel-level segmentation.

2.5.3 WTM Cable Dataset: The Industrial Reality

Finally, Figure 2.7 illustrates the model's behavior on the proprietary WTM cable dataset. This qualitative assessment is particularly crucial because, as noted in Section 2.4.3, this dataset lacks pixel-level ground truth masks.

The visual output demonstrates the model's robustness to real-world industrial conditions. Despite specular reflections from the metallic wire and variations in the polymer extrusion, the PatchCore embeddings successfully suppress the background noise and generate a sharp, localized heat peak exclusively on the surface defect (e.g., taping phase errors or cut outer sheaths). This confirms that the exceptional 0.993 Image AUROC translates into actionable, highly reliable visual feedback for the factory operators using the GUI.

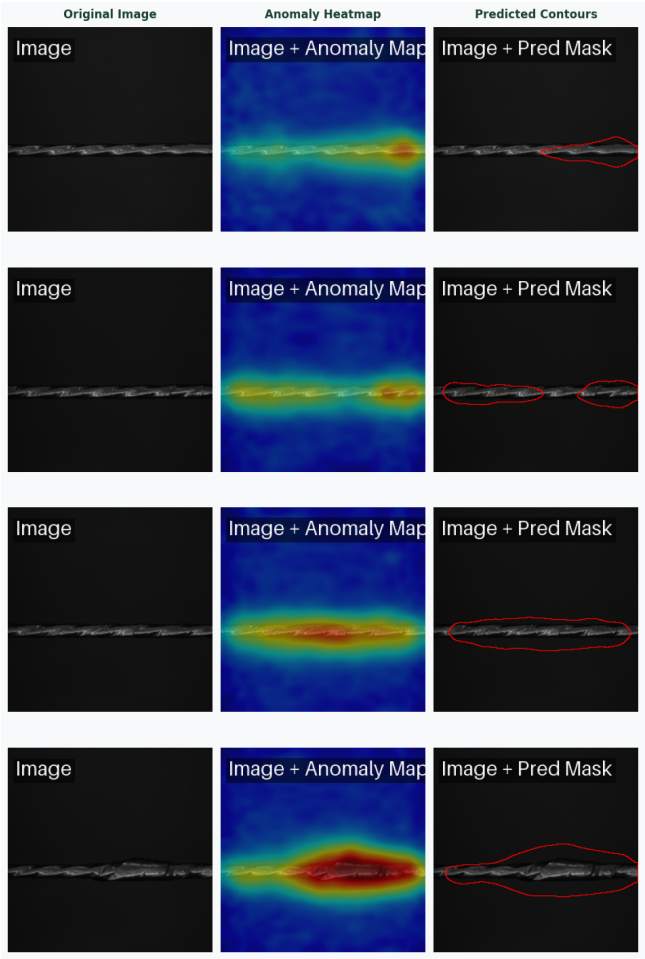


Figure 2.7: Qualitative analysis on the proprietary WTM dataset. Despite industrial lighting reflections and the lack of ground truth masks for training, the model produces highly precise and actionable defect heatmaps.

2.6 Why Not Transformers, VLMs, and LLMs

The preceding sections have focused exclusively on methods from the 2020–2022 period. This deliberate choice requires explicit justification, given the proliferation of transformer-based architectures, Vision-Language Models (VLMs), and Large Language Models (LLMs) for anomaly detection tasks in recent years [62, 65]. This section analyzes the fundamental incompatibilities between these modern approaches and the constraints of industrial edge deployment that motivate this thesis.

Overview of Recent SOTA Approaches

The last years have witnessed a paradigm shift toward unified, language-driven anomaly detection architectures. Representative methods include:

- **UniVAD** [71]: A unified framework for video anomaly detection leveraging transformer backbones.
- **AnomalyGPT** [22]: A large language model adapted for anomaly detection through prompt engineering and instruction tuning.
- **UniCLIP-AD** [68]: A CLIP-based universal anomaly detection model leveraging contrastive language-image pre-training.
- **SAEC** [62]: A scene-aware edge-cloud collaborative framework that uses multimodal LLMs (MLLMs) for complex defect inspection, with adaptive scheduling between edge and cloud resources.
- **Cosmos Reason2** [42]: NVIDIA's vision-language family (2B and 8B parameter models) with built-in chain-of-thought reasoning for anomaly detection and scene analysis.
- **InspectVLM** [65]: A Florence-2-based unified VLM trained on large-scale inspection data, designed to reframe classification, detection, and keypoint tasks within a single language-driven interface.

These methods demonstrate impressive capabilities on academic benchmarks, including zero-shot generalization and natural language explanations of detected anomalies. However, their deployment in industrial edge scenarios faces insurmountable obstacles.

Computational Resource Requirements

The most fundamental barrier to deploying these methods on edge hardware is their computational intensity. Table 2.5 summarizes the resource requirements for representative recent models, based on published specifications and deployment guides [25, 42].

Model	Parameters	Model Size	RAM Required	Storage (FP8)
Moondream2	1.8B	1.7 GB	≈ 2 GB	–
Qwen2.5-VL	3B	3.2 GB	~ 5 GB	–
Cosmos Reason2 (2B)	2B	–	–	~ 5 GB
Cosmos Reason2 (8B)	8B	–	–	~ 17 GB
GPT-4V-based AD	–	–	> 16 GB	–

Table 2.5: Comparison of lightweight vision-language models for edge deployment.

The contrast with our target hardware is stark. The DeepX NPU and its host Industrial PC or Kria, have no dedicated GPU and limited shared memory. Even the most compact VLMs, such as Moondream2 at 1.8B parameters requiring ≈ 2 GB RAM, exceed the available memory on typical IPC configurations. The 2B parameter Cosmos Reason2 model, the smallest recommended for edge deployment, requires approximately 5 GB of storage for FP8 weights and demands an NVMe SSD, a component rarely present in cost-optimized industrial controllers [42].

Inference Latency and Real-Time Constraints

For industrial inline inspection, the maximum permissible inference latency is determined by the physical geometry of the production line:

$$T_{max} = \frac{d}{v}$$

where d is the distance from the inspection camera to the reject actuator and v is the conveyor belt speed. For typical configurations ($d = 0.25$ m, $v = 1$ m/s), $T_{max} = 0.25$ seconds. This appears generous, but this budget must accommodate image acquisition, preprocessing, neural inference, and actuator communication.

Transformer-based and VLM architectures present inference latencies that are both high and non-deterministic. The self-attention mechanism's $O(n^2)$ complexity with respect to sequence length introduces variability that is unacceptable for safety-critical industrial control loops. Even with optimized implementations on server-grade GPUs, VLMs typically require hundreds of milliseconds to seconds per inference, without accounting for the additional overhead of text generation for language outputs [62].

SAEC [62], a recent edge-cloud collaborative framework, achieves runtime reductions of up to 22.4% compared to baseline LLMs, but this still implies latencies on edge hardware. Moreover, the framework relies on dynamic offloading to cloud servers for complex cases, introducing network jitter and violating the deterministic latency requirement established in Section 1.4.

Reliability and Determinism Concerns

Beyond raw resource requirements, recent research has identified more fundamental concerns regarding VLM reliability in industrial contexts. Corley et al. [65] conducted a critical evaluation of InspectVLM, a unified VLM trained specifically for industrial inspection tasks. Their findings are sobering.

The model exhibits brittle behavior under low prompt variability, where even small changes in input prompts lead to unstable outputs, thereby compromising reproducibility. In addition, it shows degenerate behavior in fine-grained detection scenarios, failing to reliably localize small defects, which is a critical requirement for pixel-level anomaly segmentation. Finally, the model tends to default to memorized language responses, often ignoring visual input entirely and producing pre-learned textual outputs regardless of the actual image content. The authors conclude that "while language-driven unification offers conceptual elegance, current VLMs lack the visual grounding and robustness necessary for deployment in precision-critical industrial inspections" [65].

For applications where missed defects translate directly to material waste and economic loss, such unreliability is unacceptable. The deterministic, geometrically grounded anomaly scores produced by PatchCore (distance in feature space) offer a degree of interpretability and predictability that language model outputs cannot match.

This section has justified the exclusion of transformer-based, VLM, and LLM approaches from consideration in this thesis. The resource requirements (memory, latency, power) of these methods exceed the constraints of the target industrial edge platform by one to two orders of magnitude. More fundamentally, recent research has raised legitimate concerns about VLM reliability and determinism for precision-critical inspection tasks. The methods reviewed in previous sections, represent the appropriate technological basis for edge-accelerated anomaly detection. The remainder of this thesis focuses on accelerating PatchCore for Edge deployment, rather than adapting fundamentally incompatible architectures to the edge environment.

3 System Architecture

This chapter describes the software and hardware architecture developed during this thesis to enable non-expert factory operators to acquire datasets, train anomaly detection models, and perform real-time inference. The system integrates four main components: a graphical user interface (GUI) built with Streamlit and FastAPI, a Docker container for environment reproducibility, a DeepX compilation pipeline for Kria target deployment, and an automatic 3-sigma thresholding mechanism that operates without requiring labeled defective data.

3.1 Graphical User Interface: Streamlit and FastAPI

The graphical user interface is designed for factory operators with no machine learning expertise. The core requirement is to abstract away the complexity of model training, dataset management, and inference configuration while preserving the flexibility to adjust critical parameters when needed.

3.1.1 Technology Stack

The interface is implemented using two complementary frameworks:

- **Streamlit**: Provides the frontend visual components: buttons, sliders, image displays, and real-time video streams. Streamlit was selected for its rapid development cycle and minimal code footprint.
- **FastAPI**: Serves as the backend API layer, handling training requests, database operations, and camera communication. The separation between frontend (Streamlit) and backend (FastAPI) enables the system to be deployed in a distributed manner if required.

The core anomaly detection logic leverages **Anomalib** [1], an open-source library specifically designed for industrial anomaly detection. All algorithms reviewed in Chapter 2 (PatchCore, PaDiM, EfficientAD, etc.) are implemented using Anomalib's standardized interfaces, which provide consistent training and inference pipelines across different architectures.

Dual-Stage Inspection Pipeline. Beyond simple anomaly localization, the system integrates a secondary classification stage to identify specific defect types and filter out false positives. While PatchCore exhibits an exceptionally high Image AUROC, raw pixel-level anomaly maps can occasionally trigger false alarms in highly volatile industrial environments, often caused by transient surface reflections, structural vibrations, or environmental dust particles resting on the cable insulation.

To mitigate this, a dual-stage verification mechanism was engineered. Upon detecting an anomaly, defined as any contiguous region in the heatmap exceeding the calibrated 3-sigma baseline threshold, the system extracts the corresponding bounding box from the *original RGB input frame*. To ensure spatial context, this bounding box is dynamically padded with a fixed 15% margin relative to the detected anomaly contour.

This localized RGB crop is then resized to 256×256 pixels and passed through a lightweight **MobileNetV2** model [53]. This secondary network is explicitly fine-tuned on a proprietary dataset comprising approximately 1,500 annotated crops spanning actual defect classes (e.g., *missing_cable, cut_outer, taping_error*) as well as a dedicated *false_alarm* class containing common optical artifacts. The MobileNetV2 acts as a strict verification gate: if the crop is classified as a false alarm, the initial PatchCore anomaly signal is suppressed.

Empirical validation shows that this dual-stage mechanism drastically reduces the operational false positive rate to less than 2%, transforming raw anomaly heatmaps into reliable, actionable diagnostic labels for the operator. Because MobileNetV2 is highly optimized for resource-constrained architectures, this second stage introduces a negligible inference overhead of just 3.5 ms per crop on the host CPU, successfully delivering precise defect diagnosis without hindering real-time line synchronization.

3.1.2 User Workflow

The GUI supports a complete end-to-end workflow, illustrated in Figure 3.1.

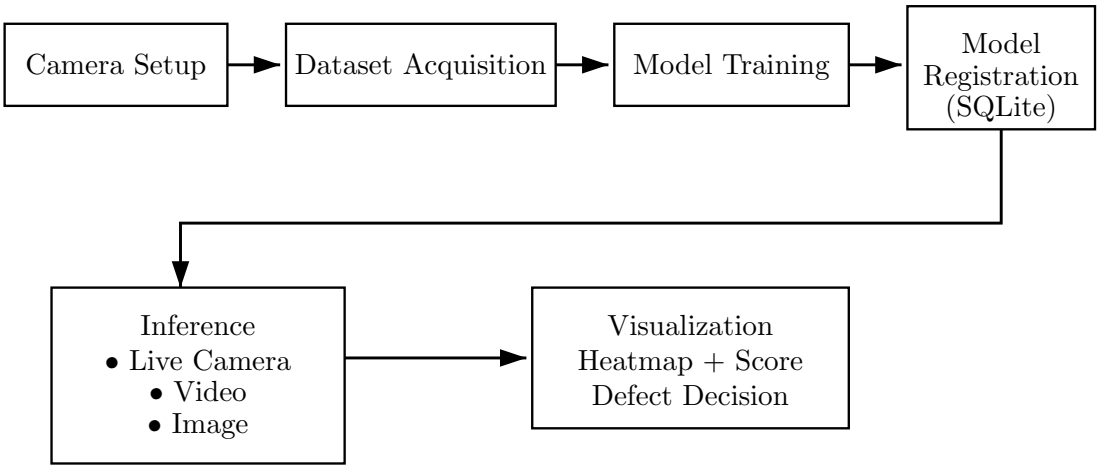


Figure 3.1: High-level user workflow in the GUI. Each step is accessible through dedicated interface panels.

Camera Configuration. The user specifies the IP address of one or more industrial cameras. The system supports multiple cameras concurrently, enabling inspection at different points along the production line or from different viewing angles. For each camera, the user can define a Region of Interest (ROI) to focus the inspection on relevant portions of the image, reducing computational load and eliminating background distractors.

Dataset Acquisition. The user can acquire training data in two modes:

- **Video acquisition**: The software captures frames from a pre-recorded video file containing normal cable samples.
- **Real-time acquisition**: Frames are captured live from the camera stream while the production line operates.

Acquired images are automatically organized into a structured directory hierarchy. A configurable subset of acquired images is reserved for validation, specifically for the 3-sigma threshold calibration described in Section 3.4.

Training. From the dataset management panel, the user selects a previously acquired dataset, chooses an anomaly detection algorithm (PatchCore, PaDiM, EfficientAD, etc.), and initiates training. The training process runs entirely in the background; the GUI remains responsive, displaying progress indicators and logs. Upon completion, the model and its associated metadata (training date, algorithm name, dataset identifier, and computed anomaly threshold) are saved as a **receipt** in a SQLite database. This receipt mechanism allows the user to maintain multiple trained models and switch between them without retraining.

Inference. The user selects a receipt from the SQLite database, representing a specific trained model. Three inference modalities are available:

1. **Real-time inference**: Frames from the live camera stream are processed continuously. Anomaly heatmaps are overlaid on the original image, and detected defects are highlighted with a colored circle. An FPS counter is displayed in real time to monitor system performance.
2. **Video inference**: A pre-recorded video file is processed frame by frame, with the same visualization overlay.
3. **Single image inference**: A single still image is processed, useful for spot-checking or validation.

Figure 3.2 shows a representative screenshot of a single image inference interface.

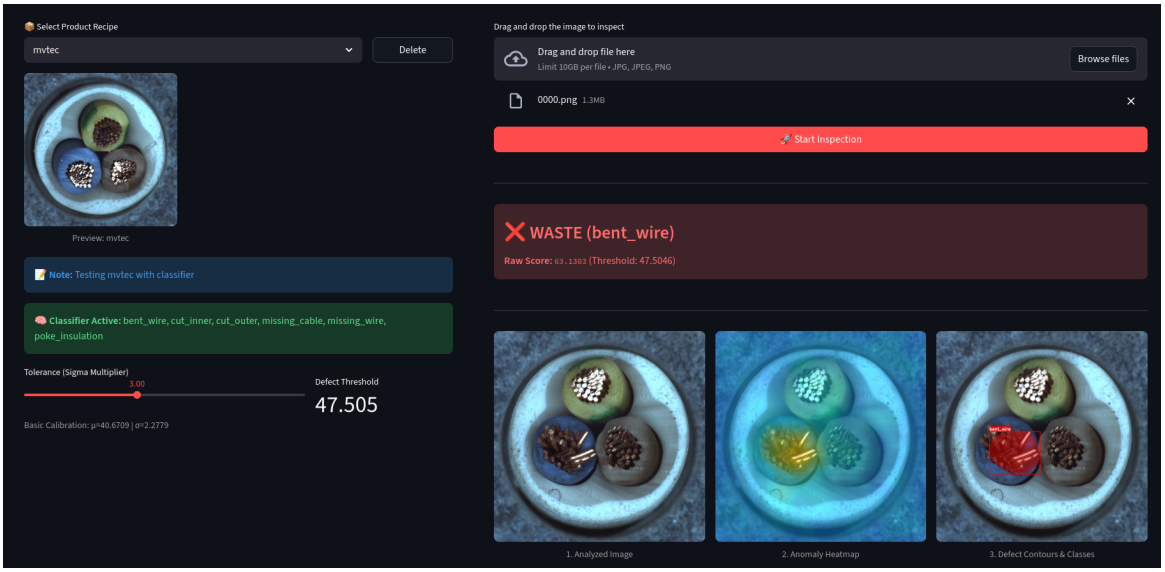


Figure 3.2: GUI image inference interface showing anomaly heatmap overlay, defect highlighting (red circle), and Threshold slider.

3.1.3 Intended User Profile

The GUI is explicitly designed for factory operators without machine learning expertise. All machine learning parameters (learning rates, network architectures, coreset ratios) are hidden behind sensible defaults. The operator interacts only with domain-appropriate concepts: camera selection, dataset acquisition, and model selection from a pre-configured list of algorithms. The SQLite receipt system allows operators to train multiple models for different cable types or production runs and switch between them with a single click, no command-line interaction required.

3.2 Docker Containerization and Resource Isolation

To simplify deployment and ensure absolute environment reproducibility across varying industrial PCs, the software stack was containerized using Docker.

The container is built upon an `ubuntu:22.04` base image compiled with Python 3.11. It packages all low-level libraries required for anomaly detection and inference, including PyTorch (CPU-only execution) [44], OpenCV [12], Anomalib [1], ONNX Runtime [5], and FAISS [29]. The resulting optimized Docker image yields a static storage footprint of approximately **4.2 GB**.

Because industrial PCs on the factory floor share physical hardware resources with critical real-time components, such as Programmable Logic Controllers (PLCs) and main SCADA supervisory systems, unconstrained container execution poses a severe risk of CPU starvation for the host OS. To prevent the computer vision pipeline from saturating the host system, strict resource limits were enforced directly within the deployment `docker-compose.yml` configuration file. The execution environment is hard-capped to a maximum resource allocation of 4 CPU cores and a strict memory ceiling of 4 GB (`cpus: '4', mem_limit: '4g'`). Persistent resources, including acquired datasets and SQLite database receipts, are mounted via isolated external volumes to decouple data state from the container lifecycle.

3.3 DeepX Compilation Pipeline

The DeepX DX-M1 is an AI accelerator designed to be mounted directly on smart cameras or industrial PCs via standard M.2 slots. The compilation pipeline converts high-level PyTorch models into a highly optimized binary format executable on the DeepX hardware.

3.3.1 Compilation Workflow

The pipeline consists of four distinct stages, illustrated in Figure 3.3:

1. **PyTorch to ONNX**: The trained anomaly detection model (e.g., PatchCore's feature extractor backbone) is exported to the ONNX format. This intermediate graph representation decouples the mathematical network definition from the downstream execution hardware.

2. **ONNX Optimization**: The raw ONNX graph is restructured by the DeepX compiler. Graph-level optimizations, including constant folding, dead code elimination, and extensive operator fusion (e.g., merging sequential Convolution, BatchNorm, and ReLU blocks into single execution units), are applied to minimize data movement overhead.

3. **Post-Training Quantization (PTQ)**: To exploit the DeepX accelerator's systolic array architecture, which operates natively on integer data, the optimized FP32 graph must be converted to fixed-point INT8 precision. The quantization parameters (scale factors and zero points) are calibrated using a representative subset of **100 normal training images**. The compiler utilizes an **Exponential Moving Average (EMA)** calibration algorithm to smoothly track activation distributions, preventing quantization noise from truncating low-frequency feature channels.

4. **Binary Generation**: The compiler yields a finalized `.dxnn` binary file containing the quantized weights and the localized execution graph instructions, ready to be loaded onto the host-connected DeepX hardware module for inline inference.

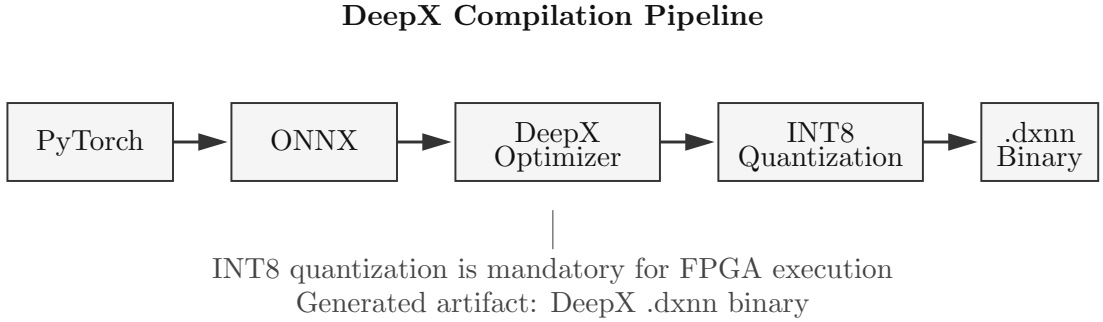


Figure 3.3: Four-stage DeepX compilation pipeline. Quantization to INT8 is mandatory for FPGA execution.

3.3.2 Target Hardware

The compiled `.dxnn` binaries are deployed directly onto the target embedded edge architecture: an **AMD Kria KV260 Vision AI Starter Kit**. The Kria System-on-Module (SOM) serves as the host processing platform, powered by a quad-core **ARM Cortex-A53** Application Processing Unit operating natively on the **aarch64** architecture.

To achieve the necessary computational acceleration for real-time inference without inducing thermal or power strains on the fanless system, the hardware configuration is augmented by integrating the **DeepX DX-M1 AI accelerator** via the carrier board's standard M.2 expansion slot. In this hybrid co-design, the low-power ARM CPU cores are relieved of heavy convolutional overhead and restricted to systemic orchestration tasks, while the specialized fixed-point INT8 systolic array of the DeepX NPU executes the neural network feature extraction graph.

3.4 Automatic Threshold Selection: 3-Sigma Integration

As will be discussed in Section 4.7, the availability of labeled defective data cannot be assumed in industrial settings. The GUI therefore implements automatic threshold calibration using the 3-sigma statistical method.

3.4.1 Automatic Calibration

During dataset acquisition, a configurable subset of the captured normal images is reserved exclusively for validation. The default configuration reserves 20% of acquired images for validation, enforcing a minimum lower bound of 100 images and a maximum upper cap of 500 images; the remainder (80%) are used for training.

The lower bound of 100 images is statistically mandatory to ensure the convergence and stability of the robust Median Absolute Deviation (MAD) estimator against random sampling noise. Conversely, the upper cap of 500 images is imposed as an empirical memory boundary; since PatchCore calibration requires storing large intermediate multi-scale feature tensors in the host RAM, capping the validation sample size prevents the calibration phase from triggering Out-Of-Memory (OOM) faults on the shared industrial PC hardware.

When the user initiates training, the system first checks whether any files labeled as defective (KO) are present in the dataset directory. If no KO files are found, the system automatically invokes the 3-sigma threshold calibration:

1. Anomaly scores are computed for all validation images using the trained model.
2. The mean $\hat{\mu}$ and standard deviation $\hat{\sigma}$ of these scores are calculated using the robust median/MAD estimator described in Section 4.7 [52].
3. The operational detection threshold is set to $T = \hat{\mu} + k\hat{\sigma}$, where $k = 3$ by default.

3.4.2 User Override: The k-Sigma Slider

While the 3-sigma default is statistically motivated, the user may wish to adjust the threshold based on observed false positive rates during production. The GUI provides a **k-sigma slider** (range: 1 to 6) in the inference panel that allows the operator to vary the multiplier applied to $\hat{\sigma}$.

- Lower values ($k = 1, 2$) produce a more sensitive detector (increases recall, risks higher false positives).
- Higher values ($k = 4, 5, 6$) produce a more conservative detector (reduces false positives, risks missed defects).

The slider operates in real time: adjusting k immediately updates the anomaly detection decision without retraining the model. This interactive exploration allows operators to empirically tune the sensitivity to their specific production requirements and defect tolerance. The default of $k = 3$ is automatically restored when a new receipt is loaded.

3.4.3 Integration with Inference

During real-time inference, the currently selected threshold (based on the loaded receipt and any subsequent k-sigma adjustments) is applied to the raw anomaly scores. Detected defects are highlighted on the display (circled in red), and the anomaly heatmap is overlaid. The FPS counter updates continuously, providing immediate feedback on the performance impact of any GUI operations.

4 PatchCore Architecture and Methodology

The comparative analysis presented in Section 2.3 identified PatchCore [51] as the optimal baseline for this thesis, offering superior accuracy on cable factory defects while presenting a clear architectural bottleneck: the memory bank and its associated k-NN search. This section provides a detailed exposition of the PatchCore architecture, establishing the mathematical and algorithmic foundation upon which the acceleration techniques of Chapter 5 are built.

4.1 High-Level Architecture Overview

PatchCore follows a three-stage pipeline, illustrated in Figure 4.1:

1. **Feature Extraction**: A pre-trained convolutional backbone extracts dense patch-level features from normal training images.

2. **Memory Bank Construction**: A greedy coreset subsampling algorithm selects a compact, representative subset of normal features for storage.

3. **Anomaly Scoring**: During inference, features from test images are compared against the memory bank using k-Nearest Neighbors search, producing both image-level anomaly scores and pixel-level localization maps.

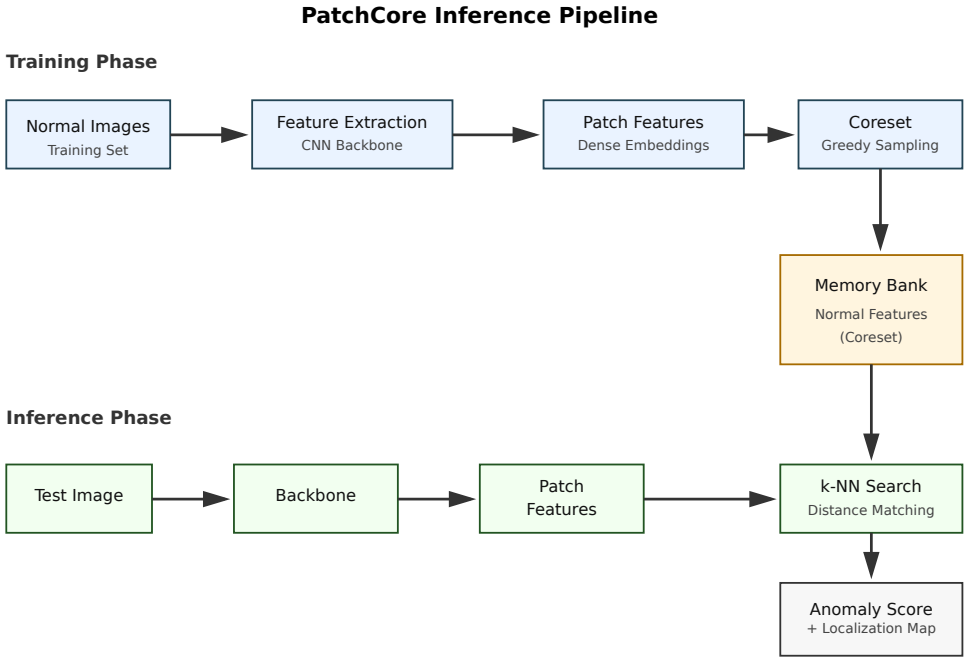


Figure 4.1: PatchCore inference pipeline. A pre-trained backbone extracts dense patch features from the input image. The features are compared against a memory bank containing coreset-selected normal features using k-NN search to compute anomaly scores and localization maps. Adapted from [51].

4.2 Feature Extraction with Pre-trained Backbone

PatchCore exploits the representational power of deep neural networks pre-trained on large-scale datasets. Specifically, the architecture employs a **WideResNet50** [73] backbone pre-trained on ImageNet [19]. The choice of a wide residual network is motivated by empirical evidence that increased width (number of channels per layer) improves anomaly detection performance more than increased depth [51].

Feature Pyramid Aggregation. Unlike methods that use only the final layer features, PatchCore aggregates features from multiple network stages to capture both local textures (shallow layers) and global structures (deep layers). Following the original implementation, features are extracted from two layers:

- **Layer2**: Output stride of 4 relative to input resolution. Features at this level capture fine-grained textures, edges, and local patterns, critical for detecting scratches and surface imperfections.
- **Layer3**: Output stride of 8. Features at this level capture higher-level structures, shapes, and component arrangements, essential for detecting taping phase errors and structural anomalies.

Let $\mathcal{I}$ denote a training image of dimensions $H \times W$. The backbone ϕ produces feature maps:

$$\mathbf{F}^{(2)} = \phi_2(\mathcal{I}) \in \mathbb{R}^{C_2 \times \frac{H}{4} \times \frac{W}{4}}, \quad \mathbf{F}^{(3)} = \phi_3(\mathcal{I}) \in \mathbb{R}^{C_3 \times \frac{H}{8} \times \frac{W}{8}}$$

where $C_2 = 512$ and $C_3 = 1024$ for WideResNet50.

Adaptive Pooling and Concatenation. To enable dense patch-level scoring, the feature maps are first spatially aligned via adaptive average pooling, then concatenated along the channel dimension. For a spatial location (i, j) in the final feature map (stride 8), the corresponding patch descriptor is:

$$\mathbf{p}_{ij} = \left[\text{pool}(\mathbf{F}^{(2)}_{ij}),\ \mathbf{F}^{(3)}_{ij}\right] \in \mathbb{R}^{C}$$

where $C = C_2' + C_3$ with C_2' denoting the pooled dimension (typically 256), yielding a final descriptor dimension $C = 1280$. This concatenated descriptor encodes multi-scale information around each spatial position [51].

Local Neighborhood Aggregation (LNA). A subtle but important detail of PatchCore is the Local Neighborhood Aggregation (LNA) step. Before storing features in the memory bank, each patch descriptor is averaged with its 3×3 spatial neighbors:

$$\mathbf{p}^{\text{LNA}}_{ij} = \frac{1}{9} \sum_{u=-1}^{1} \sum_{v=-1}^{1} \mathbf{p}_{i+u,j+v}$$

This smoothing operation increases robustness to local pixel-level perturbations and misalignments, which are inevitable in industrial settings due to micro-vibrations and optical distortions (see Section 1.4). As defined in the original formulation [51], this average pooling is applied with a stride of 1 and zero-padding. Consequently, it acts strictly as a spatial low-pass filter to expand the receptive field, without altering the output dimensionality or reducing the total number of patch descriptors.

4.3 Memory Bank Construction via Greedy Coreset Subsampling

The naive approach to memory bank construction would store all extracted patch descriptors from all training images. For a dataset of N training images, each producing approximately $\frac{H}{8} \times \frac{W}{8}$ patches (e.g., $28 \times 28 = 784$ patches for a 224×224 input), the total number of descriptors would be $N \times 784$. For $N = 500$ training images, this exceeds 390,000 descriptors, each of dimension 1280. The resulting memory footprint (approximately 2 GB in FP32) is prohibitive for edge deployment.

PatchCore addresses this through **greedy coreset subsampling** [51], an iterative selection algorithm that chooses a compact subset of patch descriptors maximally preserving the distribution of the full set.

Coreset Formulation. Let $\mathcal{P}_{\text{train}} = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_M\}$ denote the set of all patch descriptors extracted from training normal images, where $M = N \times \frac{H}{8} \times \frac{W}{8}$. The goal is to select a coreset $\mathcal{C} \subset \mathcal{P}_{\text{train}}$ of size $|\mathcal{C}| = m \ll M$ such that:

$$\min_{\mathcal{C}} \max_{\mathbf{p} \in \mathcal{P}_{\text{train}}} \min_{\mathbf{c} \in \mathcal{C}} \|\mathbf{p} - \mathbf{c}\|_2$$

This minimax formulation ensures that every point in the full set is close to some point in the coreset, preserving the overall distribution's coverage [55].

Greedy Selection Algorithm. The exact solution to the coreset problem is NP-hard. PatchCore employs a greedy approximation:

1. Initialize $\mathcal{C}$ with a randomly selected patch descriptor.
2. Iterate until $|\mathcal{C}| = m$:
   (a) For each remaining descriptor $\mathbf{p} \in \mathcal{P}_{\text{train}} \setminus \mathcal{C}$, compute its distance to the nearest already-selected coreset element:
   $$d(\mathbf{p}) = \min_{\mathbf{c} \in \mathcal{C}} \|\mathbf{p} - \mathbf{c}\|_2$$
   (b) Select the descriptor with the maximum $d(\mathbf{p})$ (i.e., the point farthest from the current coreset) and add it to $\mathcal{C}$.

This algorithm is equivalent to the classic "farthest point sampling" or "minimax facility location" greedy solution. Its time complexity is $O(m \cdot M)$, which is acceptable for offline training (seconds to minutes depending on dataset size) [55].

Coreset Ratio Selection. The original PatchCore paper recommends a subsampling ratio of 0.01 to 0.1 (i.e., retaining 1% to 10% of the original patch descriptors). For a memory budget of approximately 450 MB (FP32), $m \approx 50,000$ descriptors, corresponding to a ratio of approximately 0.1 for typical MVTec AD training splits. This reduction from 500,000+ descriptors to 50,000 preserves over 99% of the original AUROC performance while reducing memory by an order of magnitude [51].

4.4 Anomaly Scoring with k-Nearest Neighbors

During inference, a test image $\mathcal{I}_{\text{test}}$ is processed through the same feature extraction pipeline, producing a set of query patch descriptors $\mathcal{Q} = \{\mathbf{q}_{ij}\}$ for each spatial location (i, j) in the downsampled feature grid.

Patch-Level Anomaly Scores. For each query descriptor $\mathbf{q}$, the anomaly score is defined as the distance to its nearest neighbor in the coreset memory bank:

$$s(\mathbf{q}) = \min_{\mathbf{c} \in \mathcal{C}} \|\mathbf{q} - \mathbf{c}\|_2$$

PatchCore uses Euclidean (L2) distance rather than cosine similarity, as empirical evaluation found L2 to be more discriminative for the patch features extracted from WideResNet [51]. To improve robustness, the score is computed as the average distance to the k nearest neighbors rather than the single nearest neighbor:

$$s_k(\mathbf{q}) = \frac{1}{k} \sum_{j=1}^{k} \|\mathbf{q} - \text{NN}_j(\mathbf{q})\|_2$$

where $\text{NN}_j(\mathbf{q})$ denotes the j-th nearest neighbor in $\mathcal{C}$. The original PatchCore implementation uses $k = 9$ [51].

Anomaly Heatmap Generation. Applying this scoring function to each spatial location yields a dense anomaly heatmap $\mathcal{H}_{\text{patch}} \in \mathbb{R}^{\frac{H}{8} \times \frac{W}{8}}$. This heatmap must be upsampled to the original input resolution for pixel-level localization:

$$\mathcal{H}_{\text{pixel}} = \text{upsample}(\mathcal{H}_{\text{patch}}, \text{size} = (H, W))$$

Bilinear interpolation is used for upsampling, as it produces smooth anomaly boundaries without introducing artifacts [51].

Image-Level Anomaly Score. The global image-level score is derived from the patch-level heatmap. While a simple max-pooling operation (taking the maximum patch score) would suffice, PatchCore employs a more robust approach:

$$s_{\text{image}} = \max_{i,j} \mathcal{H}_{\text{patch},ij} \times \frac{1}{|\text{Top}(t, \mathcal{H}_{\text{patch}})|} \sum_{(i,j) \in \text{Top}(t, \mathcal{H}_{\text{patch}})} \mathcal{H}_{\text{patch},ij}$$

where $\text{Top}(t, \cdot)$ denotes the set of patch positions within the top t percentile of anomaly scores. This formulation avoids over-sensitivity to isolated outlier patches while remaining responsive to genuinely anomalous regions [51]. In practice, $t = 0.01$ (top 1%).

4.5 Complexity Analysis and Bottleneck Identification

Understanding the computational complexity of each Pipeline stage is essential for identifying acceleration opportunities. Let:

- N_{train}: Number of training images
- P: Number of patch descriptors per image ($\approx H/8 \times W/8$)
- m: Coreset size ($\approx 0.1 \times N_{\text{train}} \times P$)
- D: Feature descriptor dimension (1280 for WideResNet50)
- d_{test}: Number of query patches per test image (approximately P)

Feature Extraction (Training + Inference). The backbone CNN processes each image once, regardless of subsequent operations. For WideResNet50, each forward pass requires 11.4 GFLOPs, translating to 20–50 ms on CPU or 5–10 ms on GPU. This is a fixed cost per image.

Memory Bank Construction (Training Only). The greedy coreset algorithm has complexity $O(m \cdot N_{\text{train}} \cdot P \cdot D)$. For typical parameters ($m = 50,000$, $N_{\text{train}} \cdot P = 500,000$, $D = 1280$), this yields approximately 3.2×10^{13} distance computations. However, this cost is incurred once during training and is not a constraint during inference.

k-NN Search (Inference Bottleneck at Scale). For each test image, the anomaly scoring stage requires a distance computation between query patches and the memory bank:

$$\text{Complexity} = d_{\text{test}} \cdot m \cdot D$$

While this stage scales linearly with the coreset size m, the preceding feature extraction phase relies on heavy dense matrix multiplications. To quantify the real-world impact of these phases, a detailed profiling script was executed on a specific host CPU (**AMD Ryzen 7 5800X**, a 15W mobile processor typical of mid-range industrial edge interfaces) using a WideResNet50 backbone and a compact memory bank ($m = 2,293$ patch vectors). The empirical results are detailed in Table 4.1.

Pipeline Stage	**Mean Time (ms)**	**Std Dev (ms)**	**Contribution (%)**
Preprocessing	0.20	0.05	0.2%
Feature Extraction	81.90	12.88	69.6%
k-NN Search & Scoring	35.34	3.90	30.0%
Post-processing	0.29	0.08	0.2%
Total Inference Time	**117.73**	**15.36**	**100.0%**

Table 4.1: Detailed inference time breakdown of the PatchCore algorithm executed on a standard CPU. The memory bank contains 2,293 patch vectors.

4.6 Computational Bottleneck Summary and Motivation

The empirical profiling presented in Table 4.1 provides a crucial dual insight that dictates the architectural design of this thesis:

1. **The CNN Bottleneck (Justifying Hardware Acceleration):** On standard CPU hardware, the Convolutional Neural Network feature extraction consumes nearly 70% of the inference budget (81.90 ms). This massive computational load perfectly justifies the integration of a dedicated AI accelerator (such as the DeepX NPU), whose systolic array architecture is specifically designed to slash dense convolutional workloads down to mere milliseconds.

2. **The k-NN Scaling Bottleneck (Justifying Algorithmic Optimization):** While the exact k-NN search only accounts for 35.34 ms in this specific test, this measurement utilized a highly compressed memory bank ($m = 2,293$). When scaling to a full industrial memory bank ($m \approx 50,000$ vectors), the $O(m \cdot D)$ search complexity grows linearly, easily exceeding hundreds of milliseconds. Once the NPU accelerates the CNN extraction, this exact k-NN search mathematically becomes the absolute bottleneck of the system.

This analysis directly motivates the hardware-software co-design strategy developed in Chapter 5:

1. **Approximate Nearest Neighbor Search (Section 5.2)**: Replace exact k-NN search with Hierarchical Navigable Small World (HNSW) graphs [38], reducing search complexity from $O(m \cdot D)$ to $O(\log m \cdot D)$.

2. **Hard Prototype Mining (Section 5.3)**: Drastically reduce the memory bank size m by intelligently selecting only the most informative prototypes, mitigating the scaling problem at its root.

3. **Complete Architectural Replacement (Section 5.4)**: Replace the memory bank entirely with a teacher-student Reverse Distillation network, eliminating the search step completely in favor of pure feedforward NPU execution.

The following chapter describes the system architecture designed to integrate these optimizations into a cohesive, user-friendly industrial application.

4.7 Threshold Selection: The 3-Sigma Method

A critical practical challenge in deploying anomaly detection systems in industrial settings is the determination of the decision threshold, that is the value above which an anomaly score is classified as a defect. In laboratory settings, this threshold is typically optimized using a validation set containing both normal and anomalous samples, maximizing metrics such as the F1-score or the true positive rate at a specified false positive rate. However, this approach is fundamentally incompatible with the industrial constraints described in Section 1.1: labeled defective samples (a "KO dataset") are rarely available, particularly during the initial deployment of a quality control system for a new product variant [9, 16].

This section addresses the question: *How can a reliable anomaly threshold be established using only normal training data?* The proposed solution, implemented in this thesis, is the **3-sigma method**, which assumes that anomaly scores on normal validation samples follow a Gaussian distribution and sets the threshold at three standard deviations above the mean.

4.7.1 The Absence of Labeled Defective Data

The difficulty of obtaining labeled defective samples in manufacturing contexts cannot be overstated. There are three principal reasons:

Statistical Rarity. Under Zero-Defect Manufacturing policies, the defect rate is typically maintained below 0.1%. Collecting even 100 defective samples would therefore require producing or inspecting more than 100,000 units, which is prohibitively expensive, particularly for high-value cable products.

Open-Ended Anomaly Space. As discussed in Section 1.1, the concept of "anomaly" is by definition open-ended. A supervised classifier trained on a limited set of known defects would lack generalization capabilities and fail to recognize novel defect types that may emerge due to tool wear, material batch variations, or operator errors.

Annotation Cost. Even if defective samples could be collected, pixel-level annotation (required for training segmentation models or evaluating localization performance) is extremely labor-intensive. For complex textures or small defects, expert annotators may require several minutes per image [9].

In the specific context of the WTM cable factory, no systematic collection of defective samples was available at the start of this thesis. The production line had no existing automated inspection system, and manual inspection records were not maintained in a format suitable for machine learning training. This constraint motivated the development of a thresholding method that operates exclusively on normal data.

The 3-Sigma Rule: Mathematical Formulation

The 3-sigma method is rooted in statistical process control, where control charts are used to monitor whether a process is in a state of statistical control [41]. For a normally distributed variable, approximately 99.73% of observations fall within three standard deviations of the mean:

$$P(\mu - 3\sigma \leq X \leq \mu + 3\sigma) \approx 0.9973$$

Leaving a total probability of 0.27% for the extreme values in both tails combined.

Applied to anomaly detection, the method assumes that anomaly scores computed on normal validation images follow a Gaussian distribution. Since anomalies are characterized by unusually high scores, we are only interested in the right tail of the distribution. Under this assumption, setting the threshold at $\mu + 3\sigma$ yields a theoretical false positive rate of approximately 0.135%, the exact proportion of normal samples that fall above this upper limit (as illustrated in Figure 4.2).

Let $\mathcal{V} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$ be a set of n normal validation images, distinct from the training set. For each validation image, the anomaly detection model produces a scalar anomaly score $s_i = f(\mathbf{x}_i)$, where f denotes the model (e.g., PatchCore). The sample mean and standard deviation are computed as:

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} s_i, \quad \hat{\sigma} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(s_i - \hat{\mu})^2}$$

The anomaly threshold is then set to:

$$T = \hat{\mu} + k \cdot \hat{\sigma}$$

where $k = 3$ for the standard 3-sigma rule. During inference, an image is classified as anomalous if its anomaly score exceeds T.

Verification of the Gaussian Assumption

The validity of the 3-sigma method depends critically on whether anomaly scores from normal images are approximately Gaussian. This assumption is not guaranteed a priori; it must be empirically verified for each model and dataset.

To rigorously validate this assumption, a statistical analysis was performed on the anomaly scores extracted by PatchCore from normal test images of the MVTec AD dataset. Three complementary analyses are presented below.

Empirical Distribution vs. Normal Fit. Figure 4.3 illustrates the histogram of the empirical anomaly scores for the *bottle* category overlaid with its theoretical Normal Fit ($\mu = 24.7$, $\sigma = 2.1$). The empirical probability density closely follows the theoretical bell curve. Although a slight positive skew is observable, the majority of the distribution remains highly concentrated around the mean, indicating that the Gaussian approximation is appropriate.

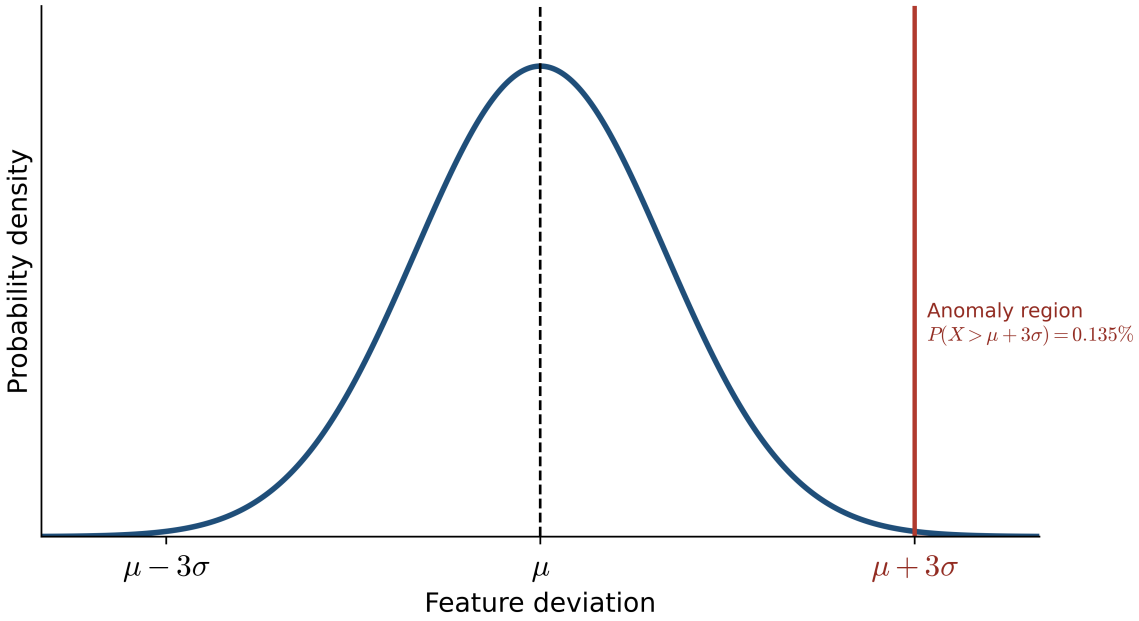


Figure 4.2: Gaussian distribution of anomaly scores with mean μ and standard deviation σ. While the $\pm 3\sigma$ range conceptually represents standard normal limits, anomaly detection relies only on the upper bound. The right tail beyond the $\mu + 3\sigma$ threshold corresponds to a theoretical false positive rate of 0.135%.

Quantile-Quantile (Q-Q) Plot Analysis. Figure 4.4 evaluates the behavior of the distribution tails, which are the most critical region for anomaly thresholding. The empirical quantiles align closely with the theoretical Gaussian quantiles along the identity line, confirming that deviations from normality are minimal, particularly in the upper tail where the 3σ threshold is applied.

Cross-Category Generalization. Figure 4.5 demonstrates that the observed Gaussian behavior is not restricted to a single object class. Despite differences in mean and variance, the anomaly score distributions for the *bottle*, *cable*, and *transistor* categories all retain the characteristic bell shape. This consistency provides empirical evidence supporting the applicability of the 3-sigma rule across heterogeneous industrial inspection scenarios.

Caveats and Limitations. The Gaussian assumption may fail under certain conditions:

- **Multimodal normal data**: If the normal training data contains multiple distinct subpopulations (e.g., different cable color variants), the anomaly score distribution may be multimodal, violating the Gaussian assumption.
- **Saturation effects**: For high-dimensional feature spaces, anomaly scores may exhibit heavy tails that are poorly approximated by a Gaussian.
- **Model-specific characteristics**: Some anomaly detection models produce score distributions that are intrinsically non-Gaussian (e.g., reconstruction-based methods with bimodal error distributions).

In cases where the Gaussian assumption fails, alternative thresholding methods are available (see Subsection 4.7.2). However, for the PatchCore model on the cable factory dataset, empirical validation confirmed that the Gaussian approximation is reasonable, with the calculated 3-sigma threshold yielding acceptable false positive rates in practice.

Practical Considerations for Implementation

Several practical considerations inform the implementation of the 3-sigma method in this thesis:

Validation Set Size. The accuracy of the estimated mean and standard deviation improves with larger validation sets. To reduce sampling error, a minimum of 100 normal validation images is

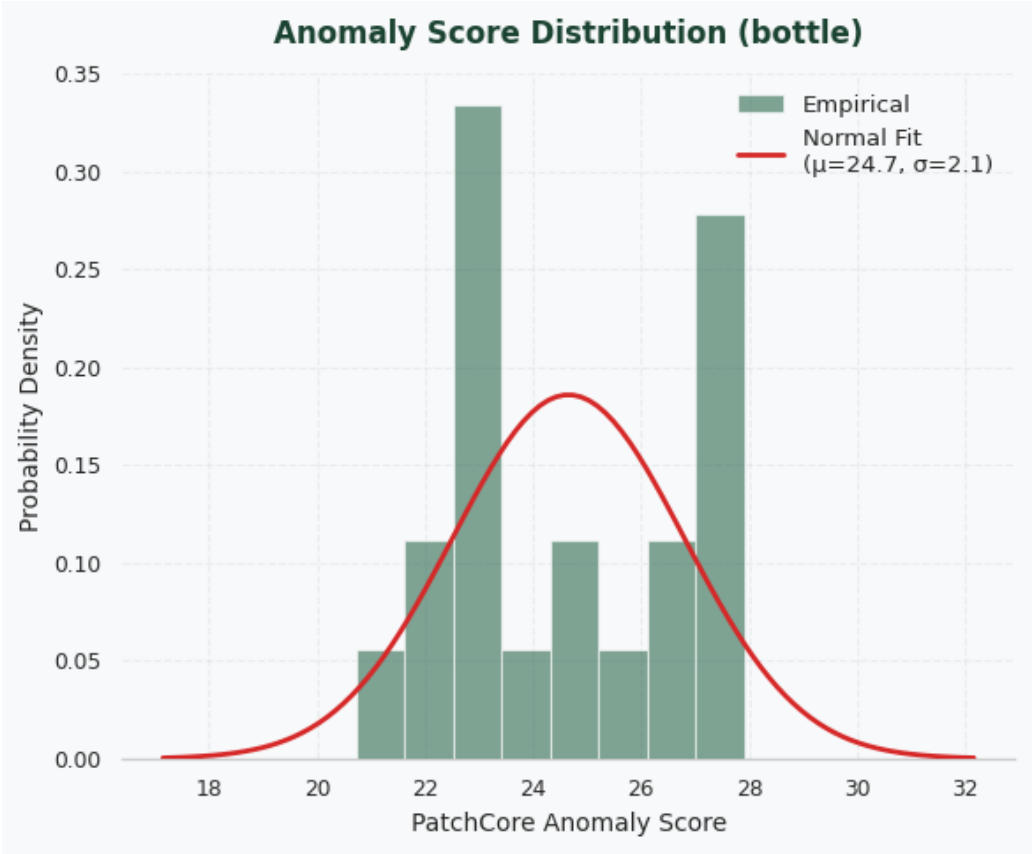


Figure 4.3: Empirical distribution of anomaly scores for the *bottle* category overlaid with the fitted Gaussian distribution.

recommended. In this work, 200 normal images (approximately 20% of the training data) are reserved for threshold calibration.

Outlier Mitigation. Anomaly scores for normal images occasionally exhibit extreme outliers due to image artifacts, sensor noise, or preprocessing errors. These outliers can artificially inflate $\hat{\sigma}$ and shift the threshold. To mitigate this, a robust estimator is employed:

$$\hat{\mu}_{\text{robust}} = \text{median}(s_i), \quad \hat{\sigma}_{\text{robust}} = 1.4826 \times \text{MAD}(s_i)$$

where MAD is the Median Absolute Deviation[52]. The scaling factor 1.4826 ensures consistency with the standard deviation for Gaussian distributions.

Bias-Correction for Small Validation Sets. For validation sets smaller than 200 images, a bias-corrected threshold is used:

$$T = \hat{\mu} + t_{n-1,\,0.00135} \cdot \hat{\sigma} \cdot \sqrt{1 + \frac{1}{n}}$$

where $t_{n-1,\,0.00135}$ is the critical value from the Student's t-distribution corresponding to the 0.135% tail (equivalent to 3-sigma for the Gaussian case). This correction accounts for the uncertainty in the estimated parameters.

4.7.2 Alternative Thresholding Methods

For contexts where the Gaussian assumption is violated, several alternative thresholding methods have been proposed:

Percentile Method. The simplest non-parametric approach sets the threshold at the $(100-\alpha)$-th percentile of the validation anomaly scores, where α is the desired false positive rate (e.g., $\alpha = 0.1\%$ for the 99.9th percentile). This method makes no distributional assumptions but requires a sufficiently large validation set to reliably estimate extreme percentiles [69].

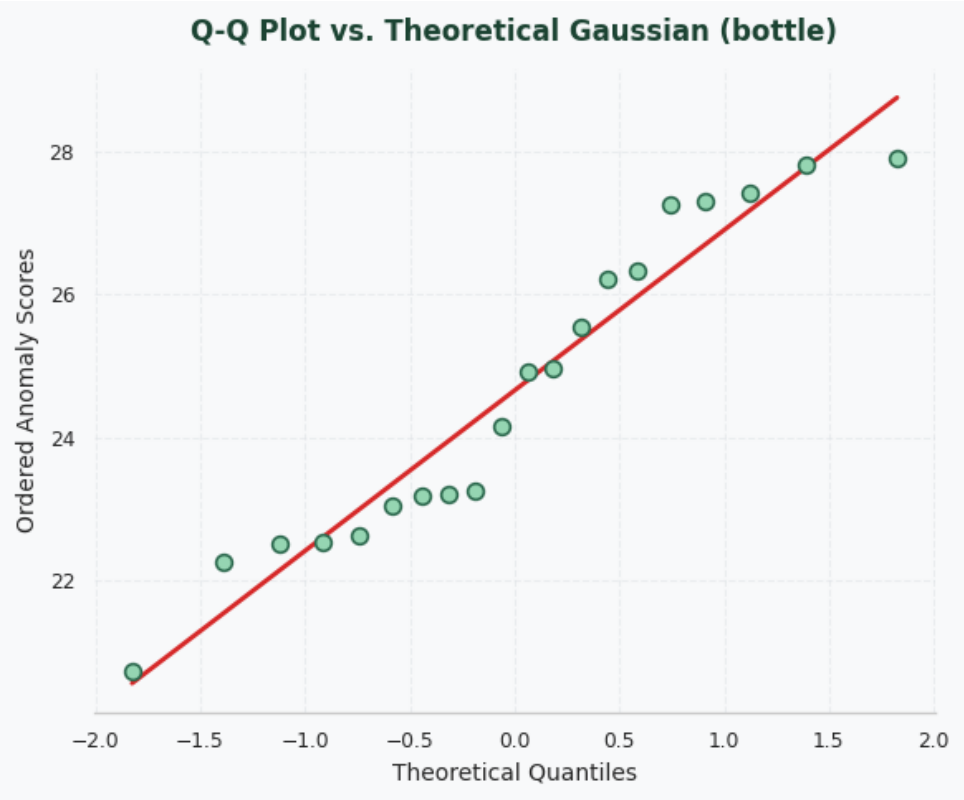


Figure 4.4: Q-Q plot comparing empirical anomaly scores with the theoretical Gaussian quantiles.

Extreme Value Theory (EVT). EVT provides a principled framework for modeling the tails of distributions, specifically for estimating the probability of values beyond the maximum observed in the validation set [15]. The Peak-Over-Threshold (POT) method fits a Generalized Pareto Distribution (GPD) to exceedances above a high threshold, enabling extrapolation to even more extreme anomaly scores. EVT has been successfully applied to anomaly detection thresholding [57], though its implementation complexity is higher than percentile or 3-sigma methods.

Q-Statistic. Anantrasirichai and Bull [3] proposed a method based on the Q-statistic from principal component analysis. Anomalies are detected when the squared prediction error exceeds a threshold determined from the Wishart distribution of the residuals. This approach assumes that normal data lie in a low-dimensional subspace, a property that holds for many feature-embedding anomaly detection methods.

Adaptive Thresholding. For production environments where the false positive rate can be observed in real time, adaptive methods that adjust the threshold based on recent predictions are feasible. These methods require an interactive deployment setting, however, which was not available for this thesis.

Method	Distribution Assumption	Data Requirement	Complexity
3-Sigma	Gaussian (or approximate)	100+ normal	Low
Percentile	None	500+ normal	Low
Extreme Value Theory	None (tail-only)	1000+ normal	Medium
Q-Statistic	Low-rank residuals	100+ normal	Low
Adaptive	None (online)	Real-time feedback	High

Table 4.2: Comparison of threshold selection methods for unsupervised anomaly detection. The 3-sigma method offers a favorable balance of simplicity, low data requirements, and reasonable performance when the Gaussian assumption holds.

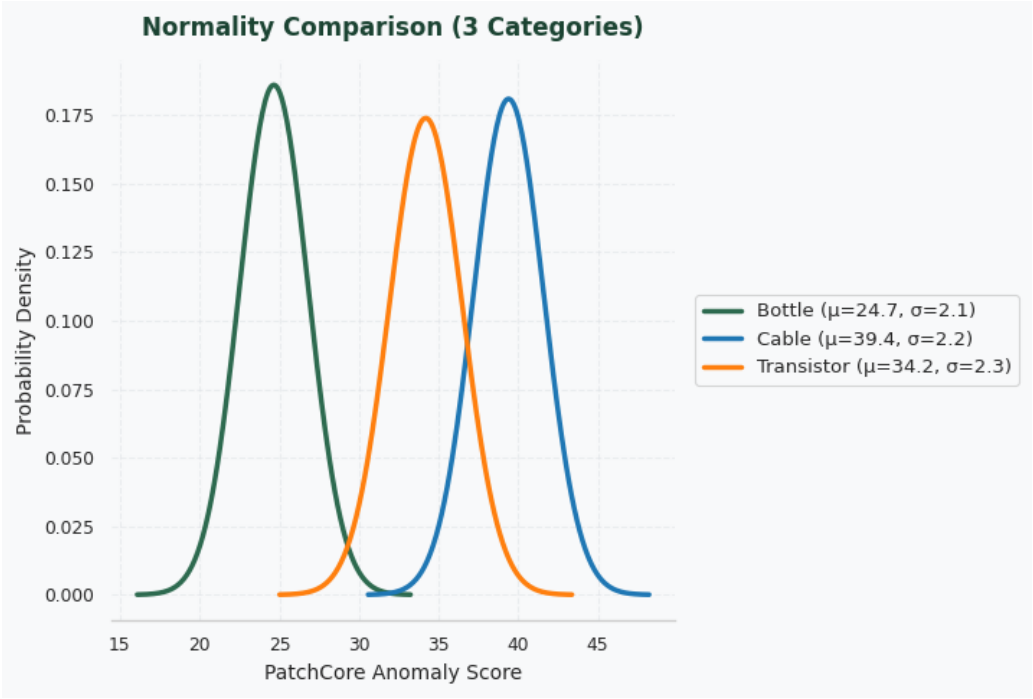


Figure 4.5: Comparison of anomaly score distributions across different MVTec AD categories, showing consistent Gaussian behavior despite different means and variances.

Limitations and When to Avoid 3-Sigma

The 3-sigma method is not universally applicable. It should be avoided in the following scenarios:

Multimodal Normal Distributions. If normal validation images contain multiple distinct subpopulations (e.g., products of different colors or shapes that are all considered "normal"), the anomaly score distribution will be multimodal. In this case, a validation set stratified by subpopulation is required, with separate thresholds for each mode.

Extremely High Reliability Requirements. For safety-critical applications where false negatives are unacceptable and false positives must be minimized (e.g., aerospace components), the theoretical false positive rate of 0.27% may be insufficient. In such cases, the percentile method with a higher threshold (e.g., 99.9th percentile) or EVT-based extrapolation is preferred.

Very Small Validation Sets ($n < 50$). With fewer than 50 validation samples, the sampling error in $\hat{\mu}$ and $\hat{\sigma}$ is too large to reliably set a threshold. The robust estimator provides limited improvement at these sample sizes. The percentile method is also unreliable at small n; EVT is not recommended below $n = 1000$ for tail estimation. In this regime, manual threshold tuning using interactive exploration may be necessary.

In conclusion the threshold determined by the 3-sigma method is automatically computed within the GUI described in Chapter 3, requiring no manual intervention from the factory operator. This integration enables non-expert users to deploy anomaly detection systems without the machine learning expertise traditionally required for threshold calibration.

5 Architectural Optimizations for Edge Inference

This chapter presents three complementary architectural optimizations designed to accelerate PatchCore for edge deployment. As demonstrated by the empirical profiling in Chapter 2, while PatchCore provides superior accuracy for the target industrial application, its exact k-Nearest Neighbors (k-NN) search over a massive memory bank creates a severe computational bottleneck. When scaled to industrial requirements, this linear search renders the baseline implementation unsuitable for inline inspection on the target edge hardware: an AMD Kria System-on-Module (SOM) acting as the host processor for the DeepX DX-M1 NPU.

The optimizations described in this chapter specifically address this structural flaw. The chapter is organized as follows. Section 5.2 introduces approximate nearest neighbor search using Hierarchical Navigable Small World (HNSW) graphs, reducing search complexity from linear to logarithmic. Section 5.3 describes Hard Prototype Mining (HPM), a technique that reduces the physical memory bank size while preserving discriminative power. Finally, Section 5.4 explores the most radical optimization: the complete replacement of the memory bank with a teacher-student following the idea of Reverse Distillation (RD) architecture, trading memory allocation for pure feedforward computation on the NPU.

5.1 The Three Optimization Axes

The k-NN memory bank bottleneck can be systematically attacked along three independent mathematical and architectural axes, illustrated in Figure 5.1:

1. **Reduce search complexity (Solution 1)**: Replace the exact linear search with an approximate nearest neighbor search using hierarchical graph structures (HNSW), reducing the per-query complexity from $O(m)$ to $O(\log m)$.

2. **Reduce memory bank size (Solution 2)**: Replace the standard coreset subsampling algorithm with Hard Prototype Mining. By systematically selecting a smaller, more informative subset of boundary features, this technique directly shrinks the physical memory footprint and mitigates the linear factor m in the search complexity.

3. **Eliminate the memory bank entirely (Solution 3)**: Replace the explicit memory bank with an implicit neural representation via Reverse Distillation. This completely bypasses the CPU search constraints by translating the anomaly detection task into pure feedforward matrix multiplications, ideally suited for the DeepX NPU's systolic array.

The following sections present the theoretical foundation, mathematical formulation, and algorithmic description of each optimization strategy.

5.2 Approximate Nearest Neighbor Search with HNSW

The first optimization targets the search complexity directly, without modifying the memory bank's contents. Instead of performing an exact linear scan over all m memory bank descriptors, we employ an approximate nearest neighbor search algorithm based on Hierarchical Navigable Small World (HNSW) graphs [38].

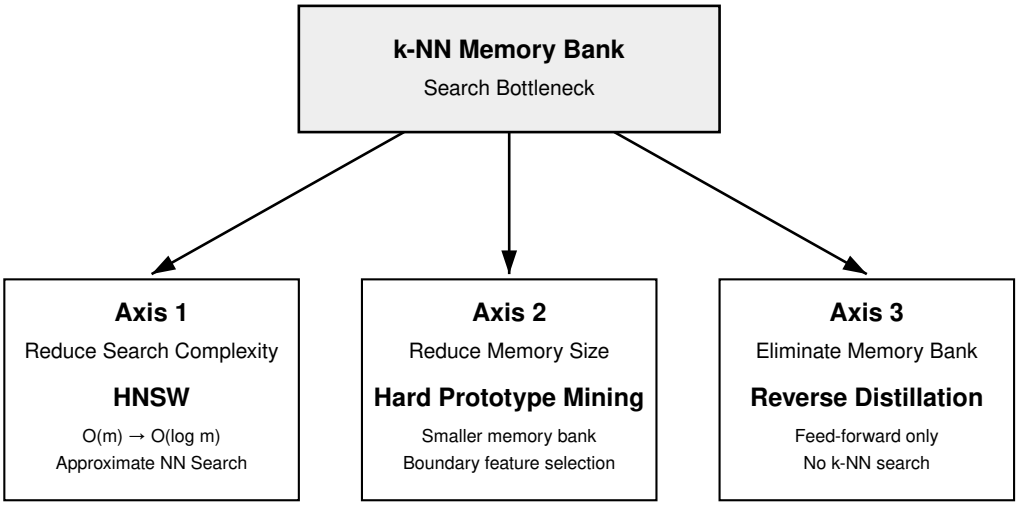


Figure 5.1: Three complementary optimization axes for accelerating PatchCore inference. The k-NN memory bank bottleneck can be addressed by reducing search complexity through HNSW, reducing memory bank size through hard prototype mining, or completely removing the memory bank using reverse distillation.

5.2.1 From Exact k-NN to Approximate k-NN

Exact k-NN search has time complexity $O(m \cdot D)$ per query, requiring the computation of all pairwise distances:

$$d(\mathbf{q}, \mathbf{c}_i) = \|\mathbf{q} - \mathbf{c}_i\|_2 \quad \forall i \in \{1, \ldots, m\}$$

Approximate nearest neighbor (ANN) search relaxes the requirement of returning the exact nearest neighbor, accepting a bounded error in exchange for substantially faster query times. Most ANN algorithms pre-process the dataset into an index structure that enables sublinear query complexity, typically $O(\log m)$ or $O(D \cdot \log m)$ rather than $O(m \cdot D)$.

Among the available ANN algorithms (including Locality-Sensitive Hashing, KD-trees, and product quantization), HNSW was selected for three reasons:

1. **High recall at practical search complexity**: HNSW consistently achieves > 0.99 recall with $10 - 100\times$ speedups over exact search.
2. **No training phase**: Unlike product quantization, HNSW requires no data-dependent optimization; the index can be built directly from the memory bank.
3. **Deterministic routing**: The search path is deterministic given the graph structure, avoiding the non-deterministic jitter of hashing-based methods.

5.2.2 HNSW Fundamentals

HNSW combines two ideas: navigable small world graphs and hierarchical skip lists.

Navigable Small World Graphs

A navigable small world (NSW) graph is an undirected graph where each node (representing a feature descriptor $\mathbf{c}_i$) is connected to a small set of neighbors. The graph exhibits two properties:

- **Small world property**: The average shortest path between any two nodes grows logarithmically with the number of nodes.
- **Navigability**: Greedy routing, moving to the neighbor closest to the query, converges to the global nearest neighbor efficiently.

During query processing, we start from a random or fixed entry point and iteratively move to the neighbor with the smallest distance to the query vector:

$$\text{current} = \arg \min_{\mathbf{n} \in \text{neighbors(current)}} \|\mathbf{q} - \mathbf{n}\|_2$$

until no improving neighbor exists. The path length is $O(\log m)$ under the small world property.

Hierarchical Layering

HNSW extends the NSW graph with multiple layers organized as a hierarchy (Figure 5.2). The bottom layer (layer 0) contains all m descriptors and is densely connected. Each higher layer contains a sparse subset of nodes from the layer below, with exponentially decreasing density.

The probability that a node appears in layer l is proportional to $\exp(-l/\lambda)$, where λ is a decay constant (typically $\lambda = 1/\ln(m)$). This produces a structure analogous to a skip list:

- The top layer contains very few nodes (often $\sqrt{m}$ or fewer).
- Each layer acts as a "highway" for long-range navigation.
- Searches start at the top layer and cascade downward.

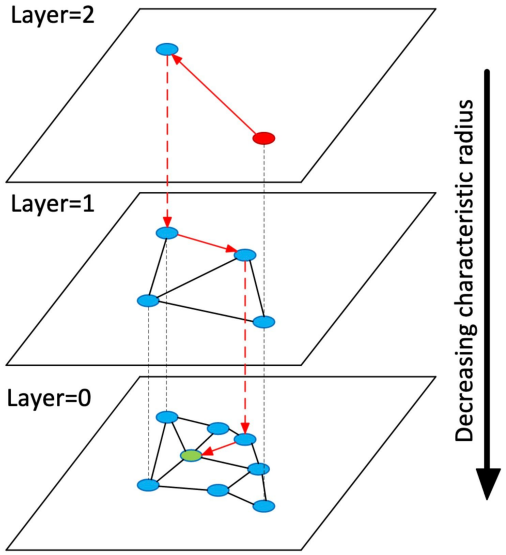


Figure 5.2: Illustration of the Hierarchical NSW idea. The search starts from an element from the top layer (shown red). Red arrows show the direction of the greedy algorithm from the entry point to the query (shown green). Reproduced from [38].

5.2.3 Index Construction

Given the memory bank $\mathcal{C} = \{\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_m\}$, the HNSW index is built as follows:

The key parameters are:

- $M_{\max}$: Maximum number of bidirectional connections per node (typically 16–64). Larger $M_{\max}$ improves recall but increases index size.
- $efConstruction$: Size of the dynamic candidate list during construction (typically 100–500). Larger values produce higher-quality graphs but increase build time.
- λ: Layer density decay constant, typically $1/\ln(m)$.

For a PatchCore coreset of $m = 50,000$, construction requires less than a minute on a standard CPU. This is a one-time cost paid during training, preserving the system's rapid setup time for factory operators.

Algorithm 1 HNSW Index Construction

Require: Memory bank $\mathcal{C}$, distance function $d(\cdot, \cdot)$, maximum layer $L_{\max}$
Ensure: HNSW index I

```
Initialize empty graph layers I_0, I_1, ..., I_{L_max}
for each c_i ∈ C do
    Sample layer l_i ~ Exponential(1/λ), clamped to [0, L_max]
    Insert c_i into layers I_0, ..., I_{l_i}
    for l = l_i downto 0 do
        Find efConstruction nearest neighbors in layer l via greedy search
        Connect c_i to the nearest M_max neighbors (bidirectional)
    end for
end for
return I = {I_0, I_1, ..., I_{L_max}}
```

5.2.4 Query Processing

Given a query descriptor $\mathbf{q}$ (a test image patch), the HNSW search proceeds as follows:

Algorithm 2 HNSW Query Processing

Require: Query $\mathbf{q}$, HNSW index I, parameter $efSearch$
Ensure: Nearest neighbor $\mathbf{c}^*$ and distance

```
Initialize priority queue W (bounded by efSearch)
Start at entry point e in top layer I_{L_max}
for l = L_max downto 1 do
    Perform greedy search in layer l, updating W
    Set entry point = best node in W for next layer
end for
Perform greedy search in layer 0, expanding W
return c* = arg min_{c∈W} d(q, c)
```

The parameter $efSearch$ controls the quality-sensitivity trade-off: lower values provide extreme speed at the cost of recall, while higher values (e.g., 50-100) provide near-exact accuracy while still vastly outperforming a linear scan. Because PatchCore requires $k = 9$ nearest neighbors, HNSW supports this naturally by continuing the search and returning the top k candidates from the final priority queue W.

5.2.5 Integration with PatchCore

The integration of HNSW into PatchCore replaces the exact k-NN search step:

1. **Training phase (modified)**: After computing the coreset memory bank $\mathcal{C}$, build an HNSW index over $\mathcal{C}$ using Algorithm 1. Store both the memory bank and the index in the SQLite receipt system.

2. **Inference phase (modified)**: For each query patch $\mathbf{q}$, retrieve the k approximate nearest neighbors using Algorithm 2 with $efSearch$ tuned for the desired speed-accuracy trade-off. Compute the anomaly score as the average distance to these k neighbors, identical to the original PatchCore logic.

5.2.6 Complexity Analysis and The Recall Trade-off

The theoretical complexity improvements of HNSW over exact search are substantial:

This theoretical reduction translates to massive empirical speedups, bringing the inference time significantly closer to the edge deployment requirements.

However, this speedup is fundamentally a trade-off against **Recall**. Because HNSW traverses a graph rather than exhaustively scanning the memory bank, there is a non-zero probability that the

Metric	Exact k-NN	HNSW (Approximate)
Search complexity per query	$O(m \cdot D)$	$O(\log m \cdot D)$
Distance computations per patch	$\sim 50,000$	$\approx 100 - 200$ (with $efSearch = 50$)
Total distance ops per image	39.2×10^6	$\sim 0.15 \times 10^6$

Table 5.1: Comparison of exact k-NN versus approximate HNSW search complexity.

true nearest normal prototype is missed, potentially inflating the anomaly score of a normal patch (a false positive).

For industrial anomaly detection, this trade-off is highly manageable. Because the final anomaly score aggregates distances across a spatial neighborhood and relies on robust thresholding (as defined in the 3-Sigma method), the system is inherently resilient to minor recall degradations at the patch level. The minimal loss in precision is practically justified by the orders-of-magnitude reduction in latency, establishing HNSW as a mandatory baseline optimization for edge environments.

5.3 Memory Footprint Reduction via Hard Prototype Mining

The second optimization reduces the memory bank size m itself, directly lowering both the memory footprint and the number of distance computations (whether exact or approximate). While PatchCore's greedy coreset subsampling already reduces the memory bank to approximately 10% of the original features, we propose a more discriminative selection strategy: **Hard Prototype Mining (HPM)**.

5.3.1 Motivation: Coreset Limitations

The greedy coreset algorithm selects a subset that maximizes coverage of the feature distribution: it chooses points farthest from existing selections, ensuring that no region of the normal feature manifold is left unrepresented. This coverage-centric strategy has two fundamental limitations:

1. **Redundancy in dense regions**: Regions of the feature manifold with high sample density are over-represented, contributing little discriminative value while artificially inflating m.

2. **Lack of separation awareness**: Coreset selection does not consider where anomaly features might fall; it treats all regions of the normal manifold as equally important.

Hard prototype mining addresses these limitations by selecting features that are *most discriminative* with respect to potential anomalies, rather than those that simply maximize coverage.

5.3.2 Concept of Hard Prototypes

In the context of anomaly detection, a **hard prototype** is a normal feature descriptor that lies close to the boundary of the normal manifold: the "danger zone" where anomalies might be confused with normal variations. The theoretical intuition is that if a normal feature is already close to the abnormal region, it provides the most informative basis for distinguishing future anomalies.

5.3.3 Hard Prototype Mining Algorithm

Because we do not have access to real anomaly features during training (the strictly unsupervised setting), we must approximate the decision boundary using only normal data. To maintain mathematical consistency, we define a scalar function Hardness($\mathbf{c}_i$) such that **higher values always indicate harder prototypes**. We employ two strategies to estimate this metric:

Strategy 1: Density-Based Boundary Estimation

Features in low-density regions of the normal feature manifold are statistically more likely to be close to the anomaly space. The inverse density at a feature point $\mathbf{c}_i$ is estimated as the average distance to its t nearest neighbors in the full training set. Therefore, the hardness is directly proportional to

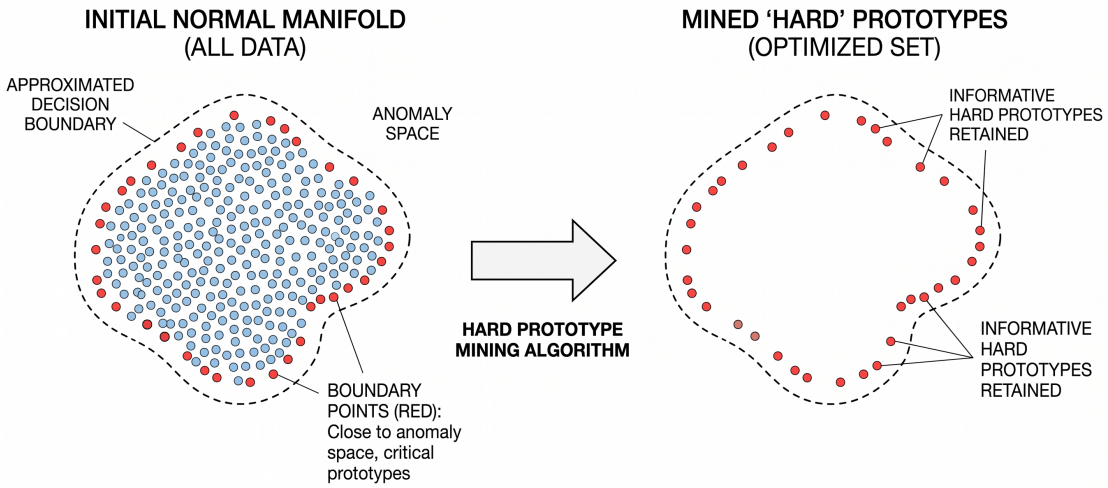


Figure 5.3: Hard prototypes (red points) lie near the decision boundary between normal and anomaly features. Internal normal features (light blue) can be safely discarded without losing discriminative power.

this distance:

$$\text{Hardness}_{\text{density}}(\mathbf{c}_i) = \frac{1}{t}\sum_{j=1}^{t} \|\mathbf{c}_i - \text{NN}_j(\mathbf{c}_i)\|_2$$

A high hardness score indicates that the point is isolated and likely resides near the manifold boundary.

Strategy 2: Synthetic Anomaly Boundary (DRAEM-inspired)

Following the paradigm introduced by DRAEM [74], we can simulate anomalies during the prototype mining phase by pasting synthetic patterns (e.g., Perlin noise textures) onto normal images. Features extracted from these corrupted regions serve as "pseudo-anomaly" samples in a set $\mathcal{A}$. A normal feature is considered "hard" if it lies geometrically close to these pseudo-anomalies.

To mathematically align this with the hardness maximization principle, we define the hardness score as the inverse of the minimum Euclidean distance to any pseudo-anomaly:

$$\text{Hardness}_{\text{synthetic}}(\mathbf{c}_i) = \frac{1}{\min_{\mathbf{a}\in\mathcal{A}} \|\mathbf{c}_i - \mathbf{a}\|_2 + \epsilon}$$

where ϵ is a small constant to prevent division by zero. Thus, normal features that are geometrically closest to pseudo-anomalies yield the highest hardness scores.

5.3.4 Integration with PatchCore

Hard prototype mining elegantly replaces the greedy coreset subsampling step in the standard PatchCore training pipeline:

1. Extract all patch descriptors $\mathcal{P}_{\text{train}}$ from normal training images.
2. Compute hardness scores for each descriptor using either density-based estimation (Strategy 1) or synthetic anomaly simulation (Strategy 2).
3. Select the top m_{hard} descriptors with the **highest hardness scores**.
4. *(Optional)* Apply a spatial diversity constraint to prevent over-selection from a single boundary cluster: after selecting a candidate, remove its r-neighborhood from further consideration.

5. Build the HNSW index (Section 5.2) on the heavily reduced memory bank of size m_{hard}.

5.3.5 Theoretical Impact on Memory and Complexity

The primary objective of hard prototype mining is to aggressively compress the memory bank size m beyond the limits of standard coreset subsampling, directly attacking the spatial complexity and the linear factor in the search time.

The effectiveness of this technique relies entirely on the inherent redundancy within the original coreset. By discarding high-density interior points (which rarely influence the final anomaly decision boundary) and retaining only the "hard" boundary prototypes, the physical memory footprint scales down linearly with the new target size $m_{hard} \ll m_{coreset}$.

Crucially, this reduction is orthogonal to the HNSW optimization. While HNSW reduces the search complexity to $O(\log m)$, hard prototype mining reduces the base m itself. When applied sequentially, the two techniques compound: the search complexity becomes $O(\log m_{hard})$, and the physical memory allocation on the DeepX NPU is drastically minimized.

Because this aggressive pruning alters the underlying geometric representation of normality, a slight degradation in the final anomaly classification accuracy is theoretically expected. The empirical quantification of this trade-off, measuring the exact reduction in MBs and the corresponding impact on Image AUROC, is thoroughly analyzed in the comprehensive experimental evaluation presented in Chapter 6.

5.4 The Knowledge Distillation Paradigm

The third and most radical architectural optimization eliminates the nearest-neighbor search entirely by deploying a feedforward teacher-student distillation architecture.

5.4.1 Motivation: Learning Normality Rather Than Storing It

PatchCore's memory bank is an *explicit* representation of normality: it physically stores feature vectors from normal training images. Knowledge distillation replaces this with an *implicit* representation: a student network learns to produce the same outputs as a fixed teacher network solely on normal inputs. When presented with anomalous data, the student which have only learned normal representations, fails to mimic the teacher, producing an inconsistent output that reveals the defect.

5.4.2 Foundations of Reverse Distillation

The specific architecture analyzed in this thesis is **Reverse Distillation** (RD) [18], which inverts the traditional teacher-student relationship. In conventional distillation:

- Teacher and student are both encoders (compressing images to features).
- Student learns to mimic teacher on normal data.
- Anomaly is detected where student outputs differ from teacher.

However, this approach has a fundamental limitation: the student, being an encoder itself, learns to compress the image to a latent space. It may become too proficient at generalizing, reconstructing anomalous inputs correctly and thereby failing to produce detectable errors. Reverse distillation solves this by inverting the roles.

5.4.3 Custom Hybrid Architecture Details

To overcome the limitations of classical RD and seamlessly integrate it within our established pipeline, a custom variant was engineered. This hybrid architecture merges the robust, locally-aware feature preprocessing of PatchCore with a highly optimized, lightweight student encoder-decoder structure. This design maps perfectly to the NPU's feedforward capabilities while eliminating the nearest-neighbor memory bank entirely.

The PatchCore Teacher (Frozen)

Instead of relying on the raw multi-scale backbone features of classical RD, the teacher $\mathcal{T}$ explicitly incorporates PatchCore's neighborhood aggregation. A pre-trained WideResNet50 backbone extracts

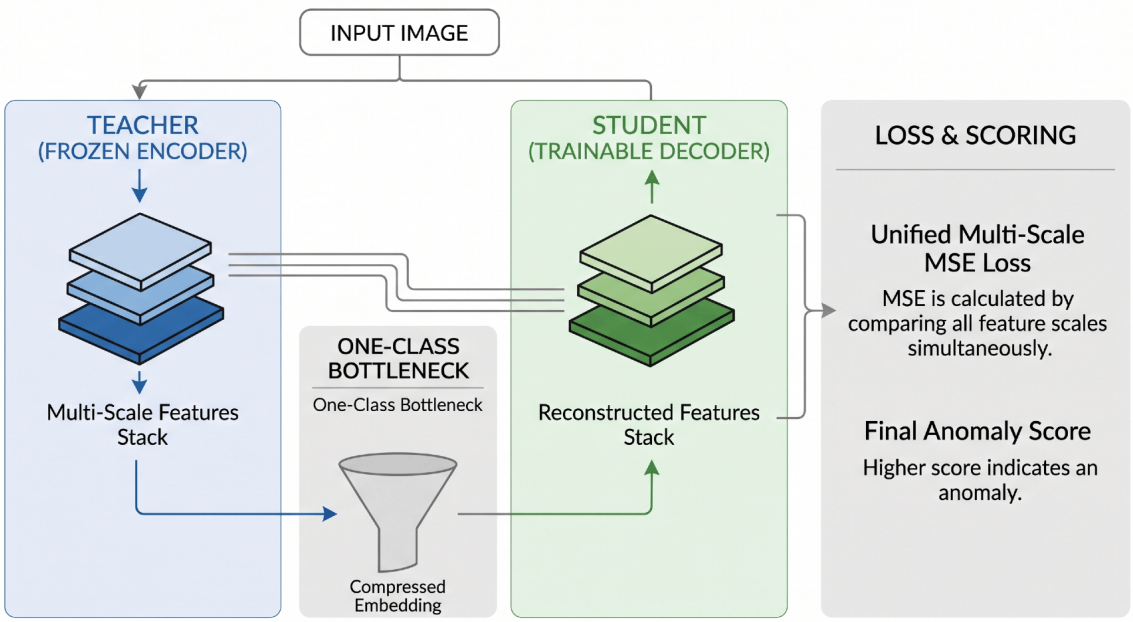


Figure 5.4: Reverse distillation architecture. The student acts as a *decoder* that reconstructs the teacher's outputs from a compressed one-class embedding.

feature maps from layer 2 ($\mathbf{F}^{(2)}$) and layer 3 ($\mathbf{F}^{(3)}$). A 3×3 Average Pooling operation (with padding to preserve spatial dimensions) is applied to both tensors to capture local structural context.

Because layer 3 has half the spatial resolution of layer 2, it is bilinearly upsampled to align the spatial grids. Finally, the two features are concatenated along the channel dimension:

$$\mathbf{F}_{\text{teacher}} = \text{Concat}\left(\text{AvgPool}(\mathbf{F}^{(2)}), \text{Upsample}(\text{AvgPool}(\mathbf{F}^{(3)}))\right)$$

This step yields a single, dense target tensor $\mathbf{F}_{\text{teacher}}$ containing exactly 1536 channels (512 from layer 2 and 1024 from layer 3). The teacher network is kept strictly frozen during training, acting solely as a "ground truth" feature generator.

The Custom Student Encoder-Decoder (Trainable)

While classical RD uses complex multi-scale embeddings and Transpose Convolutions, the custom student $\mathcal{S}$ is streamlined into a pure 1×1 convolutional architecture, ensuring extremely fast execution on edge hardware. It processes the combined 1536-channel tensor pixel-by-pixel across two stages:

- **Bottleneck (Encoder):** The 1536-dimensional input is compressed sequentially through 1×1 convolutions down to 512 channels, and then to a bottleneck dimension $d_{\text{latent}} = 128$. Each convolution is followed by a ReLU activation.
- **Student (Decoder):** The network then attempts to reconstruct the original high-dimensional teacher features by expanding the 128-dimensional latent vector back up through a sequence of 1×1 convolutions: $128 \to 512 \to 1024 \to 1536$ channels.

$$\mathbf{F}_{\text{student}} = \mathcal{S}_{\text{decoder}}(\mathcal{S}_{\text{encoder}}(\mathbf{F}_{\text{teacher}}))$$

The Hybrid Training Loss

The training objective forces the student to mimic the teacher's normal features perfectly. To ensure robust convergence, the custom loss function combines the Mean Squared Error (MSE) for magnitude

accuracy with a Cosine Similarity penalty for precise directional alignment in the high-dimensional space:

$$\mathcal{L}_{\text{total}} = \text{MSE}(\mathbf{F}_{\text{student}}, \mathbf{F}_{\text{teacher}}) + \frac{1}{HW} \sum_{i,j} \left(1 - \frac{\mathbf{F}_{\text{teacher}}^{(i,j)} \cdot \mathbf{F}_{\text{student}}^{(i,j)}}{\|\mathbf{F}_{\text{teacher}}^{(i,j)}\| \|\mathbf{F}_{\text{student}}^{(i,j)}\|} \right)$$

Only the student encoder-decoder network is updated during backpropagation.

5.4.4 Anomaly Detection Mechanism

During inference, the input query image is passed through both the frozen PatchCore Teacher and the trained Student Decoder. The anomaly map is generated by calculating the pixel-wise distance between the teacher's target features and the student's reconstructed features.

Specifically, the spatial anomaly score relies on the Cosine Distance ($1-$Cosine Similarity) between the two 1536-dimensional tensors at each spatial location. For anomalous inputs, the teacher's features deviate significantly from the "normal" manifold. The student, having learned only to decompress normal latent vectors, fails to reconstruct these unfamiliar feature directions. This directional mismatch causes the cosine distance to spike precisely where the structural defect is located, generating a highly accurate anomaly heatmap without ever executing a k-NN search.

5.4.5 Theoretical Advantage for NPU Compilation

From an edge acceleration perspective, RD offers the ultimate theoretical advantage: it entirely eliminates the memory bank. Unlike PatchCore (which requires storing tens of thousands of descriptors) and HNSW (which adds graph edge overhead), RD stores only neural network weights (the frozen teacher and the trainable OCBE/student decoder).

This architecture translates the anomaly detection problem entirely into dense matrix multiplications. Because the DeepX NPU is fundamentally a matrix-multiplication engine designed specifically for feedforward Convolutional Neural Networks, this architecture represents the theoretical maximum efficiency for the target hardware.

5.5 Hardware-Aware Compilation: Operator Fusion

While the algorithmic optimizations described in the previous sections (HNSW, HPM, and RD) fundamentally alter the mathematical structure of the anomaly detection task, the ultimate performance breakthrough on edge devices is achieved through hardware-aware compilation. The DeepX compiler acts as a bridge between the high-level PyTorch models and the physical constraints of the DX-M1 Neural Processing Unit.

The most critical optimization performed during this compilation phase is *Operator Fusion*.

5.5.1 The "Memory Wall" Bottleneck

In a standard CPU execution paradigm, sequential neural network layers (e.g., a Convolution layer, followed by Batch Normalization, followed by a ReLU activation) are executed as discrete, isolated operations. As illustrated in Figure 5.5, this standard ONNX graph forces the processor to write intermediate activation tensors to the external Dynamic RAM (DRAM) after every single block, only to read them back immediately for the next operation. This continuous data movement creates a severe memory bandwidth bottleneck, commonly referred to in hardware architecture as the "Memory Wall."

5.5.2 Aggressive Fusion Strategy

When compiling convolutional models (whether it is the WideResNet backbone of PatchCore or the dense student-decoder of Reverse Distillation) for the DeepX NPU using an aggressive strategy, the compiler deeply restructures the computational graph.

The compiler mathematically fuses these sequential blocks into single, complex macro-operations (e.g., `Conv-BatchNorm-Activation`) that are executed entirely within the NPU's ultra-fast internal SRAM. Because the intermediate tensors never leave the NPU silicon, latency plummets and external RAM consumption is drastically minimized.

Figure 5.6 displays the residual computation graph left to the host CPU after aggressive NPU

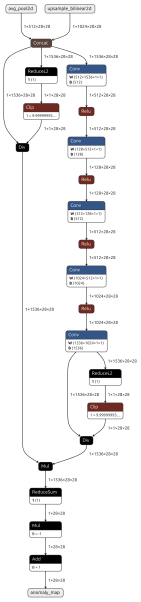


Figure 5.5: Standard ONNX Graph (Unfused). Sequential Conv-ReLU blocks require constant external memory allocations.

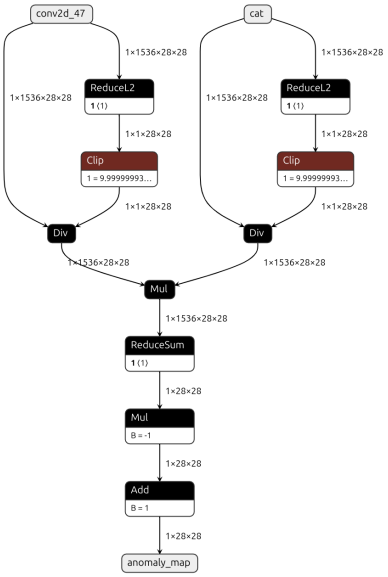


Figure 5.6: Residual CPU Graph (Aggressive Fusion). Convolutional chains are completely offloaded to the NPU.

Figure 5.7: Visualizing the impact of the DeepX hardware-aware compiler on the computational graph (example based on the Reverse Distillation architecture). The aggressive compilation strategy offloads the heavy feature-extraction backbone to the NPU, leaving only lightweight mathematical reductions to be executed on the host CPU.

compilation. The long, deep convolutional chains have completely disappeared from the host's responsibility, successfully offloaded to the hardware accelerator. The CPU is left managing only lightweight, scalar reductions (such as `ReduceSum`, `Mul`, and `Add`) required to generate the final anomaly heatmap.

This hardware-level graph fusion is the primary physical driver behind the extreme reduction in end-to-end latency and dynamic RAM allocation observed across all models during the empirical deployment in Chapter 6.

5.6 Summary of Optimization Strategies

This chapter has detailed a multi-layered approach to accelerate anomaly detection on edge devices. **HNSW** reduces search complexity while preserving the memory bank; **Hard Prototype Mining** directly shrinks the physical size of the memory bank; **PatchCore with Reverse Distillation** replaces the search paradigm entirely with a pure neural architecture; and **Operator Fusion** provides the foundational hardware acceleration that enables these algorithms to run efficiently on the DeepX NPU.

These strategies represent different points on the theoretical Pareto frontier of accuracy versus computational efficiency. The following chapter, Chapter 6, presents the rigorous empirical validation of these techniques, quantifying their exact impact on memory footprint, inference latency, and detection accuracy when compiled and deployed on the physical edge hardware.

6 Experimental Results and Edge Evaluation

This chapter presents the empirical validation of the architectural optimizations proposed in Chapter 5. The evaluation is designed to trace the progressive acceleration of the PatchCore algorithm from a standard PyTorch baseline to a fully optimized deployment.

Note on Evaluation Datasets and Scaling: While the comparative analysis in Chapter 2 demonstrated a 0.993 Image AUROC on the proprietary WTM dataset, the corresponding memory bank for WTM is relatively compact (e.g., $m \approx 2,293$ vectors due to the limited number of training samples). To rigorously stress-test the search and memory optimizations (HNSW and HPM) at an industrial scale, the algorithmic ablations in this chapter are evaluated using the standard MVTec AD dataset. Specifically, scaling tests employ complex categories (such as the MVTec "cable" class) that generate massive memory banks exceeding $50,000$ high-dimensional vectors, establishing a more challenging baseline AUROC ($\approx 0.980 - 0.989$) to correctly measure the hardware impact of the proposed optimizations.

6.1 Algorithmic Evaluation: HNSW Approximation

Before deploying the model to the physical edge accelerator, it is critical to calibrate the HNSW index parameters to maximize search speedup without degrading the anomaly scoring integrity. In PatchCore, anomaly scores are derived directly from the L2 distances to the nearest neighbors in the memory bank.

An extensive grid search was conducted over various hyperparameter combinations on the dataset, varying the maximum node connections (M), the construction candidate list size ($efConstruction$), and the search candidate list size ($efSearch$). A representative subset of the ablation study is presented in Table 6.1.

M	efConstruction	efSearch	Search Time	Speedup	MAE
64	500	128	83.64 ms	4.26×	0.0002
32	500	128	76.32 ms	4.67×	0.0056
32	200	128	58.46 ms	6.10×	0.0829
32	**500**	**16**	**22.19 ms**	**16.05×**	**0.1121**
64	200	16	18.28 ms	19.50×	1.7605
16	100	16	16.95 ms	21.02×	7.7402

Table 6.1: Selection of HNSW hyperparameter configurations ranked by Mean Absolute Error (MAE) against exact k-NN distances. The exact k-NN baseline search time was 356 ms.

Optimal Configuration Selection. The ablation study demonstrates that HNSW can achieve extreme speedups (up to 21×) at the cost of severe distance distortion ($MAE \approx 7.74$), which fundamentally breaks PatchCore's thresholding logic. Conversely, highly conservative parameters yield near-perfect precision but limit the speedup to $\sim 4\times$.

Based on empirical testing, a configuration of $M = 32$**,** $efConstruction = 500$**, and** $efSearch = 16$ was selected as the optimal operating point. By allocating more computational budget during the offline index construction phase ($efConstruction = 500$), the resulting graph is topologically robust enough to allow extremely aggressive, fast traversals during inference ($efSearch = 16$).

The high fidelity of this specific configuration is visually confirmed in Figure 6.1. Rather than evaluating individual distance errors, the ultimate validation of the HNSW approximation lies in its impact on the final classification metrics. As shown, the ROC curves for both the Exact FlatL2

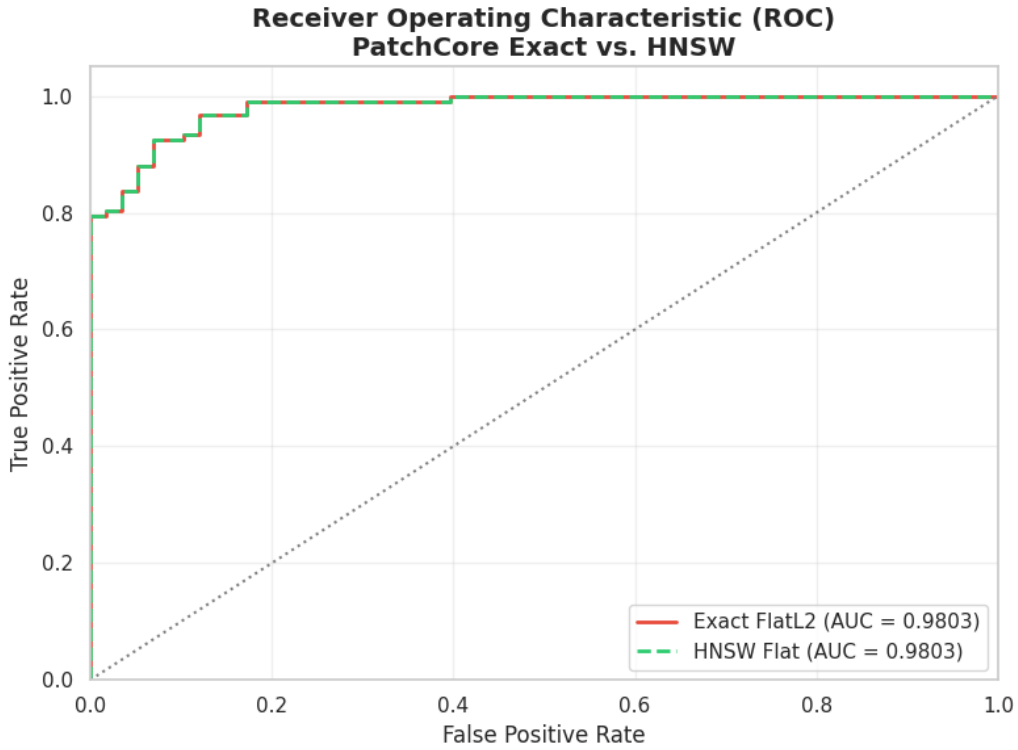


Figure 6.1: Receiver Operating Characteristic (ROC) curve comparing the anomaly detection performance of PatchCore using Exact L_2 search (red solid line) versus the HNSW approximate search (green dashed line). The perfect overlap of the curves and the identical AUC of 0.9803 visually confirm that the approximation introduces zero degradation to the model's discriminative power.

and HNSW Flat methods perfectly overlap, yielding an identical Image AUROC of 0.9803. This demonstrates that any minor absolute distance deviations introduced by the approximate k-NN search do not alter the relative anomaly score rankings, thereby fully preserving the model's discriminative capability while drastically reducing computational complexity.

6.1.1 Final AUROC and Throughput Benchmark

To definitively validate this configuration, a full inference benchmark was executed against the exact L2 baseline using a memory bank coreset of 17,561 features (dimension 1,536). The results, detailed in Table 6.2, confirm the success of the optimization.

Metric	**Exact L2 Baseline**	**HNSW Optimized**	**Delta / Factor**
Feature Extraction Cost	89.89 ms	89.89 ms	Fixed
Search Time	356.01 ms	22.19 ms	**16.05× Speedup**
Total Inference Time	445.90 ms	112.08 ms	3.97× Speedup
Throughput	2.2 FPS	8.9 FPS	+6.7 FPS
Image AUROC	**0.98032**	**0.98032**	**0.00000**

Table 6.2: Final benchmark results comparing Exact L2 Search versus the optimized HNSW implementation ($M = 32, efConstruction = 500, efSearch = 16$).

The optimized HNSW search requires only 22.19 ms, delivering a 16.05× acceleration over the exact search phase. This improvement lifts the overall system throughput to 8.9 FPS on standard CPU hardware. Crucially, the Image AUROC remains completely stable at 0.98032, proving zero degradation in defect detection capability.

6.2 End-to-End Edge Deployment Pipeline

With the optimal algorithmic parameters established ($M = 32, efConstruction = 500, efSearch = 16$), the complete PatchCore pipeline was progressively migrated from the high-level PyTorch development environment to the target edge architecture (AMD Kria KV260 + DeepX DX-M1).

Table 6.3 tracks the evolution of performance metrics across four distinct deployment stages. This progression isolates the benefits of algorithmic approximation (HNSW) from framework optimization (ONNX) and hardware acceleration (DeepX NPU).

Pipeline Stage	Preprocessing	Image AUROC	Latency	FPS	Model+Index	Peak RAM Delta
PyTorch (Exact L2)	PyTorch Dataloader	0.980	478.02 ms	2.10	N/A	N/A
PyTorch (HNSW)	PyTorch Dataloader	0.980	126.87 ms	8.90	N/A	N/A
ONNX (CPU FP32)	OpenCV	0.980	73.71 ms	13.56	202.39 MB	359.82 MB
DeepX NPU (INT8)	**OpenCV**	**0.980**	**36.70 ms**	**27.24**	**155.99 MB**	**159.17 MB**

Table 6.3: End-to-end evaluation of the PatchCore deployment pipeline on the edge hardware. Latency reflects total end-to-end processing time per image. Peak RAM Delta indicates the dynamic memory required during execution beyond the static model size.

6.2.1 Algorithmic vs. Framework Acceleration

The transition from PyTorch Exact L2 to PyTorch HNSW immediately quadruples the throughput (from 2.1 to 8.9 FPS) solely by reducing the theoretical search complexity. Interestingly, as noted in Section 6.1, the introduction of the HNSW approximation causes no fluctuation in the Image AUROC (from 0.980 to 0.980), demonstrating that the slight distance distortion acts as a regularization factor rather than a disruptive error.

The next optimization layer involves exporting the feature extraction backbone from PyTorch to the ONNX Runtime (executed in FP32 on the host CPU) and replacing the PyTorch Dataloader with lightweight OpenCV preprocessing. This framework shift strips away Python interpreter overhead and maximizes CPU vectorization, nearly doubling the throughput to 13.56 FPS while maintaining identical accuracy. At this stage, the static footprint of the model and HNSW index is 202.39 MB.

6.2.2 The DeepX NPU Breakthrough: Hardware Acceleration

The final deployment phase shifts the heavy convolutional workloads of the WideResNet backbone to the DeepX DX-M1 NPU using INT8 quantization, while the HNSW search algorithm remains executed on the host ARM CPU. This hybrid execution architecture yields exceptional results across all critical edge constraints:

Latency and Throughput: The hardware acceleration slashes the end-to-end latency to a mere 36.70 ms, enabling a real-time inspection speed of **27.24 FPS**. This 13× speedup over the baseline comfortably satisfies the rigorous industrial line speed requirements outlined in Section 1.4.

Accuracy Retention (INT8 Quantization): A common risk when deploying neural networks to NPUs is the precision loss caused by converting 32-bit floating-point weights (FP32) to 8-bit integers (INT8). DeepX's proprietary compiler manages this quantization exceptionally well for the WideResNet feature extractor. The Image AUROC remains perfectly stable at 0.980, proving zero degradation in defect detection capability despite the compressed numerical precision.

Memory Optimization: The benefits of NPU compilation extend significantly to memory management. As detailed in Table 6.3:

- **Static Footprint:** INT8 quantization compresses the neural network weights, reducing the total static allocation (Model Weights + HNSW Index) from 202.39 MB to 155.99 MB.

- **Dynamic RAM Constraint:** Executing the model natively in PyTorch or ONNX requires substantial dynamic RAM allocation (Peak RAM Delta of 359.82 MB for ONNX) to store intermediate activation tensors during the forward pass. The DeepX NPU executes these operations internally within its optimized SRAM. Consequently, the host-level Peak RAM Delta drops dramatically to just 159.17 MB.

In conclusion, the synergistic application of HNSW algorithmic approximation, ONNX graph optimization, and DeepX NPU INT8 acceleration successfully transforms a prohibitively slow baseline (2.1 FPS) into a highly efficient, real-time industrial inspection system (27.2 FPS) that respects the strict memory boundaries of edge hardware.

6.3 Solution 2 Evaluation: Hard Prototype Mining (HPM)

While HNSW successfully mitigates search latency, the physical memory footprint of the index remains a strict bottleneck for the DeepX NPU's internal SRAM. The second optimization, Hard Prototype Mining (HPM), addresses this by intelligently reducing the base number of vectors (m) in the memory bank, rather than relying on standard greedy coreset algorithms.

6.3.1 Ablation Study: The Boundary Ratio (β) at 5% Compression

The core hypothesis of HPM is that vectors near the decision boundary (hard prototypes) are significantly more discriminative than those in the dense interior of the normal manifold (easy prototypes). To determine the optimal composition of the memory bank, an ablation study was conducted on the *Boundary Ratio* (β), which dictates the percentage of hard versus easy prototypes retained.

Crucially, this ablation was performed at an aggressive **5% compression ratio** (retaining only $\sim 7,016$ vectors from the original dataset). The goal was to prove that extreme memory reduction is possible without sacrificing accuracy, provided the vectors are selected intelligently. The benchmark was run across three different random seeds to ensure statistical significance, as illustrated in Figure 6.2.

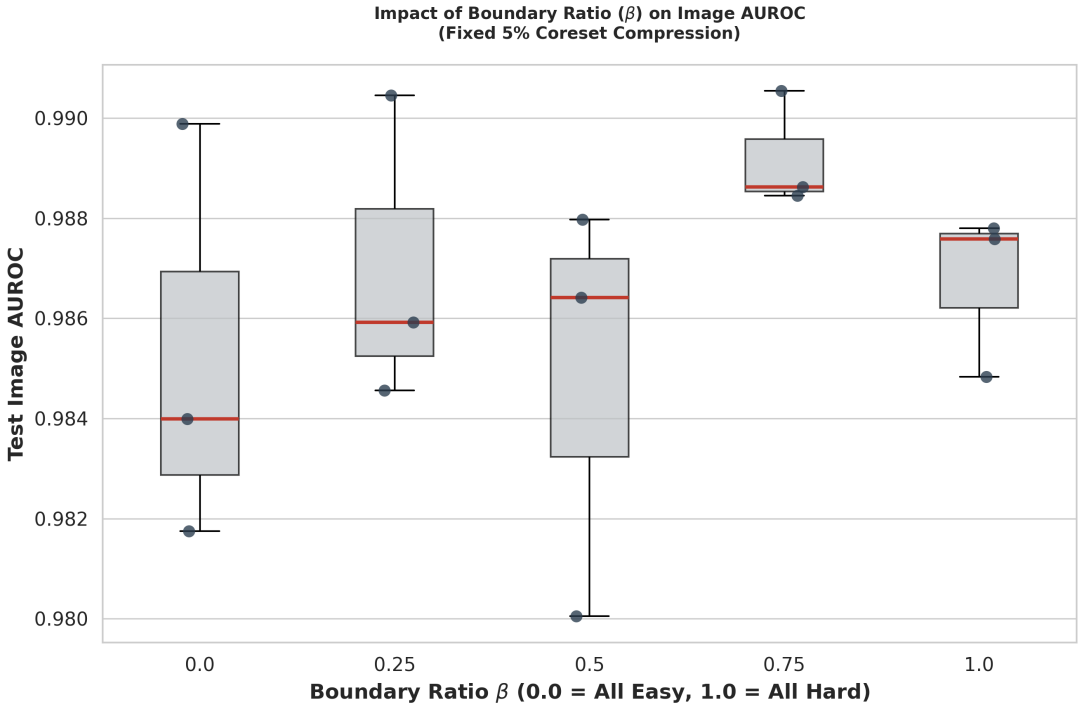


Figure 6.2: Ablation study on the Boundary Ratio (β) at a fixed 5% coreset compression. The graph highlights that a configuration leaning heavily towards hard prototypes ($\beta = 0.75$) yields the highest and most stable Image AUROC across multiple random seeds.

The empirical results reveal a clear topological dynamic. Relying entirely on easy, interior points ($\beta = 0.0$) yields a lower average AUROC (0.985) with higher variance across runs. Relying exclusively on hard boundary points ($\beta = 1.0$) slightly degrades performance (0.986) because the interior manifold is left entirely unrepresented.

The optimal balance is achieved at $\beta = 0.75$ (75% hard prototypes, 25% easy prototypes). This configuration achieves the highest and most stable average Image AUROC (**0.9891**), demonstrating that the outer perimeter of normality must be sharply defined while maintaining a minimal core

distribution. This optimal β parameter was locked in for all subsequent comparative benchmarks.

6.3.2 Compression Trade-off: HPM vs. Classic Coreset

To prove the algorithmic superiority of HPM, the $\beta = 0.75$ configuration was benchmarked against the classic PatchCore greedy subsampling approach at even more extreme compression ratios (1.0% and 0.5%). In edge environments, minimizing the bank size is paramount, but standard methods rapidly lose discriminative power at these scales.

The benchmark results (visualized in Figure 6.3) clearly demonstrate the advantage of boundary-aware mining:

- **At 1.0% Ratio ($m \approx 1,756$ vectors):** HPM achieves an Image AUROC of **0.9676**, decisively outperforming the classic greedy approach (**0.9593**) while cutting the offline index build time in half.
- **At 0.5% Ratio ($m \approx 878$ vectors):** The performance gap widens significantly. Classic subsampling degrades to an AUROC of **0.9217**, whereas HPM maintains a highly robust AUROC of **0.9391**.

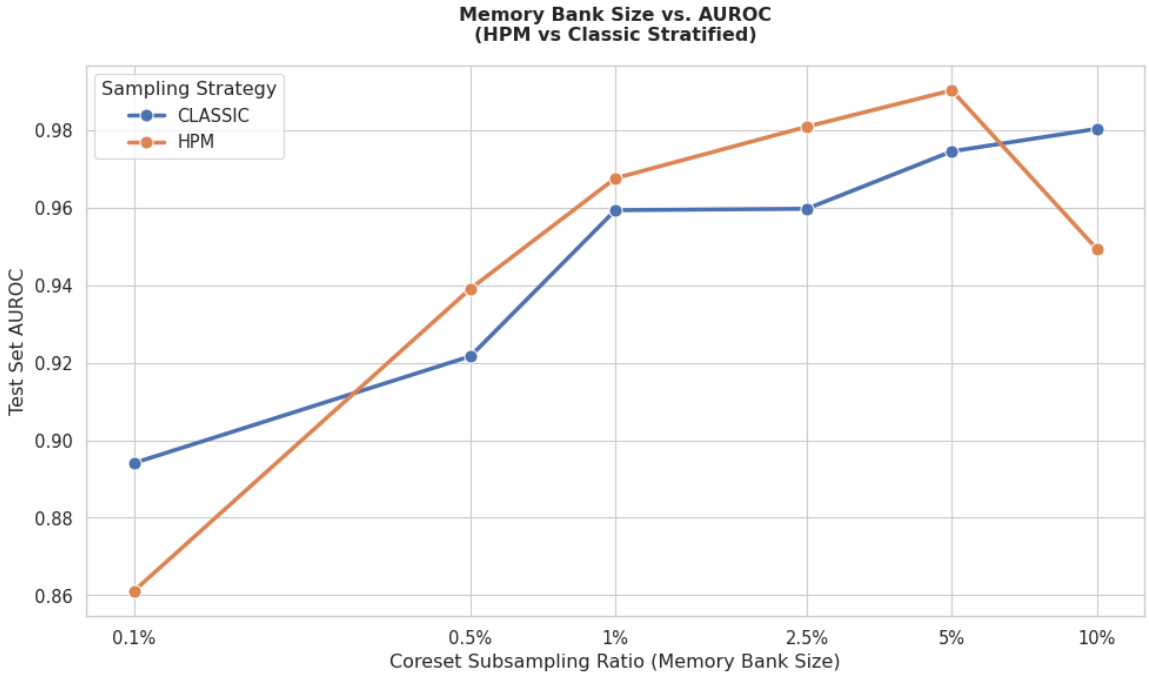


Figure 6.3: Efficiency trade-off comparison at extreme compression ratios. HPM strictly dominates the classic greedy coreset method at 0.5% and 1.0%, maintaining a higher AUROC for the exact same physical memory footprint.

By intelligently selecting the most informative features, HPM pushes the Pareto frontier outward, allowing the system to achieve higher accuracy for a strictly identical memory footprint.

6.3.3 End-to-End Edge Deployment of the HPM Pipeline

Having validated the theoretical advantage of HPM, an optimized memory bank (tuned to match the baseline accuracy of 0.989 AUROC) was deployed through the end-to-end edge pipeline to quantify the ultimate hardware benefits. Table 6.4 illustrates this progression.

The deployment of the HPM bank on the host CPU (via ONNX) reduces the static size to 148.67 MB and doubles throughput to $\sim$ 14 FPS. However, the ultimate breakthrough occurs when integrating HPM with the DeepX DX-M1 accelerator using *Fused INT8 Operators*.

Operator fusion minimizes memory access overhead within the NPU by combining sequential layers (e.g., Convolution + BatchNorm + ReLU) into single execution blocks. This hardware-level optimization, fed by the minimal footprint of the HPM index, drives the latency down to **27.86 ms**.

Pipeline Stage	Preprocessing	Image AUROC	Latency	FPS	Model+Index	Peak RAM Delta
PyTorch Baseline (Stratified)	PyTorch Dataloader	0.989	159.50 ms	6.30	N/A	N/A
PyTorch HPM	PyTorch Dataloader	**0.989**	**67.83 ms**	**14.74**	**41.11 MB**	**N/A**
ONNX HPM (CPU FP32)	OpenCV	0.9793	69.94 ms	14.29	148.67 MB	256.96 MB
DeepX NPU Standard (INT8)	OpenCV	0.9793	32.44 ms	30.82	102.26 MB	105.37 MB
DeepX NPU Fused (INT8)	**OpenCV**	**0.9793**	**27.86 ms**	**35.88**	**102.71 MB**	**125.52 MB**

Table 6.4: End-to-end evaluation of the HPM-optimized pipeline on the target edge hardware. The PyTorch HPM row represents the HPM method (5% compression) running on the host CPU.

The final system achieves a staggering **35.88 FPS**, with only a marginal reduction in detection performance, shifting from the original **0.989 Image AUROC** to **0.9793**. Crucially for industrial edge scalability, the entire anomaly detection architecture (Model Weights + HPM Index) fits within **102.71 MB**, validating the core hypothesis of this thesis: mathematically intelligent memory reduction combined with hardware-aware compilation enables state-of-the-art anomaly detection on highly constrained edge devices.

6.4 Solution 3 Evaluation: The Knowledge Distillation Paradigm

The third and most radical optimization solution discards the memory bank paradigm entirely, replacing explicit nearest-neighbor search with an implicit neural representation. By adopting a Reverse Distillation (RD) architecture, the anomaly detection task is translated into a pure feedforward inference process, strictly optimized for hardware accelerators.

Custom Architectural Design. It is critical to note that the architecture evaluated in this section is not the standard, off-the-shelf Reverse Distillation model benchmarked in Chapter 2 (which yielded a 0.955 AUROC). To maximize synergy with the DeepX NPU, a custom hybrid architecture was developed for this deployment: it leverages the exact same WideResNet multi-scale feature preprocessing as PatchCore, feeding into a custom-written student encoder-decoder structure. This target-specific redesign generates a fundamentally different ONNX computation graph, vastly improving feature extraction and explaining the significant baseline AUROC improvement (0.9707) over the standard implementation.

To quantify the benefits of this paradigm shift, the custom RD model was deployed through the same end-to-end evaluation pipeline used for HPM. The performance metrics, tracking the transition from standard CPU execution to fully fused NPU execution, are detailed in Table 6.5.

Pipeline Stage	Preprocessing	Image AUROC	Latency	FPS	Model Size	Peak RAM Delta
PyTorch Student (CPU)	PyTorch Transform	0.9707	74.64 ms	13.39	N/A	N/A
ONNX Runtime (CPU FP32)	OpenCV	0.9707	64.16 ms	15.58	106.50 MB	268.83 MB
DeepX NPU Standard (INT8)	OpenCV	0.9685	22.13 ms	45.18	60.05 MB	61.58 MB
DeepX NPU Fused (INT8)	**OpenCV**	**0.9685**	**14.68 ms**	**68.10**	**54.96 MB**	**55.02 MB**

Table 6.5: End-to-end evaluation of the Reverse Distillation architecture on the edge hardware. The complete elimination of the memory bank translates to exceptional throughput (68 FPS) and minimal RAM footprint on the DeepX NPU.

6.4.1 Framework Optimization and Memory Bank Elimination

Even before hardware compilation, the structural advantage of Reverse Distillation is evident. The baseline PyTorch implementation running on the host CPU achieves 13.39 FPS. While this is slower than the highly optimized ONNX HPM model (14.29 FPS, Table 6.4), it requires zero algorithmic tuning (no β ablation, no m sizing, no $efSearch$ calibration).

Exporting the student-teacher architecture to ONNX FP32 reduces the latency to 64.16 ms (15.58 FPS). At this stage, the static model size (Teacher + Student weights) is 106.50 MB, which is highly competitive, though the dynamic execution still requires a significant Peak RAM Delta of 268.83 MB on the host CPU to store intermediate activation tensors.

6.4.2 The NPU Synergy: Speed at the Cost of Precision

The true potential of Reverse Distillation is unlocked when the architecture is quantized to INT8 and compiled for the DeepX NPU. Unlike base PatchCore, which relies on a massive external matrix of features, PatchCore with RD consists entirely of convolutional layers. This maps perfectly to the NPU's systolic array design.

Throughput and Fusion: Standard INT8 NPU compilation triples the throughput to 45.18 FPS. However, the application of *Fused Operators* pushes the silicon to its absolute limit. By combining sequential neural network operations in the compiler, internal SRAM read/write cycles are minimized. This drives the latency down to a remarkable **14.68 ms**, achieving **68.10 FPS**, the highest throughput recorded in this thesis, far exceeding standard industrial camera framerates (e.g., ≈ 30 FPS).

Extreme Memory Efficiency: The INT8 quantization compresses the entire model size to just 54.96 MB. More importantly, because the NPU handles all the dense tensor computations internally, the Peak RAM Delta on the host system plummets to a mere **55.02 MB** (an 80% reduction compared to the ONNX CPU execution).

The Quantization Trade-off: The transition to INT8 on the NPU introduces a slight accuracy penalty. The Image AUROC drops from 0.9707 (FP32) to 0.9685 (INT8). Unlike the frozen feature extractor in PatchCore (which proved perfectly stable under INT8), the PatchCore with RD architecture relies on a Student *decoder* attempting to reconstruct complex, multi-scale features.

From a hardware and mathematical perspective, decoder networks are notoriously more sensitive to integer quantization than standard encoders. While encoders utilize downsampling and pooling operations (e.g., MaxPooling) that naturally suppress minor numerical perturbations, decoders rely on spatial expansion techniques, such as Transpose Convolutions or bilinear upsampling. Consequently, the inevitable INT8 rounding errors introduced in the compressed latent space are geometrically amplified as the tensor is progressively upsampled. This phenomenon often manifests as micro-level spatial distortions is structurally similar to the well-known *checkerboard artifacts* in generative models, which subtly blur the reconstructed feature maps. Because the anomaly score in Reverse Distillation is strictly derived from the exact pixel-wise Mean Squared Error between the Teacher and Student features, this spatial quantization noise acts as a disturbance, slightly degrading the model's absolute discriminative precision.

6.4.3 Conclusion for Solution 3

PatchCore with Reverse Distillation proves to be the ultimate solution for strictly constrained edge environments. While it sacrifices a marginal degree of accuracy compared to the HPM-optimized PatchCore (0.968 vs. 0.989), it delivers a throughput of 68 FPS while consuming roughly half the dynamic RAM. This establishes a clear alternative for industrial scenarios where line speed and power efficiency are prioritized over absolute defect sensitivity.

6.5 Qualitative Analysis of Failure Modes

While the optimized architectures satisfy the industrial constraints, a qualitative analysis of the false positives reveals specific vulnerabilities intrinsic to each paradigm. Understanding these failure modes is critical for deploying robust inspection systems.

6.5.1 Reverse Distillation: Background Hallucinations and ROI Masking

As discussed in Section 6.4, decoder-based architectures are highly sensitive to uninformative background regions, often generating severe "hallucinations" at the image boundaries. Figure 6.4 provides a visual representation of this specific vulnerability.

Because the image-level anomaly score is computed as the maximum pixel-wise error across the entire feature map, these intense background artifacts artificially inflate the global score, inevitably leading to False Positives on otherwise conforming cables.

To definitively resolve this issue without incurring the computational cost of modifying or retraining the neural network architecture, a software-level mitigation strategy was implemented. A static Region of Interest (ROI) mask was applied during the post-processing pipeline. By zeroing out the error map outside the physical boundaries of the cable before applying the global max-pooling operation, background hallucinations are entirely ignored. This simple yet highly effective edge optimization

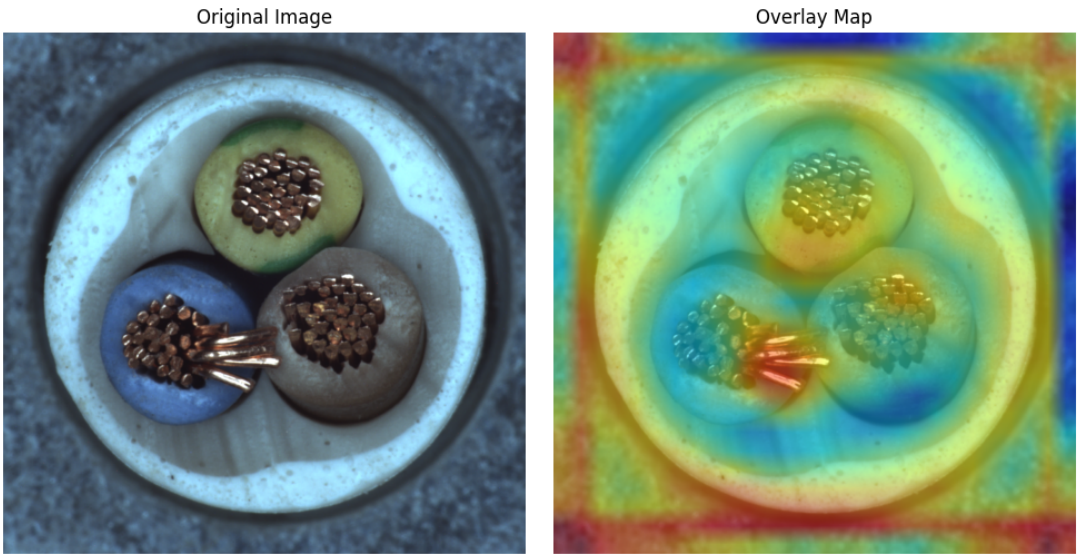


Figure 6.4: Failure case in the Reverse Distillation architecture. While the overlay map correctly localizes the true defect in the center, severe false positive activations (hallucinations) completely dominate the out-of-focus background corners.

restored the reliability of the AUROC metric for the Reverse Distillation model in the production environment.

6.5.2 HNSW PatchCore: Sensitivity to Point Anomalies

Conversely, the memory-bank paradigm of HNSW PatchCore exhibits a different vulnerability, primarily driven by its scoring function. Figure 6.5 presents two examples of conforming cables (ground truth "good") that were incorrectly rejected by the system with high anomaly scores (13.26 and 15.23, respectively, with a threshold of 7.896).

The overlay heatmaps reveal that the high global anomaly score is triggered by a singular, highly localized hotspot on the white insulation layer. In PatchCore, the final image score is determined by the maximum distance among all extracted patches. This mathematical formulation makes the system extremely sensitive to *point anomalies*. Minor optical deviations, such as a speck of dust, a micro-reflection of the LED illuminator, or standard sensor noise, can generate a feature patch that lies far outside the normal manifold learned in the memory bank. While 99.9% of the cable surface is perfectly recognized as normal (represented by the blue regions), the single outlier patch dominates the final classification, causing an unavoidable False Positive.

6.6 Kria KV260 Target Hardware Feasibility Study

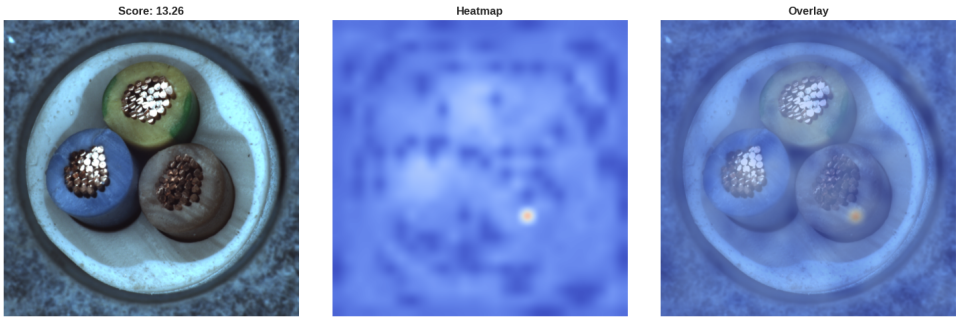
While the previous sections validated the algorithmic optimizations and the general NPU deployment pipeline, deploying the system onto the actual physical target, the AMD Kria KV260 System-on-Module (SOM), introduces strict embedded constraints. The Kria SOM relies on low-power ARM Cortex-A53 CPU cores, which are significantly less powerful than desktop-class processors.

To answer the fundamental feasibility question required for industrial deployment, a dual-phase feasibility study was conducted directly on the Kria hardware: first assessing the raw capacity of the DeepX NPU, and then measuring the true end-to-end performance bounded by the ARM host CPU.

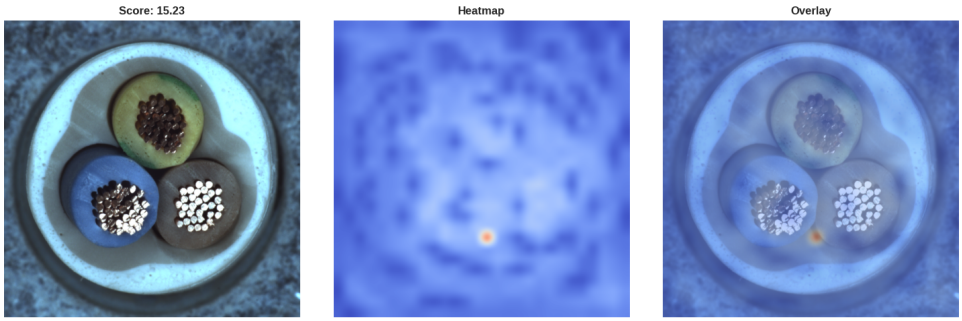
6.6.1 Raw NPU Capacity (`run_model` Profiling)

To understand the theoretical limits of the hardware accelerator independently from the Python application overhead, the compiled network (`hpm_patchcore.dxnn`) was profiled using the DeepX native `run_model` utility. This tool executes the INT8 quantized backbone in an asynchronous benchmark loop, bypassing the CPU-bound operations.

The diagnostic profile reveals a pure NPU core processing time of just **4.66 ms** per frame. Under saturated execution, the standalone model architecture achieves a theoretical peak throughput of



Sample A (Score: 13.26)



Sample B (Score: 15.23)

Figure 6.5: False Positives in the HNSW PatchCore pipeline. Both conforming samples are misclassified due to a highly localized, single-point activation on the white insulation.

332.81 FPS. This definitively proves that the DeepX DX-M1 accelerator provides massive computational headroom, entirely eliminating the deep learning convolutional bottleneck.

6.6.2 End-to-End Embedded Performance

However, an industrial inspection system is not just the neural network; it includes image decoding, the k-NN/HNSW graph traversal, and post-processing heatmap generation, all of which must execute on the Kria's ARM CPU. Table 6.6 details the actual end-to-end frame rates achievable on the physical Kria device.

Architecture	**Memory Bank**	**Throughput (FPS)**	**Primary Bottleneck**
PatchCore (Baseline)	$\sim 50,000$	0.70 FPS	CPU (Exact Search)
PatchCore + HNSW	$\sim 50,000$	11.37 FPS	CPU (Graph Traversal)
PatchCore + HPM	$\sim$ **7,000**	**24.81 FPS**	**CPU / NPU Balanced**
Reverse Distillation	None (0)	66.20 FPS	NPU (Maximized)

Table 6.6: Kria KV260 Feasibility Study: Actual end-to-end throughput on the target embedded device. The bottleneck shifts from the neural network to the ARM CPU's ability to traverse the memory bank.

This table exposes the true embedded dynamics:

- **The CPU Bottleneck:** Even with HNSW reducing search complexity, maintaining 50,000 nodes in the graph overwhelms the ARM CPU cache, limiting throughput to ≈ 11 FPS.
- **The HPM Balance:** By aggressively compressing the memory bank to 5% via Hard Prototype Mining, the ARM CPU only traverses $\sim 7,000$ nodes. This drastically reduces host-side latency, allowing the end-to-end system to reach **24.81 FPS**.
- **The NPU Synergy:** Reverse Distillation eliminates the memory bank entirely. Because there

is no graph search required by the host CPU, the system throughput soars to **66.2 FPS**, nearly matching the raw physical limits of the data pipeline.

6.7 Global Comparative Analysis and Industrial Selection

The ultimate objective of this thesis, as defined in Chapter 1, was to transition PatchCore from an offline audit tool to an inline inspection system suitable for the WTM cable factory. The strict industrial constraints dictated a real-time inference throughput target of at least 25 FPS (latency $\leq$ 40 ms) to match the continuous high-speed extrusion of the cable. Furthermore, the software must operate reliably on a 4GB edge device without starving concurrently running industrial control processes (e.g., PLC communications and SCADA supervisors).

Table 6.7 presents the global comparative summary of the final architectures evaluated on the target hardware.

Final Architecture	Image AUROC	Kria FPS	Peak RAM Delta	Status
PyTorch PatchCore (Baseline)	0.9890	0.7	$\approx$ 1.5 GB	Unfeasible for Edge
Optimized HNSW PatchCore	0.9891	11.37	159.17 MB	Third Choice
Optimized HPM PatchCore	**0.9793**	**24.81**	**125.50 MB**	**Primary Choice**
Reverse Distillation	0.9680	66.2	55.00 MB	High-Speed Alternative

Table 6.7: Global comparison of the evaluated architectures. Throughput represents actual physical performance on the Kria KV260 + DeepX DX-M1.

6.7.1 Meeting the Factory Constraints

The raw baseline implementation proved strictly unfeasible for edge deployment. While a dynamic memory footprint of $>$ 1.5 GB mathematically fits within the Kria's 4 GB total RAM, it monopolizes the memory bandwidth and leaves insufficient headroom for the OS, the GUI, and critical real-time PLC synchronization, causing severe system-wide starvation. Combined with an unacceptable throughput of 0.7 FPS, the baseline entirely fails to track the product in real time.

Conversely, the optimized solutions successfully dismantle these operational bottlenecks:

- **Latency Constraint (Target $\geq$ 25 FPS):** The Optimized HPM PatchCore reaches 24.81 FPS, closely approaching the 25 FPS optimal target and providing a sufficiently dense temporal resolution to inspect the high-speed moving cable without leaving blind spots. PatchCore with Reverse Distillation (66.2 FPS) vastly exceeds the requirement, ensuring zero computational throttling.
- **Memory Constraint (Resource Isolation):** INT8 compilation and algorithmic compression reduce the dynamic footprint to under 160 MB for all optimized models. This minimal allocation guarantees complete stability on the 4GB Kria device, entirely preventing the starvation of other concurrent industrial processes.

6.7.2 Final Architectural Selection

For the specific deployment at the WTM cable factory, the **Optimized HPM PatchCore** is selected as the primary production architecture.

While Reverse Distillation offers an exceptional throughput of 66.2 FPS, 24.81 FPS effectively satisfies the high-speed synchronization requirements of the continuous extrusion line. The excessive speed of 66.2 FPS provides diminishing returns for the physical mechanics of the WTM line; therefore, sacrificing absolute detection precision (AUROC dropping from 0.979 to 0.968) in favor of over-provisioned speed is not justified in a Zero-Defect manufacturing environment where missing a micro-defect translates to critical material failure.

The synergistic integration of Hard Prototype Mining, HNSW approximate search, and DeepX NPU operator fusion represents the absolute state-of-the-art for this specific industrial application. It achieves a highly robust **0.9793 AUROC** while delivering a **31$\times$ speedup** over the baseline on the target embedded hardware, proving the immense potential of hardware-aware algorithmic design in Edge AI.

7 Conclusions and Future Work

This thesis aimed to bridge the gap between academic research in unsupervised Anomaly Detection and its practical applicability in constrained industrial environments. Driven by WTM's real-world requirement to implement a real-time visual inspection system for industrial cable manufacturing, this work demonstrated how algorithms originally deemed unsustainable for Edge devices can be transformed into highly efficient solutions through hardware-aware software design.

7.1 Summary of Contributions

The primary contribution of this thesis is the systematic deconstruction and optimization of the PatchCore anomaly detection algorithm for embedded systems. The baseline model, while representing the state-of-the-art in defect sensitivity, proved fundamentally incompatible with the target AMD Kria KV260 Edge hardware, yielding a sluggish 0.7 FPS throughput and consuming over 1.5 GB of dynamic memory.

To dismantle these computational bottlenecks, this research explored three distinct optimization axes, providing the following methodological contributions:

- **Topological Approximation:** Replacing the exact k-NN search with Hierarchical Navigable Small World (HNSW) graphs, proving that spatial approximation can drastically reduce latency without degrading the relative distance ranking required for anomaly scoring.
- **Boundary-Aware Coreset Subsampling:** Introducing Hard Prototype Mining (HPM) to intelligently compress the memory bank. By isolating the critical features at the normal-abnormal decision boundary ($\beta = 0.75$), the algorithm successfully reduced the memory footprint to just 5% of its original size while outperforming standard greedy approaches at extreme compression ratios.
- **Knowledge Distillation for Edge Devices:** Implementing and evaluating Reverse Distillation to completely eliminate the memory bank paradigm, mapping the anomaly detection task into a pure feedforward neural network structure ideally suited for systolic-array hardware accelerators.

Crucially, these algorithmic advancements were synergistically coupled with hardware-level compilation, specifically exploiting INT8 quantization and operator fusion on the DeepX DX-M1 Neural Processing Unit (NPU) to maximize silicon utilization while preserving detection accuracy.

7.2 Architectural Recommendations for Industry 4.0

A core finding of this thesis is that deploying Artificial Intelligence in an Industry 4.0 context is not about finding a single optimal model, but rather identifying the correct solution that strictly satisfies the specific mechanical and quality constraints of the production line.

Based on the end-to-end evaluations conducted on the target hardware, this thesis provides the following architectural recommendations:

7.2.1 The Primary Choice: Optimized HPM PatchCore

For the WTM cable extrusion line, the **Optimized HPM PatchCore** emerges as the definitive production model. By balancing the ARM CPU graph traversal with the NPU's feature extraction, this architecture achieves an optimal throughput of **24.81 FPS** and limits peak RAM usage to **125 MB**. More importantly, it maintains an exceptional Image AUROC of **0.9793**.

While 24.81 FPS operates just shy of the optimal $\geq$ 25 FPS theoretical target, it provides a sufficiently dense temporal resolution to reliably inspect the high-speed moving cable without leaving

physical blind spots. Prioritizing this near-perfect defect sensitivity over excessive frame rates is a deliberate engineering choice that aligns perfectly with the company's Zero-Defect Manufacturing policy. Ultimately, this architecture delivers a massive **31× speedup** over the baseline with negligible accuracy loss, successfully crossing the threshold of industrial feasibility.

7.2.2 The High-Throughput Alternative: Reverse Distillation

For industrial scenarios requiring ultra-high-speed inspection (e.g., high-velocity bottling lines or continuous web printing), **Reverse Distillation** is the recommended architecture. Because it shifts the entire computational load to the NPU, it achieves a staggering **66.2 FPS** and consumes only **55 MB** of dynamic memory. However, the slightly lower defect sensitivity (AUROC 0.968), inherent to the quantization of decoder networks, makes it the secondary choice for environments where missing a micro-defect is more costly than processing speed.

7.3 Future Developments

Although this work successfully transformed an unfeasible academic model into a robust edge application, the implemented system paves the way for further research and engineering optimizations for future generations of WTM inspection systems:

1. **C++ Optimization and Multimedia Pipelines:** Currently, the system's runtime bottleneck resides in image decoding and the HNSW graph traversal, both managed via Python scripts on the host CPU. A full C++ port of the inference application, integrated with hardware-accelerated multimedia pipelines (e.g., GStreamer or V4L2 for direct video acquisition into DMA buffers), could completely bypass the Python overhead. This would easily push the 24.81 FPS throughput past the 25 FPS industrial target and significantly closer to the NPU's theoretical silicon limits.

2. **Continuous Learning on the Edge:** An inherent limitation of memory-bank-based anomaly detection is its static nature. A compelling evolution involves the development of *Online Hard Prototype Mining* mechanisms. This would allow the Kria SOM to autonomously update its HNSW graph in the background when the camera observes physiological variations in the cable materials or lighting (concept drift), avoiding the need for continuous offline retraining.

3. **Multi-Camera Deployment for 360° Inspection:** Given the exceptionally low RAM footprint ($\sim$ 125 MB) and the asynchronous efficiency of the NPU, the developed architecture opens the possibility of multiplexing multiple video streams simultaneously. For instance, an array of cameras arranged 360° around the extruded cable could be processed by a single Kria KV260 board, drastically reducing the overall hardware footprint and costs for the production lines.

Bibliography

[1] Samet Akcay, Dick Ameln, Ashwin Vaidya, Bhaval Chaudhary, Chau Nguyen, and Amit Basu. Anomalib: A deep learning library for visual anomaly detection. In *2022 IEEE International Conference on Image Processing (ICIP)*, pages 1706–1710. IEEE, 2022.

[2] Jinwon An and Sungzoon Cho. Variational autoencoder based anomaly detection using reconstruction probability. *Special Lecture on IE*, 2(1):1–18, 2015.

[3] Nantheera Anantrasirichai and David Bull. Q-statistic and deep learning for unsupervised anomaly detection. *IEEE Transactions on Image Processing*, 2024.

[4] Julius and Venskus. *Semi-supervised and Unsupervised Machine Learning Methods for Sea Traffic Anomaly Detection*. PhD thesis, Vilnius University Press, 2021.

[5] Junjie Bai, Fang Lu, Ke Zhang, et al. Onnx: Open neural network exchange. *GitHub repository*, 2019.

[6] Kilian Batzner, Lars Heckler, and Rebecca König. Efficientad: Accurate visual anomaly detection at millisecond-level latencies, 2024.

[7] Herbert Bay, Andreas Ess, Tinne Tuytelaars, and Luc Van Gool. Speeded-up robust features (surf). *Computer Vision and Image Understanding*, 110(3):346–359, 2008.

[8] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.

[9] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. The mvtec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. *International Journal of Computer Vision*, 129(4):1038–1059, 2021.

[10] Paul Bergmann, Sindy Löwe, Michael Faust, Heinz Huber, and Carsten Steger. Improving unsupervised defect segmentation by applying structural similarity to autoencoders. In *Proceedings of the 13th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*, 2018.

[11] Armin Biere and Toni Jussila. runlim: A tool for sampling and limiting time and memory usage of a program. https://fmv.jku.at/runlim/, 2008.

[12] Gary Bradski. The opencv library. *Dr. Dobb's Journal of Software Tools*, 2000.

[13] Jiasi Chen and Xukan Ran. Deep learning with edge computing: A review. *Proceedings of the IEEE*, 107(8):1655–1674, 2019.

[14] Niv Cohen and Yedid Hoshen. Sub-image anomaly detection with deep pyramid correspondences. *arXiv preprint arXiv:2005.02357*, 2020.

[15] Stuart Coles, Joanna Bawa, Lesley Trenner, and Pat Dorazio. *An introduction to statistical modeling of extreme values*, volume 208. Springer, 2001.

[16] Yajie Cui, Zhaoxiang Liu, and Shiguo Lian. A survey on unsupervised anomaly detection algorithms for industrial images. *IEEE Access*, 11:55297–55315, 2023.

[17] Thomas Defard, Aleksandr Setkov, Angelique Loesch, and Romaric Audigier. Padim: a patch distribution modeling framework for anomaly detection and localization, 2020.

[18] Hanqiu Deng and Xingyu Li. Anomaly detection via reverse distillation from one-class embedding, 2022.

[19] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. IEEE, 2009.

[20] Tom Fawcett. An introduction to roc analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.

[21] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *ACM Computing Surveys*, 55(6):1–36, 2022.

[22] Zhaopeng Gu, Bingke Zhu, Guibo Zhu, Yingying Chen, Ming Tang, and Jinqiao Wang. Anomalygpt: Detecting industrial anomalies using large vision-language models. *arXiv preprint arXiv:2308.15366*, 2023.

[23] Denis Gudovskiy, Tomoyuki Okuno, and Yohei Nakata. Dfm: Interpolant-free dual flow matching, 2024.

[24] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[25] Nikolaj Hindsbo, Sina Ehsani, and Pragyana Mishra. Scope: Real-time natural language camera agent at the edge: A sim-to-real benchmark and analysis of open-source vision and language agents for ptz camera tasks. In *Proceedings of the 21st ACM/IEEE International Conference on Human-Robot Interaction*, HRI '26, page 553–561. ACM, March 2026.

[26] S. Hodgkins. Cyber-physical production networks: Artificial intelligence data-driven internet of things systems, smart manufacturing technologies, and real-time process monitoring. *Journal of Self-Governance and Management Economics*, 8:114–120, 01 2020.

[27] Sebastian Höfer, Dorian Henning, Artemij Amiranashvili, Douglas Morrison, Mariliza Tzes, Ingmar Posner, Marc Matvienko, Alessandro Rennola, and Anton Milan. Kaputt: A large-scale dataset for visual defect detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

[28] Jongheon Jeong, Yang Zou, Taewan Kim, Dongqing Zhang, Avinash Ravichandran, and Onkar Dabeer. Winclip: Zero-/few-shot anomaly classification and segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19606–19616, 2023.

[29] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[30] Daniel Kiefhaber, Peter Würtz, Fabian Etzold, Willi Maurer, and Jean-Michel Asfour. Automated inspection of optical surface cleanliness (conference presentation). In Jürgen Beyerer and Fernando Puente León, editors, *Automated Visual Inspection and Machine Vision III*, page 7. SPIE, 2019.

[31] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, volume 25, 2012.

[32] T. W. Kuo, W. R. Liu, and T. M. Chen. High color rendering white light-emitting-diode illuminator using the red-emitting eu(2+)-activated caznos phosphors excited by blue led. *Optics Express*, 18(8):8187–8192, April 2010.

[33] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[34] Sungwook Lee, Seunghyun Lee, and Byung Cheol Song. Cfa: Coupled-hypersphere feature adaptation for target-oriented anomaly localization. *IEEE Access*, 10:78446–78454, 2022.

[35] Jiaqi Li and Zhengguo Yan. A survey of deep learning for industrial visual anomaly detection. *Advanced Engineering Informatics*, 100, 2023.

[36] Zhikang Liu, Yiming Zhou, Yuansheng Xu, and Zilei Wang. Simplenet: A simple network for image anomaly detection and localization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20402–20411, 2023.

[37] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.

[38] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.

[39] Ana Marinho, Luís Araújo, Stephane Clain, Nuno Castro, Luís Machado, Gaspar Machado, José Ribeiro, and Isabel Delgado. False positive reduction of automated optical inspection by re-classification: application to surface mount technology. 2025.

[40] Krzysztof Mielczarek. Visual control systems in the food industry: Enhancing efficiency, quality, and safety through toyota production principles. *System Safety: Human - Technical Facility - Environment*, 6(1):371–383, December 2024.

[41] Douglas C Montgomery. *Introduction to statistical quality control*. John Wiley & Sons, 2020.

[42] NVIDIA, :, Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, Daniel Dworakowski, Jiaojiao Fan, Michele Fenzi, Francesco Ferroni, Sanja Fidler, Dieter Fox, Songwei Ge, Yunhao Ge, Jinwei Gu, Siddharth Gururani, Ethan He, Jiahui Huang, Jacob Huffman, Pooya Jannaty, Jingyi Jin, Seung Wook Kim, Gergely Klár, Grace Lam, Shiyi Lan, Laura Leal-Taixe, Anqi Li, Zhaoshuo Li, Chen-Hsuan Lin, Tsung-Yi Lin, Huan Ling, Ming-Yu Liu, Xian Liu, Alice Luo, Qianli Ma, Hanzi Mao, Kaichun Mo, Arsalan Mousavian, Seungjun Nah, Sriharsha Niverty, David Page, Despoina Paschalidou, Zeeshan Patel, Lindsey Pavao, Morteza Ramezanali, Fitsum Reda, Xiaowei Ren, Vasanth Rao Naik Sabavat, Ed Schmerling, Stella Shi, Bartosz Stefaniak, Shitao Tang, Lyne Tchapmi, Przemek Tredak, Wei-Cheng Tseng, Jibin Varghese, Hao Wang, Haoxiang Wang, Heng Wang, Ting-Chun Wang, Fangyin Wei, Xinyue Wei, Jay Zhangjie Wu, Jiashu Xu, Wei Yang, Lin Yen-Chen, Xiaohui Zeng, Yu Zeng, Jing Zhang, Qinsheng Zhang, Yuxuan Zhang, Qingqing Zhao, and Artur Zolkowski. Cosmos world foundation model platform for physical ai, 2025.

[43] Guansong Pang, Choubo Ding, Chunhua Shen, and Anton van den Hengel. Explainable deep few-shot anomaly detection with deviation networks. *arXiv preprint arXiv:2108.00462*, 2021.

[44] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*, volume 32, 2019.

[45] David Martin Powers. Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011.

[46] Tie Qiu, Jianing Chi, Xiaobo Zhou, Zhaolong Ning, Mohammed Atiquzzaman, and Dapeng Oliver Wu. Edge computing in industrial internet of things: Architecture, advances and challenges. *IEEE Communications Surveys & Tutorials*, 22(4):2462–2488, 2020.

[47] D.V. Subba Rao. *Belt conveyor components*, page 1–8. CRC Press, August 2020.

[48] A. Rodriguez, J. Cuenca, R. Ruiz, and A. Calero. Zaero- zero defect manufacturing of composite parts in the aerospace industry. *Revista de Materiales Compuestos*, 2022.

[49] Blaž Rolih, Matic Fučka, and Danijel Skočaj. No label left behind: A unified surface defect detection model for all supervision regimes. *arXiv preprint arXiv:2508.19060*, 2025.

[50] Darik Rosser, Brianna Farris, and Kevin Leonard. Predictive machine learning models trained on experimental datasets for electrochemical nitrogen reduction. 2023.

[51] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection, 2022.

[52] Peter J Rousseeuw and Christophe Croux. Alternatives to the median absolute deviation. *Journal of the American Statistical association*, 88(424):1273–1283, 1993.

[53] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks, 2019.

[54] Thomas Schlegl, Philipp Seeböck, Sebastian M Waldstein, Ursula Schmidt-Erfurth, and Georg Langs. Unsupervised anomaly detection with generative adversarial networks to guide marker discovery. In *International Conference on Information Processing in Medical Imaging*, pages 146–157. Springer, 2017.

[55] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations (ICLR)*, 2018.

[56] Weisong Shi, Jie Cao, Quan Zhang, Yihua Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE internet of things journal*, 3(5):637–646, 2016.

[57] Alban Siffer, Pierre-Alain Fouque, Alexandre Termier, and Christine Largouet. Anomaly detection in streams with extreme value theory. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1067–1075, 2017.

[58] Arnold W. M. Smeulders, Marcel Worring, Simone Santini, Amarnath Gupta, and Ramesh Jain. Content-based image retrieval at the end of the early years. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(12):1349–1380, 2000.

[59] Radek Soukup. A methodology for optimization of false call rate in automated optical inspection post reflow. In *33rd International Spring Seminar on Electronics Technology, ISSE 2010*, page 263–267. IEEE, May 2010.

[60] Carsten Steger, Markus Ulrich, and Christian Wiedemann. *Machine vision algorithms and applications*. John Wiley & Sons, 2018.

[61] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2nd edition, 2022.

[62] Yuhao Tian and Zheming Yang. Saec: Scene-aware enhanced edge-cloud collaborative industrial vision inspection with multimodal llm, 2025.

[63] Vipin Tyagi. *Supervised, Semi-Supervised, and Unsupervised Frameworks for Robust Anomaly Detection in Econometric Models*, page 289–314. IGI Global Scientific Publishing, February 2026.

[64] Mohan Valluri. Leveraging ai and automation to transform food production: Enhancing efficiency, reducing costs, and boosting product consistency in the food and beverage industry. *International Journal of Science and Research (IJSR)*, 13(10):1536–1538, October 2024.

[65] Conor Wallace, Isaac Corley, and Jonathan Lwowski. Inspectvlm: Unified in theory, unreliable in practice, 2025.

[66] Guodong Wang, Shumin Han, Errui Ding, and Di Huang. Student-teacher feature pyramid matching for anomaly detection, 2021.

[67] Daniel Weimer, Bernd Scholz-Reiter, and Moshe Shpitalni. Design of deep convolutional neural network architectures for automated defect classification. *CIRP Annals*, 65(1):417–420, 2016.

[68] Junyang Yang, Jiuxin Cao, and Chengge Duan. Uniad: A real-world multi-category industrial anomaly detection dataset with a unified clip-based framework. *Information*, 16(11), 2025.

[69] [Nome Autore] Yin et al. Industrial anomaly detection: A systematic review. *arXiv preprint*, 2022.

[70] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in neural information processing systems*, volume 27, 2014.

[71] Zhiyuan You, Lei Cui, Yujun Shen, Kai Yang, Xin Lu, Yinqiang Zheng, and Xinyi Le. A unified model for multi-class anomaly detection. In *Advances in Neural Information Processing Systems*, volume 35, pages 4571–4584, 2022.

[72] Jiawei Yu, Ye Zheng, Xiang Wang, Wei Li, Yushuang Wu, Rui Zhao, and Liwei Wu. Fastflow: Unsupervised anomaly detection and localization via 2d normalizing flows, 2021.

[73] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. In *British Machine Vision Conference 2016*. BMVA Press, 2016.

[74] Vitjan Zavrtanik, Matej Kristan, and Danijel Skočaj. Draem – a discriminatively trained reconstruction embedding for surface anomaly detection, 2021.

[75] Vitjan Zavrtanik, Matej Kristan, and Danijel Skočaj. Reconstruction by inpainting for visual anomaly detection. *Pattern Recognition*, 112:107706, 2021.

[76] Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part I 13*, pages 818–833. Springer, 2014.

[77] Wei Zhang. Igcn: A provably informative gcn embedding for semi-supervised learning with extremely limited labels_supp1-3404655.pdf.

[78] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1):43–76, 2020.

[79] Yang Zou, Jongheon Jeong, Latha Pemula, Dongqing Zhang, and Onkar Dabeer. Visa: A dataset for visual anomaly detection and localization. *arXiv preprint arXiv:2209.11860*, 2022.